Appunti di Information Retrieval

Appunti del Corso

Anno Accademico 2026/2027

Indice

0 Introduzione e Architettura dell'Information Retrieval **1**
- 0.1 Definizione e Motivazioni . . . 1
- 0.2 I Due Macro-Obiettivi: Efficacia ed Efficienza . . . 3
- 0.3 Architettura di un Sistema di Information Retrieval . . . 3
- 0.4 Proprietà del Linguaggio e Modelli di Reperimento . . . 4
- 0.5 L'Inverted Index (Indice Invertito) . . . 5
- 0.6 Tecniche di Compressione dell'Indice . . . 6
- 0.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità 7
- 0.8 Evoluzione verso il Neural Information Retrieval . . . 7

1 Valutazione dei Sistemi di Information Retrieval **9**
- 1.1 Valutare un Sistema di Information Retrieval: Metodologia e Obiettivi . . . 9
- 1.2 Valutazione della Rilevanza e Metodologie di Annotazione . . 13
- 1.3 Evoluzione Storica e Iniziative Attuali: Il Framework TREC . 17
- 1.4 Metriche di Valutazione Binarie per Insiemi Non Ordinati . . . 20
- 1.5 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria) . . . 23
- 1.6 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG . . . 29

2 Efficienza Algoritmica sulle Architetture CPU Moderne **35**
- 2.1 Il Ruolo dell'Efficienza nell'Information Retrieval . . . 35
- 2.2 La Gerarchia di Memoria e i Principi di Località . . . 36
- 2.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing* 39
- 2.4 Parallel Computing e Forme di Parallelismo nella CPU . . . 41
- 2.5 Esecuzione Superscalare e Interleaving Aritmetico . . . 43
- 2.6 Pipeline Hazards: Tipologie e Risoluzione . . . 45
- 2.7 Branch Prediction ed Esecuzione Speculativa . . . 46
- 2.8 Vettorizzazione SIMD e la Risoluzione del Mistero di L1 Cache 49
- 2.9 Quadro Sinottico e Implicazioni per l'Information Retrieval . . 52

3 Proprietà del Linguaggio Naturale e Preprocessing del Testo 53
3.1 Proprietà Fondamentali del Linguaggio Naturale 53
3.2 Relazioni con Altri Tipi di Dati e Architettura nei Motori Web 59
3.3 Text Processing in Python: Dalla Rete ai Dati Puliti 60
3.4 Quadro Sinottico della Pipeline di Text Processing 64

4 Rappresentazioni del Testo, Modello Booleano ed Evoluzione del Retrieval 65
4.1 Operazioni Fondamentali su Collezioni di Testo 65
4.2 Rappresentazioni del Testo: Dal Testo alle Feature 66
4.3 Perché Serve una Rappresentazione Documentale: Verso il Vector Space Model 69
4.4 Il Modello Booleano Tradizionale (Boolean Retrieval) 70
4.5 Come Estendere il Modello Booleano: Verso il Ranked Retrieval 71
4.6 Quadro Sinottico Comparativo 73

5 Vector Space Model, Ponderazione TF-IDF e Similarità Coseno 74
5.1 Dall'Estensione del Booleano alla Matrice di Incidenza Binaria 74
5.2 Misure di Similarità Set-Based: Il Coefficiente di Jaccard . . . 75
5.3 Frequenza dei Termini e Rilevanza: Term Frequency e Matrice di Conteggio 76
5.4 Frequenza Documentale e Rarità: DF vs. CF e Inverse Document Frequency (IDF) 77
5.5 Lo Schema di Ponderazione TF-IDF Completo 79
5.6 Documenti e Query come Vettori N-dimensionali 80
5.7 Come Rispondere a una Query nello Spazio Vettoriale: Distanza L_2 (Pro e Contro) 80
5.8 Dagli Angoli alla Cosine Similarity e Normalizzazione L2 . . . 82
5.9 Esercizio Pratico Completo: Confronto Stilistico tra Tre Romanzi 83
5.10 Varianti di Ponderazione e la Notazione SMART 85
5.11 Domande di Verifica e Concetti Chiave 87
5.12 Mappa Concettuale Riassuntiva e Verso il Query Processing . 88

6 Architettura dei Motori di Ricerca e Indicizzazione 90
6.1 Architettura di Sistema e Paradigma a Due Stadi 90
6.2 Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval 93
6.3 Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito 94

6.4 Anatomia, Layout di Memoria e Compressione dell'Indice Invertito . . . 96
6.5 Tipologie di Query, Indici Posizionali e Riscrittura del Testo . 98

7 Query Processing: Algoritmi di Scansione ed Efficienza di Salto 101
7.1 Anatomia Statistica dell'Indice Inverso: Dizionario e Posting Lists . . . 101
7.2 Paradigmi di Scansione: TAAT vs. DAAT . . . 104
7.3 Elaborazione Term-at-a-Time (TAAT) in Dettaglio . . . 109
7.4 DAAT per Query Disgiuntive (OR) e Congiuntive (AND Base) 111
7.5 La Primitiva di Salto 'nextGEQ' e Strutture di Skipping . . . 113
7.6 Algoritmi di Intersezione Avanzati con 'nextGEQ' . . . 116
7.7 Laboratorio Pratico: Indice Invertito e Query Processing in Python su Dataset C4 . . . 120
7.8 Rilevanza nei Motori Vettoriali e Sistemi Moderni (Vector Search) 126
7.9 Quadro Sinottico Comparativo degli Algoritmi . . . 126

8 Elaborazione Efficiente di Query Top-k: WAND, MaxScore e Varianti a Blocchi 128
8.1 Architettura di Retrieval a Due Stadi e Motivazione del Top-k 128
8.2 Exact Top-k Retrieval e l'Algoritmo Esaustivo con Min-Heap . 130
8.3 L'Algoritmo WAND (Weak AND) e il Dynamic Pruning . . . 133
8.4 Analisi Critica e Casi Limite nel Dynamic Pruning . . . 140
8.5 L'Algoritmo MaxScore: Partizionamento in Liste Essenziali e Non-Essenziali . . . 142
8.6 Varianti a Blocchi: BMW e VBMW . . . 147
8.7 Sistemi Industriali: L'Ecosistema Apache Lucene ed Elasticsearch 151
8.8 Quadro Sinottico Comparativo degli Algoritmi Top-k . . . 153

9 Learning to Rank I: Fondamenti, Tassonomia e Framework Generale 154
9.1 Okapi BM25: Storia, Intuizioni e Term Frequency Smoothing . 154
9.2 Framework Generale di Ranking e Oltre il Bag of Words . . . 158
9.3 Il Paradigma del Machine Learning Applicato al Ranking . . . 161
9.4 Definizione Formale di Learning to Rank e Tassonomia delle Feature . . . 164
9.5 Classificazione per l'Ad-Hoc IR: Formulazione ed Esempio Geometrico . . . 167
9.6 La Tassonomia Classica di Learning to Rank: Pointwise, Pairwise e Listwise . . . 171

9.7 Quadro Sinottico e Considerazioni Conclusive 175

10 Learning to Rank II: Pipeline Industriale, BM25F e Formulazione del Ranking **177**
10.1 La Pipeline Completa di Learning to Rank e Vincoli Operativi 177
10.2 Classificatori per l'Ad-Hoc Search e Meccanica delle Loss LtR 180
10.3 Estensione a BM25F e Ottimizzazione dei Parametri 185
10.4 I Problemi Fondamentali dell'Apprendimento del Ranking . . 190

Bibliografia **192**

Capitolo 0

Introduzione e Architettura dell'Information Retrieval

0.1 Definizione e Motivazioni

L'**Information Retrieval** (IR, *Reperimento delle Informazioni*) è la disciplina scientifica e ingegneristica che studia la rappresentazione, la memorizzazione, l'organizzazione e l'accesso a collezioni di dati prevalentemente non strutturati o semi-strutturati, al fine di soddisfare un **bisogno informativo** (*information need*) espresso da un utente [1, 2].

Definizione 0.1.1 (Information Retrieval (Manning et al.))**.** L'Information Retrieval consiste nel reperire materiale di natura prevalentemente non strutturata (solitamente documenti testuali) che soddisfa un bisogno informativo all'interno di collezioni di grandi dimensioni memorizzate su calcolatori elettronici.

L'IR non si limita ai soli motori di ricerca Web generalisti (Google, Bing), ma costituisce l'infrastruttura abilitante di e-commerce e marketplace, enterprise/desktop search, sistemi **RAG** (*Retrieval-Augmented Generation*, che alimentano i Large Language Model con documenti esterni contestualmente rilevanti) e motori di raccomandazione.

0.1.1 Dai Motori Web all'Agentic Search

L'avvento dei modelli linguistici conversazionali (LLM) non ha reso obsoleto l'Information Retrieval, ma ne ha ampliato l'impatto: un LLM moderno si appoggia costantemente ai principi cardine dell'IR (comprensione semantica della query, matching e ranking, reperimento contestuale) per superare il

limite del *knowledge cutoff*. Nel paradigma **RAG**, il sistema intercetta la richiesta dell'utente, interroga un motore IR, inietta i documenti estratti nel contesto dell'LLM e genera una risposta supportata da citazioni verificabili.

Definizione 0.1.2 (Agentic Search)**.** L'**Agentic Search** è un processo di reperimento dinamico in cui un agente intelligente determina in modo autonomo quali ricerche successive condurre, basandosi sulle evidenze parziali raccolte: ispeziona le fonti, individua lacune conoscitive (*information gaps*), formula nuove query di raffinamento e reitera il ciclo fino alla convergenza informativa.

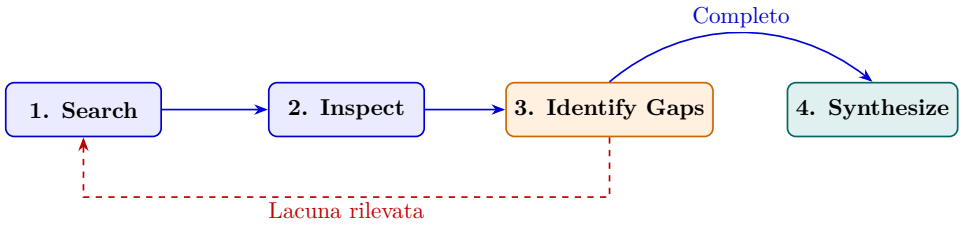


Figura 1: Ciclo a retroazione adattivo (*Adaptive Feedback Loop*) dell'Agentic Search.

0.1.2 Information Retrieval vs. Data Retrieval (Basi di Dati)

Dimensione	**Data Retrieval (Database SQL)**	**Information Retrieval**
Struttura dati	Rigidamente schematizzata (tabelle, tipi, vincoli)	Non strutturata o semi-strutturata (testo libero, HTML)
Linguaggio di query	Formale e deterministico (SQL)	Naturale o a parole chiave, spesso ambiguo
Matching	Booleano esatto	Parziale e probabilistico (gradi di rilevanza)
Ordinamento	Nessuno se non esplicito (`ORDER BY`)	Fondamentale, basato sulla stima di rilevanza
Tolleranza all'errore	Nulla	Elevata (refusi, sinonimia, polisemia)

Tabella 1: Confronto tra Data Retrieval tradizionale e Information Retrieval.

La Rilevanza: Il Cardine dell'IR

Mentre in una base di dati un record soddisfa o viola in modo booleano un predicato logico, nell'Information Retrieval il fulcro è la stima del grado di **rilevanza** rispetto al bisogno informativo dell'utente: soggettiva, situata nel contesto operativo ed evolutiva nel tempo.

0.2 I Due Macro-Obiettivi: Efficacia ed Efficienza

Un sistema di reperimento su larga scala persegue simultaneamente due obiettivi spesso in tensione:

1. **Efficacia (*Effectiveness*):** quanto accuratamente il ranking soddisfa il bisogno informativo dell'utente.
2. **Efficienza (*Efficiency*):** costo computazionale, throughput e latenza di risposta, tipicamente nell'ordine dei 10–50 millisecondi per query su archivi di miliardi di documenti.

L'efficienza di un motore di ricerca moderno dipende dall'interazione con l'architettura hardware sottostante: pipelining e processori superscalari, gerarchia di memoria e località di accesso, vettorizzazione SIMD e calcolo parallelo/distribuito su cluster (*sharding*).

0.3 Architettura di un Sistema di Information Retrieval

Un sistema IR adotta una separazione netta tra la **Pipeline Offline** (indicizzazione e addestramento dei modelli di ranking) e la **Pipeline Online** (interrogazione a bassa latenza), come illustrato in Figura 2.

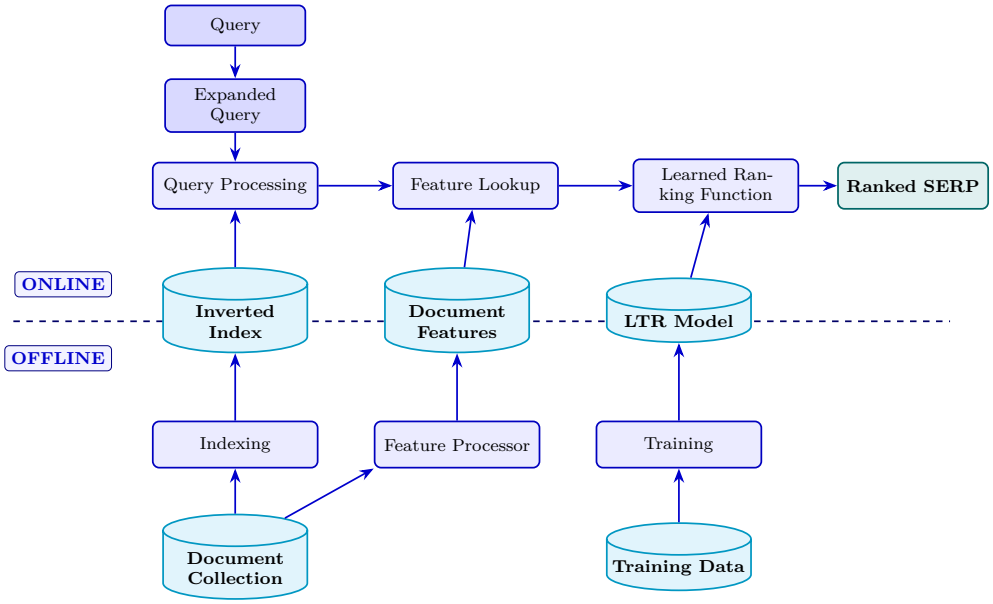


Figura 2: Architettura a griglia di un moderno motore di ricerca (pipeline Online vs. Offline).

0.3.1 Pipeline di Text Processing

Prima dell'indicizzazione, ogni flusso testuale attraversa: **tokenizzazione** (segmentazione in token lessicali), **normalizzazione/case folding** (minuscolo, diacritici), **filtraggio delle stop word** (rimozione di vocaboli ad alta frequenza ma privi di valore discriminante) e **stemming/lemmatizzazione** (riduzione di una forma flessa alla radice o al lemma canonico, es. `"searching"` $\rightarrow$ `"search"`).

0.4 Proprietà del Linguaggio e Modelli di Reperimento

- **Legge di Zipf:** la frequenza $f(r)$ di un termine è inversamente proporzionale al suo rango r nella graduatoria di frequenza, $f(r) \propto 1/r$: poche parole frequentissime occupano la maggioranza delle occorrenze, mentre la maggior parte dei termini compare pochissime volte (*long tail*).
- **Legge di Heaps:** la dimensione del vocabolario cresce come legge di potenza rispetto al numero totale di parole N della collezione, $|V| = k \cdot N^{\beta}$ (con $0.4 \leq \beta \leq 0.6$): il vocabolario non si satura mai completamente.

I principali modelli di reperimento sono il **Modello Booleano** (match esatto tramite operatori logici, privo di punteggio di rilevanza), il **Vector Space Model** (query e documenti come vettori pesati con TF-IDF, rilevanza tramite similarità del coseno) e il **Modello Probabilistico BM25**, standard de facto del reperimento lessicale:

$$\text{score}_{\text{BM25}}(q,d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t,d) \cdot (k_1+1)}{tf(t,d) + k_1 \cdot \left(1 - b + b \cdot \dfrac{|d|}{\text{avgdl}}\right)} \tag{1}$$

dove $\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right)$ pesa la specificità del termine, k_1 (tipicamente 1.2–2.0) controlla la saturazione del term frequency e $b \in [0,1]$ (tipicamente ≈ 0.75) normalizza la lunghezza del documento $|d|$ rispetto alla lunghezza media avgdl.

0.5 L'Inverted Index (Indice Invertito)

L'**Inverted Index** inverte l'orientamento logico della memorizzazione documentale, realizzando una mappatura $t \mapsto \mathcal{P}_t$, dove $\mathcal{P}_t$ è la lista ordinata dei documenti contenenti il termine t [1].

Definizione 0.5.1 (Inverted Index)**.** Un Inverted Index è costituito da un **dizionario** (l'insieme ordinato dei termini unici, con frequenza documentale df_t e puntatore alla posting list) e, per ogni termine, da una **posting list**: un array ordinato per DocID crescente, i cui elementi possono includere metadati come frequenza locale (tf) e offset posizionali.

Esempio 0.5.1 (Collezione Giocattolo di Riferimento)**.** Si consideri una piccola collezione di tre documenti sull'indicizzazione e la ricerca testuale. La Figura 3 illustra l'Inverted Index generato per alcuni termini chiave: il termine `a` compare nei documenti $1, 2, 3$; `an` solo nel documento 1; `index` e `inverted` nei documenti $1, 2$; `query` solo nel documento 3.

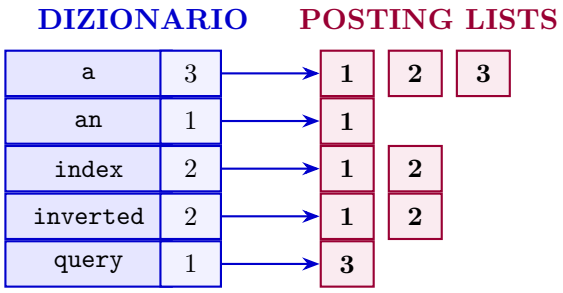


Figura 3: Struttura dell'Inverted Index: dizionario e posting list.

I costrutti booleani (AND, OR, NOT) e la ricerca per frase trovano diretta corrispondenza nell'interfaccia di ricerca avanzata dei motori commerciali (Tabella 2).

Operatore Logico IR	**Campo Google Advanced Search**	**Sintassi**
Congiunzione (AND)	*all these words*	`tri-colour rat terrier`
Ricerca per frase	*this exact word or phrase*	`"rat terrier"`
Disgiunzione (OR)	*any of these words*	`miniature OR standard`
Negazione (NOT)	*none of these words*	`-rodent -"Jack Russell"`
Intervallo numerico	*numbers ranging from...to*	`10..35 kg`

Tabella 2: Mappatura tra operatori booleani IR e Ricerca Avanzata di Google.

0.6 Tecniche di Compressione dell'Indice

La compressione dell'Inverted Index riduce drasticamente l'occupazione di memoria (sia su RAM che su disco), convertendo un collo di bottiglia di I/O in un carico di decompressione CPU, generalmente molto più veloce.

0.6.1 Binary Interpolative Coding

Il **Binary Interpolative Coding** (Moffat e Stuiver [3]) comprime sequenze strettamente crescenti di interi (i DocID delle posting list) partizionandole ricorsivamente: data una sottolista di n interi nota per essere compresa in $[l, h]$, si codifica l'elemento centrale x_m (con $m = \lfloor (n+1)/2 \rfloor$) usando $\lceil \log_2 R \rceil$ bit, dove $R = h - l - n + 1$ è il numero di valori ammissibili, e si ricorre poi separatamente sulle due metà. Quando i DocID sono densamente clusterizzati, R tende a 1 e molti elementi vengono codificati a costo pressoché nullo.

0.6.2 Compressione Basata su Dizionario: LZ77

Per testi generali, l'algoritmo **Lempel-Ziv '77** (LZ77) comprime tramite una finestra scorrevole, sostituendo ripetizioni lessicali con triple costituite da $(\text{offset}, \text{lunghezza}, \text{carattere})$ riferite a occorrenze precedenti nella finestra. Questa famiglia di algoritmi è alla base di strumenti diffusi come `gzip`, `7-Zip` e `LZ4`.

0.7 Learning-to-Rank (LTR) e la Sfida della Non-Differenziabilità

I candidati selezionati dall'Inverted Index vengono inoltrati a un modello di Machine Learning che li riordina (**Learning-to-Rank**, LTR) [4]. Il dataset di addestramento associa a ciascuna query una lista di documenti con giudizio di rilevanza graduato $y \in \{0, \ldots, 4\}$ (da irrilevante a perfettamente rilevante); un modello calcola uno score $s_i = f_\theta(\vec{x}_{q,d_i})$ per ciascuna coppia query-documento, e la lista finale viene ottenuta ordinando per punteggio decrescente.

Il Cardine dell'LTR

The goal is to learn the ranking, not the label: l'obiettivo non è stimare accuratamente il singolo valore y, ma l'ordinamento relativo ottimale, massimizzando metriche cumulative come **NDCG** o **MAP**.

Poiché l'operazione di ordinamento è discreta e a gradini, il gradiente delle metriche di ranking rispetto ai parametri del modello è nullo quasi ovunque: non è possibile applicare direttamente la discesa del gradiente. La letteratura ha sviluppato tre famiglie di soluzioni:

1. **Pointwise:** tratta ogni documento come un'istanza indipendente (regressione o classificazione su y), ignorando la natura comparativa del ranking.

2. **Pairwise** (es. RankNet): confronta coppie di documenti, modellando con una sigmoide differenziabile la probabilità che d_i preceda d_j.

3. **Listwise** (es. LambdaMART): ottimizza l'intera lista tramite gradienti surrogati λ_{ij}, pesati con la variazione effettiva di NDCG derivante dallo scambio di d_i e d_j; combinato con alberi di decisione potenziati, è stato per oltre un decennio lo standard nei motori di ricerca commerciali.

0.8 Evoluzione verso il Neural Information Retrieval

Dal 2018, i modelli Transformer (BERT) hanno inaugurato l'era del Neural Information Retrieval [5, 6], pre-addestrati tramite **Masked Language Modeling**. Gli approcci neurali per il reperimento si dividono in due famiglie:

- **Interazione (Cross-Encoder):** query e documento sono concatenati ed elaborati congiuntamente da un Transformer, catturando interazioni

profonde a costo computazionale elevato ($\mathcal{O}(|\mathcal{D}| \times \text{Transformer})$); usati solo come re-ranker su un piccolo insieme di candidati.

- **Rappresentazione (Bi-Encoder):** query e documento sono proiettati indipendentemente in uno spazio vettoriale da due encoder distinti; i vettori dei documenti sono pre-calcolabili offline, riducendo il matching online a un prodotto scalare o a una ricerca di vicini approssimati.

Tra i modelli a rappresentazione si distinguono tre famiglie di embedding: **dense single-vector** (un unico vettore denso per documento/query, interrogato con indici k-ANN su grafi come HNSW), **dense multi-vector** (es. ColBERT [5], un vettore per token con aggregazione *Late Interaction*/MaxSim, $S(q,d) = \sum_{t_q \in q} \max_{t_d \in d}(\vec{E}_{t_q} \cdot \vec{E}_{t_d}^T)$) e **sparse single-vector** (es. SPLADE [6], interpretabile per design e più efficiente in spazio/tempo rispetto al multi-vector).

k-Approximate Nearest Neighbor Search

Su collezioni di milioni di documenti rappresentati come vettori densi ad alta dimensionalità, il recupero esatto dei k più simili alla query è computazionalmente proibitivo: si ricorre quindi a indici di **ricerca approssimata dei vicini** (k-ANN), che scambiano una minima perdita di accuratezza per un guadagno di ordini di grandezza in velocità. **TusKANNy** (`https://tuskanny.com/`), sviluppato da ISTI-CNR e Università di Pisa, ne è un esempio open-source ad alte prestazioni scritto in Rust.

Capitolo 1

Valutazione dei Sistemi di Information Retrieval

1.1 Valutare un Sistema di Information Retrieval: Metodologia e Obiettivi

Un sistema di Information Retrieval (IR) è un'infrastruttura ingegneristica complessa progettata per connettere persone e bisogni informativi a collezioni estese di dati non strutturati [1, 2]. La valutazione scientifica e industriale di tali sistemi costituisce una disciplina cardine dell'IR, affrontata storicamente a partire dagli esperimenti pionieristici di Cyril Cleverdon negli anni '60 [7].

1.1.1 Efficienza, Servizi Accessori ed Efficacia Intrinseca

Valutare la qualità complessiva di un motore di ricerca richiede di disaccoppiare tre dimensioni di indagine complementari:

1. **Metriche di Efficienza Operativa e di Sistema:**
    - *Throughput di indicizzazione:* Velocità di assorbimento e indicizzazione di una collezione documentale, misurata tipicamente in documenti elaborati per ora (doc/h).
    - *Indicizzazione incrementale:* Capacità del sistema di aggiornare l'indice con nuovi flussi di dati (ad esempio l'inserimento di $+10\,000$ nuovi prodotti al giorno in un portale e-commerce) senza interrompere la disponibilità operativa né degradare i tempi di accesso.

- *Throughput e latenza di ricerca:* Tempo di risposta medio e percentili critici (95° e 99° percentile, tipicamente < 50–100 ms) unitamente all'impegno di CPU, memoria RAM e I/O per l'elaborazione contemporanea di decine di migliaia di query al secondo su collezioni di scala planetaria (decine di milioni o miliardi di documenti).

2. **Qualità ed Efficacia dei Servizi Accessori:**
   - Valutazione della precisione dei suggerimenti ortografici (*query spelling correction* o *"forse cercavi: ..."*), dell'espansione e riscrittura automatica delle query, della pertinenza delle raccomandazioni di prodotti o pagine correlate, e della leggibilità e densità informativa dei riassunti contestuali (*result snippets*).
   - Sebbene questi servizi accessori migliorino l'esperienza d'uso, essi non certificano di per sé la bontà fondamentale dell'algoritmo di ranking.
3. **Efficacia Intrinseca della Graduatoria (Ranking Intrinseco):**
   - Misura la capacità fondamentale del motore di ricerca di soddisfare l'utente, preparando per ciascuna interrogazione una pagina dei risultati (*Search Engine Results Page*, SERP) che posizioni i documenti pertinenti ai vertici della graduatoria.
   - L'obiettivo primario è massimizzare la soddisfazione complessiva dell'utente (*User Happiness*).

La Figura 1.1 schematizza la duplice pipeline offline/online di un moderno motore di ricerca, evidenziando i punti di rilevazione delle metriche di efficienza e di efficacia.

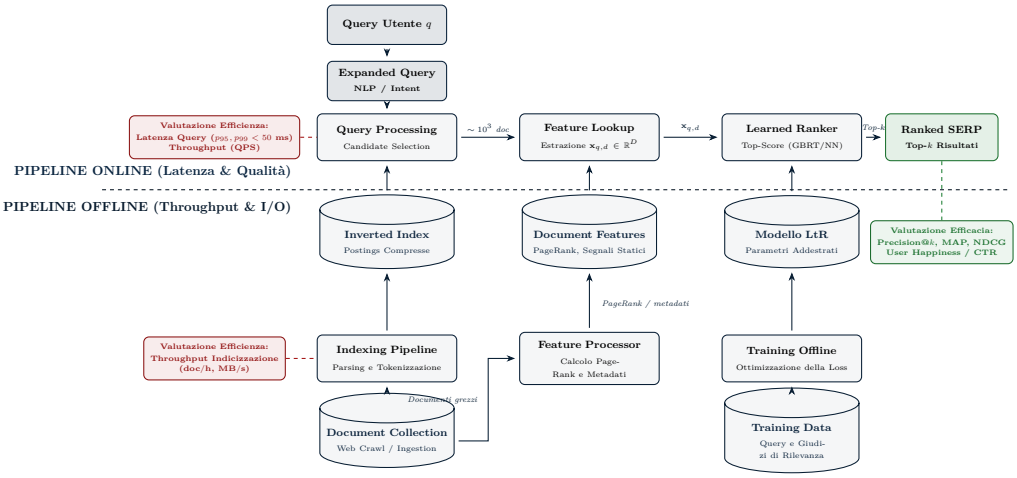


Figura 1.1: Architettura integrata di un moderno motore di ricerca e mappatura dei punti di rilevazione delle metriche di valutazione: efficienza offline (throughput di indicizzazione), efficienza online (latenza di query processing) ed efficacia qualitativa sulla SERP (NDCG, MAP, soddisfazione dell'utente).

1.1.2 Misurare la Soddisfazione dell'Utente (*User Happiness*)

Nel mondo operativo dei motori di ricerca commerciali, l'obiettivo finale consiste nel massimizzare la soddisfazione soggettiva dell'utente (*User Happiness*). Poiché lo stato di appagamento cognitivo non è direttamente osservabile dall'infrastruttura hardware, i sistemi reali monitorano una combinazione di **proxy comportamentali online**:

- **Clic e Click-Through Rate (CTR):** Proporzione di volte in cui un risultato presentato sulla SERP viene cliccato rispetto al numero totale di visualizzazioni.
  - *Criticità dei titoli ingannevoli:* Titoli accattivanti o sintesi fuorvianti (*clickbait*) generano un CTR artificialmente elevato, benché la pagina di destinazione si riveli completamente inutile per l'utente.
  - *La ricerca a zero clic (Zero-Click Search):* L'assenza di clic non costituisce sistematicamente un indice di fallimento. Quando la query esprime un bisogno informativo fattuale (ad esempio il meteo, una definizione dizionariale, il risultato di una partita o una conversione di valuta), la visualizzazione immediata della risposta corretta tramite *Knowledge Card* o *Featured Snippet* soddisfa perfettamente l'utente senza richiedere alcuna navigazione esterna.

- **Acquisti e Conversioni Economiche:** Nei motori per l'e-commerce, la metrica suprema risiede nella quota di ricerche che sfociano in un acquisto effettivo e nel volume economico transato dall'utente.
- **Fidelizzazione e Visite Ripetute:** Frequenza con cui gli utenti ritornano a impiegare la piattaforma nell'arco di una settimana o di un mese.
- **Dwell Time (Tempo di Permanenza):** Intervallo temporale che intercorre tra il clic su un risultato e il ritorno dell'utente alla SERP (*bouncing back*). Un Dwell Time prolungato segnala che la risorsa era ricca di contenuti utili, mentre un rimbalzo immediato sulla SERP testimonia che il documento era irrilevante o inadeguato.

1.1.3 La Valutazione nell'Era dell'Agentic Search e dei Modelli Generativi

Riflessione Metodologica: La Felicità dell'Utente con i Sistemi Generativi

L'affermazione di interfacce conversazionali avanzate basate su Large Language Model (quali ChatGPT o motori ibridi Agentic Search) mette in crisi l'impianto tradizionale fondato su CTR e Dwell Time: l'utente non scorre più una lista di link esterni, ma interagisce con un agente in grado di sintetizzare risposte complete e orchestrare chiamate a tool esterni.
In tale orizzonte emergono nuove sfide e metriche di valutazione:

- **Numero di turni conversazionali come proxy di insoddisfazione:** In un contesto dialogico, una sessione prolungata con molteplici iterazioni e raffinamenti non indica necessariamente un elevato gradimento, ma può denotare l'incapacità del sistema di cogliere l'intento iniziale dell'utente, costringendolo a ripetuti chiarimenti correttivi.
- **Sensibilità al prompting e varianza statistica:** Mentre una query IR lessicale produce un comportamento deterministico sull'indice, lievi variazioni nella formulazione di un prompt o fluttuazioni probabilistiche della temperatura di campionamento possono stravolgere la completezza e la veridicità dell'output generativo.
- **Dicotomia tra reperimento e sintesi (Il Paradigma RAG):** Nei sistemi RAG (*Retrieval-Augmented Generation*), la valutazione deve scorporare la qualità del recupero documentale (accuratezza del retrieval top-k) dalla fedeltà della sintesi testuale generata (assenza di allucinazioni e rigorosa attribuzione alle fonti citate).

1.2 Valutazione della Rilevanza e Metodologie di Annotazione

Poiché i segnali online (clic, conversioni) risentono pesantemente del *Position Bias* (gli utenti tendono a cliccare i primi risultati a prescindere dal loro effettivo valore) e non sono riproducibili sperimentalmente prima del rilascio in produzione di un nuovo modello, la misura regina per confrontare algoritmi

di Information Retrieval rimane la **rilevanza intrinseca** dei documenti reperiti rispetto a un'esigenza informativa data [1].

1.2.1 La Rilevanza come Proxy Fondamentale e gli Esperimenti di Cranfield

L'idea di fondare la valutazione dell'IR su un banco di prova controllato fu introdotta da Cyril Cleverdon nei celebri **esperimenti di Cranfield** (Inghilterra, fine anni '50 e anni '60) [7]. In quell'epoca pionieristica non esistevano motori Web né calcolatori elettronici interattivi: Cleverdon operava su collezioni cartacee specializzate in aeronautica, confrontando l'efficacia di diversi linguaggi di indicizzazione e schede perforate attraverso interrogazioni reali formulate da ingegneri.

Il contributo concettuale di Cleverdon si è consolidato nella nozione di **Collezione di Test** (*Test Collection*):

Definizione 1.2.1 (Collezione di Test Benchmark)**.** Una collezione di test standard per l'Information Retrieval è definita formalmente dalla tripletta:

$$\mathcal{T} = (\mathcal{D}, \mathcal{Q}, \mathcal{J}) \tag{1.1}$$

dove:

1. $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$ è la collezione finita di documenti benchmark.
2. $\mathcal{Q} = \{q_1, q_2, \ldots, q_M\}$ è una suite rappresentativa di bisogni informativi e query di prova.
3. $\mathcal{J} : \mathcal{Q} \times \mathcal{D} \to \mathcal{S}$ è l'insieme dei giudizi di rilevanza (*relevance judgments* o *qrels*), che associa a ciascuna coppia query-documento un valore di pertinenza appartenente a un insieme discreto $\mathcal{S}$ (binario $\mathcal{S} = \{0, 1\}$ o graduato $\mathcal{S} = \{0, 1, \ldots, m\}$).

1.2.2 Bisogno Informativo (*Information Need*) vs. Query

Un principio metodologico assoluto della teoria dell'IR risiede nella netta demarcazione tra l'intento mentale dell'essere umano e la sequenza di simboli immessa nella macchina:

- **Bisogno Informativo (*Information Need*):** È lo stato cognitivo dell'utente, l'esigenza o la lacuna di conoscenza che egli intende colmare. Esempio reale: *"Il fondo della mia piscina si sta macchiando di nero e desidero capire come pulirlo senza svuotarla"*.

- **Query:** È la stringa di testo sintetica e spesso imperfetta che l'utente compone nel campo di ricerca per estrinsecare il proprio bisogno. Esempio: `"pulitore piscina fondo"`.

Proprietà 1.2.1 (Principio di Valutazione della Rilevanza)**.** *La rilevanza di un documento deve essere rigorosamente valutata rispetto all'**Information Need** originario dell'utente, e **mai** rispetto alla mera rispondenza letterale dei token presenti nella query. Un documento che ripete ossessivamente le parole chiave della query ma non risponde al quesito sottostante è da considerarsi irrilevante.*

1.2.3 La Sfida della Scalabilità Combinatoria

Nelle collezioni Web e aziendali contemporanee, l'annotazione esaustiva della funzione $\mathcal{J}$ si scontra con una barriera combinatoria insormontabile. Si consideri uno scenario operativo standard:

$$|\mathcal{D}| = 5 \times 10^6 \text{ documenti}, \quad |\mathcal{Q}| = 50\,000 \text{ query} \tag{1.2}$$

Il numero totale di coppie query-documento da esaminare risulta:

$$|\mathcal{D}| \times |\mathcal{Q}| = 5 \times 10^6 \times 50\,000 = 2.5 \times 10^{11} = 0.25 \times 10^{12} \text{ coppie} \tag{1.3}$$

ovvero **un quarto di trilione di giudizi**!

Ipotizzando che un annotatore umano impieghi in media 2.5 secondi per leggere e qualificare la pertinenza di ciascun documento rispetto al topic:

$$\begin{aligned} \text{Tempo Complessivo} &= 2.5 \times 10^{11} \times 2.5 \text{ s} = 6.25 \times 10^{11} \text{ secondi} \\ &\approx 173\,611\,111 \text{ ore/persona} \\ &\approx 20\,000 \text{ anni/uomo continuativi!} \end{aligned} \tag{1.4}$$

Questo vincolo fisico dimostra l'impossibilità teorica e pratica di procedere a una valutazione esaustiva dell'intero archivio, rendendo obbligatorio restringere il giudizio a un sottoinsieme accuratamente campionato di documenti per ciascuna query.

1.2.4 Provenienza e Rappresentatività delle Query di Test

La scelta della suite $\mathcal{Q}$ deve garantire che le interrogazioni rispecchino la reale distribuzione delle esigenze informative degli utenti finali [1]:

- **Campionamento da Query Log Reali (Motori Web Moderni):** Sistemi commerciali quali Google, Microsoft Bing e Yahoo campionano le query di test direttamente dai propri log di transazione storici (come l'AOL Query Log o collezioni Microsoft ORCAS). Tale strategia preserva la validità ecologica del test, catturando sia le query generiche ad altissimo traffico (*head queries*) sia le ricerche complesse della coda lunga (*tail queries*).
- **Definizione Manuale da Esperti di Dominio (Approccio Classico):** Nelle collezioni in cui non si dispone di registrazioni d'uso massive (o in contesti iperspecialistici), specialisti del settore redigono a tavolino scenari strutturati d'uso (*topics*) e le correlate espressioni di ricerca.
- **Fallacia del Campionamento Sintetico:** Generare query campionando casualmente sequenze di termini o frasi dai documenti della collezione costituisce un errore metodologico gravissimo: le stringhe risultanti riflettono unicamente il lessico dei testi archiviati e non l'intento di ricerca umano, distorcendo irreparabilmente la valutazione del ranking.

1.2.5 Metodologie di Annotazione: Crowdsourcing ed Esperti

La compilazione dei giudizi di rilevanza può seguire diverse strategie operative:

- **Annotatori Esperti Dedicati:** Professionisti formati specificamente sul dominio di applicazione. Garantiscono la massima coerenza e precisione, ma comportano costi elevatissimi e bassa scalabilità.
- **Piattaforme di Crowdsourcing (es. Amazon Mechanical Turk):** Sottomettono coppie query-documento a una moltitudine di lavoratori distribuiti a basso costo. Sebbene abbatta i costi monetari, la letteratura evidenzia una **varianza statistica molto elevata**, dovuta alla presenza di utenti frettolosi, incompetenti o spammer. Per ottenere dati stabili occorre sottoporre ogni coppia a molteplici valutatori indipendenti e applicare metriche di accordo inter-annotatore (quali la κ di Cohen o la κ di Fleiss) e filtri di maggioranza.
- **Valutazione Tramite LLM (*LLM-as-a-Judge*):** L'impiego di modelli linguistici di frontiera per annotare la pertinenza query-documento sta rivoluzionando la velocità di generazione dei benchmark. Tuttavia, occorre monitorare e correggere i noti *bias intrinseci* degli LLM: il

length bias (tendenza a premiare testi verbosi e lunghi a prescindere dal contenuto), il *self-preference bias* (preferenza per risposte stilisticamente affini a quelle del modello stesso) e il *position bias* nel prompt.

1.3 Evoluzione Storica e Iniziative Attuali: Il Framework TREC

1.3.1 Dalle Collezioni Pioniere all'Esplosione Dimensionale

La Tabella 1.1 documenta l'evoluzione storica dei principali benchmark di valutazione in Information Retrieval, evidenziando il drastico passaggio di scala avvenuto negli anni '90.

Collezione	**Documenti (N)**	**Query (M)**	**Dimensione**	**Termini/Doc**	**Valutazioni (Q-D)**
ADI	82	35	<1 MB	—	Esaustive
AIT	2 109	14	2 MB	400.0	$>10\,000$
CACM	3 204	64	2 MB	24.5	Parziali
CISI	1 460	112	2 MB	46.5	Parziali
Cranfield	**1 400**	**225**	**2 MB**	**53.1**	**Esaustive** (100%)
LISA	5 872	35	3 MB	—	Parziali
Medline	1 033	30	1 MB	—	Esaustive
NPL	11 429	93	3 MB	—	Parziali
OSHMED	348 566	106	400 MB	250.0	16 140
Reuters-21578	21 578	672	28 MB	131.0	Parziali
TREC (Tipica)	**740 000**	**200**	**2 000 MB (2 GB)**	**89–3 543**	$>100\,000$ **(Pool)**

Tabella 1.1: Panoramica dimensionale dei corpora benchmark storici e raffronto con l'architettura TREC [1].

Nelle collezioni pioniere degli anni '60 e '70 (come Cranfield e Medline), le ridotte dimensioni (poche migliaia di articoli scientifici e meno di 2 MB di dati) permettevano agli annotatori di analizzare fisicamente ogni singolo documento della collezione per ciascuna query, compilando una matrice di rilevanza completa al 100%. Con l'avvento di archivi da centinaia di migliaia di documenti e gigabyte di testo, questo approccio è divenuto impraticabile, determinando la nascita della Text REtrieval Conference.

1.3.2 L'Infrastruttura TREC (*Text REtrieval Conference*)

Istituita nel 1992 e sponsorizzata congiuntamente dal National Institute of Standards and Technology (NIST) e dal Dipartimento della Difesa statunitense (DARPA), la conferenza **TREC** ha fornito alla comunità scientifica

internazionale l'infrastruttura metodologica e computazionale per testare sistemi IR su grandissima scala [1].

L'organizzazione di TREC si fonda su tre pilastri:

1. **Tracce Tematiche (*Tracks*):** Aree di ricerca focalizzate su specifici compiti operativi o domini applicativi (ricerca Web, e-discovery legale, genomica e biomedicina, question answering, deep retrieval, enterprise search).

2. **Definizione Formale dei Topic:** A differenza delle brevi query del Web, un **Topic TREC** descrive formalmente l'esigenza informativa mediante tre campi distinti:

   - **Title:** Una brevissima sequenza di parole chiave (simile a una query Web reale, es. `"acquisizione idrocarburi artico"`).
   - **Description:** Una frase che sintetizza l'obiettivo del bisogno informativo (es. *"Identificare accordi commerciali per lo sfruttamento di giacimenti di petrolio o gas nel Mare Glaciale Artico"*).
   - **Narrative:** Un testo esplicativo redatto dal committente che specifica in dettaglio cosa renda un documento rilevante e cosa debba essere rigorosamente escluso (es. *"I documenti pertinenti devono citare trattati o contratti stipulati tra governi o corporazioni. Sono esclusi studi meramente geologici o ambientali privi di risvolti economici diretti"*).

3. **Il Concetto di Run:** Il file contenente i risultati (tipicamente i primi 1 000 documenti per ciascun topic, ordinati per punteggio decrescente) inviato da ciascun gruppo di ricerca prima della scadenza della gara prende il nome di **Run**.

1.3.3 La Tecnica del Pooling e l'Ipotesi dei Documenti Non Giudicati

Per risolvere l'impasse combinatoria senza rinunciare all'affidabilità statistica, le prime edizioni di TREC hanno introdotto la tecnica del **Pooling** (Figura 1.2):

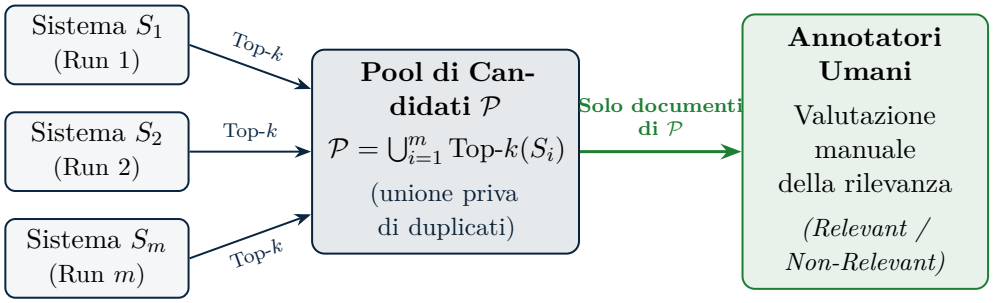


Figura 1.2: Architettura operativa del processo di Pooling: i primi k documenti estratti da ciascuna run indipendente confluiscono nel pool deduplicato $\mathcal{P}$ sottoposto ad annotazione umana.

Il workflow operativo del pooling procede come segue:

1. Si ricevono le run prodotte da un insieme variegato di m sistemi partecipanti (costruiti con paradigmi algoritmici eterogenei: booleano esteso, BM25, vettoriale VSM, modelli probabilistici e di espansione query).

2. Per ogni query, si prelevano i primi k documenti restituiti da ciascun sistema (tipicamente $k = 50$ o $k = 100$).

3. Si calcola l'unione insiemistica di questi elenchi, rimuovendo tutti i duplicati per costituire il pool dei candidati:

$$\mathcal{P}(q) = \bigcup_{i=1}^{m} \text{Top-}k(S_i, q) \tag{1.5}$$

4. Gli annotatori umani del NIST esaminano e giudicano la pertinenza **soltanto** dei documenti compresi nel pool $\mathcal{P}(q)$.

L'Assunzione dei Documenti Non Giudicati (*Unpooled Documents*)

Tutti i documenti dell'archivio $\mathcal{D}$ che non compaiono nel pool $\mathcal{P}(q)$ vengono considerati convenzionalmente **non rilevanti** ($r = 0$).
Validità empirica e limiti: Sebbene questa assunzione introduca un potenziale *pooling bias* a sfavore di algoritmi innovativi futuri che dovessero recuperare documenti rilevanti non intercettati da alcuno dei sistemi originari, le verifiche sperimentali condotte da NIST hanno dimostrato che:

- La stragrande maggioranza dei documenti pertinenti della collezione viene intercettata purché il pool includa run generate da modelli sufficientemente diversificati.
- Sebbene le metriche assolute di un nuovo sistema possano risultare leggermente sottostimate, il **ranking comparativo** tra sistemi rimane eccezionalmente stabile ed equo.

1.4 Metriche di Valutazione Binarie per Insiemi Non Ordinati

Nel paradigma dell'*Unranked Retrieval* (tipico dei sistemi booleani esatti), il motore restituisce un insieme piatto di documenti privo di ordinamento qualitativo. Ciascun documento della collezione è bipartito rispetto alla rilevanza effettiva e allo stato di recupero.

1.4.1 Matrice di Contingenza e Spazio delle Decisioni

La valutazione si fonda sulla scomposizione dello spazio campionario nella classica **matrice di contingenza** 2×2 (Tabella 1.2):

	Documento Rilevante	**Documento Non Rilevante**
Documento Reperito	True Positive (TP)	False Positive (FP)
Documento Non Reperito	False Negative (FN)	True Negative (TN)

Tabella 1.2: Matrice di contingenza per la classificazione query-documento.

La Figura 1.3 offre una chiara lettura insiemistica mediante diagramma di Eulero-Venn.

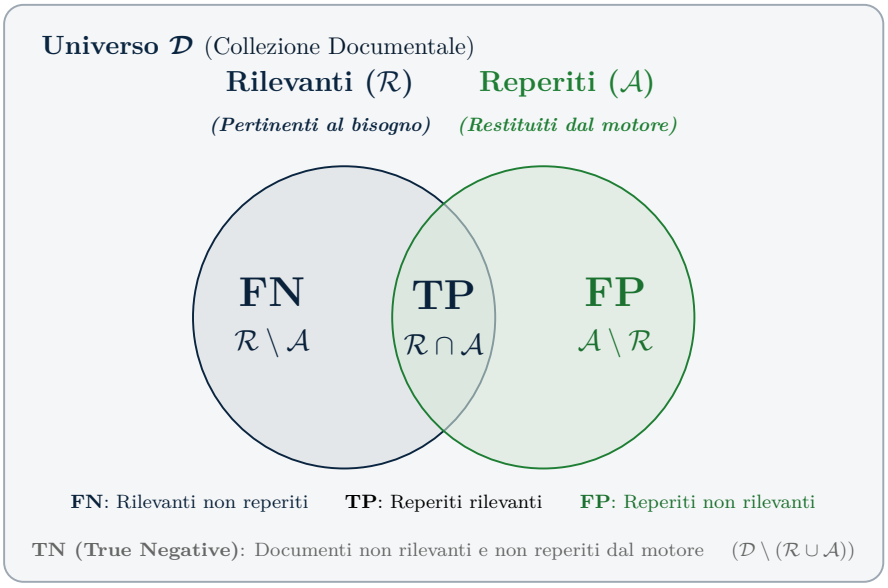


Figura 1.3: Rappresentazione insiemistica del reperimento documentale e partizione delle regioni della matrice di confusione.

1.4.2 Precisione, Richiamo e loro Trade-off

Definizione 1.4.1 (Precisione (*Precision*, P))**.** Frazione dei documenti restituiti dal motore che risultano effettivamente rilevanti rispetto al bisogno informativo:

$$P = \frac{TP}{TP + FP} = P(\text{rilevante} \mid \text{reperito}) \tag{1.6}$$

Definizione 1.4.2 (Richiamo (*Recall*, R))**.** Frazione dei documenti rilevanti complessivamente presenti nella collezione che il sistema è stato in grado di individuare ed estrarre:

$$R = \frac{TP}{TP + FN} = P(\text{reperito} \mid \text{rilevante}) \tag{1.7}$$

Tra Precisione e Richiamo sussiste un **trade-off intrinseco**:

- Un sistema banale che restituisce l'intero archivio documentale ($\mathcal{A} = \mathcal{D}$) raggiunge un richiamo perfetto $R = 1.0$ (100%), ma la sua precisione collassa a valori irrisori ($P \approx 0$).

- All'opposto, un sistema iper-selettivo che restituisce esclusivamente il singolo documento in assoluto più probabile può ottenere $P = 1.0$ (100%), sacrificando quasi integralmente il richiamo ($R \approx 0$).

1.4.3 La Media Armonica: F-Measure (F_1-Score)

Per sintetizzare la bontà di un sistema in un indicatore scalare unico bilanciando P ed R, si definisce la **F-Measure** (o F_1-score) come la **media armonica** di Precisione e Richiamo:

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R} = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \tag{1.8}$$

Proprietà 1.4.1 (Proprietà della Media Armonica e Superiorità rispetto alla Media Aritmetica)**.** *La media armonica si rivela l'unica scelta rigorosa per la valutazione congiunta in virtù delle sue proprietà:*

1. *È sempre dominata dalla media geometrica e dalla media aritmetica:*

$$\frac{2}{\frac{1}{P} + \frac{1}{R}} \leq \sqrt{P \cdot R} \leq \frac{P + R}{2} \tag{1.9}$$

2. *Quando i due parametri divergono in modo marcato, la media armonica collassa verso il* ***minimo*** *tra i due valori, punendo severamente ogni squilibrio algoritmico.*

Esempio 1.4.1 (La Fallacia della Media Aritmetica)**.** Si consideri il motore banale che restituisce l'intera collezione: $R = 1.0$ e $P = 0.0001$.

- La media aritmetica assegna un punteggio assurdo superiore al 50%:

$$\text{Media Aritmetica} = \frac{1.0 + 0.0001}{2} \approx 0.50005 \quad (50.0\%) \tag{1.10}$$

  permettendo a un sistema totalmente inservibile di apparire mediocremente efficace.

- La media armonica (F_1) demolisce immediatamente il sistema:

$$F_1 = \frac{2 \cdot 1.0 \cdot 0.0001}{1.0 + 0.0001} \approx 0.0002 \quad (0.02\%) \tag{1.11}$$

  restituendo una misura realistica dell'inutilità del recupero.

1.4.4 La Misura Pesata F_β

Se l'applicazione impone di dare priorità asimmetrica a una delle due grandezze, si introduce un fattore di ponderazione $\alpha \in [0, 1]$:

$$F = \frac{1}{\alpha \frac{1}{P} + (1 - \alpha) \frac{1}{R}} \tag{1.12}$$

Ponendo convenientemente $\alpha = \frac{1}{1+\beta^2}$, si ricava la formulazione canonica di F_β:

$$F_\beta = \frac{(1 + \beta^2) \cdot P \cdot R}{\beta^2 \cdot P + R} \tag{1.13}$$

Il parametro $\beta \in [0, +\infty)$ controlla il bilanciamento relativo:

- $\beta = 1.0$: Si riottiene la classica F_1 a pesatura identica.
- $\beta < 1.0$ (ad esempio $\beta = 0.5$): Il modello valorizza la **Precisione** rispetto al Richiamo. Tipico della ricerca Web generica, dove all'utente interessa trovare immediatamente pochi documenti perfetti.
- $\beta > 1.0$ (ad esempio $\beta = 2.0$): Il modello valorizza il **Richiamo** rispetto alla Precisione. Imprescindibile nelle ricerche brevettuali (*patent search*), nelle perizie medico-legali e nell'e-discovery giuridico, dove mancare anche un solo documento rilevante comporta danni irreparabili.

1.5 Metriche Basate sul Ranking (*Ranked Measures* - Rilevanza Binaria)

Nei motori di ricerca operativi i documenti non formano insiemi caotici, ma vengono presentati in una graduatoria ordinata in base al punteggio decrescente di rilevanza stimata. Poiché gli utenti esaminano quasi esclusivamente i primissimi risultati della SERP, la valutazione deve premiare la concentrazione dei documenti pertinenti in cima alla lista.

1.5.1 Precision@k e Recall@k: Valutazione alla Soglia di Rango

Definizione 1.5.1 (Precision@k ($P@k$))**.** Data una soglia di rango $k \in \mathbb{N}^+$, $P@k$ corrisponde alla proporzione di documenti rilevanti presenti nei primi k posti della graduatoria restituita:

$$P@k = \frac{1}{k} \sum_{i=1}^{k} \text{rel}_i \tag{1.14}$$

dove $\text{rel}_i \in \{0, 1\}$ rappresenta l'esito binario di rilevanza del documento collocato al rango i.

Esempio 1.5.1 (Calcolo Discreto di $P@k$ ed Evidenza del Comportamento Non Monotono)**.** Sia data la graduatoria ordinata dei primi 5 documenti con rilevanze $\mathbf{r} = \langle 1, 1, 0, 0, 1 \rangle$:

- $P@1 = \frac{1}{1} = 1.00$
- $P@2 = \frac{2}{2} = 1.00$
- $P@3 = \frac{2}{3} \approx 0.67$
- $P@4 = \frac{2}{4} = 0.50$ (**si noti come** $P@4 < P@3$)
- $P@5 = \frac{3}{5} = 0.60$ (**la precisione torna a crescere**)

In modo complementare, *Recall*@k ($R@k$) calcola la frazione dei documenti rilevanti complessivi R recuperati entro la soglia k:

$$R@k = \frac{1}{R} \sum_{i=1}^{k} \text{rel}_i \tag{1.15}$$

1.5.2 Limiti Metodologici di $P@k$ nella Media tra Query

La Distorsione di $P@k$ nella Media Macro

Sebbene $P@k$ fornisca un'indicazione immediata della pulizia della prima schermata, presenta un difetto metodologico strutturale quando viene mediato su insiemi di query eterogenee:

- Il numero totale R di documenti rilevanti presenti nella collezione varia sensibilmente da query a query.
- Se per la query q_A esistono soltanto $R_A = 2$ documenti rilevanti nell'intera collezione, il punteggio massimo teorico di $P@10(q_A)$ è pari a $\frac{2}{10} = 0.20$, persino qualora il motore sia infallibile e collochi entrambi i documenti esattamente ai primi due posti!
- Viceversa, per una query generica q_B con $R_B = 100$, il sistema può conseguire comodamente $P@10(q_B) = 1.0$.
- Mediando aritmeticamente $P@10$, l'algoritmo viene penalizzato dalla scarsità oggettiva dei documenti rilevanti di talune query e non da deficienze del proprio ordinamento.

1.5.3 Average Precision (AP)

L'**Average Precision** (AP) risolve i limiti delle metriche a soglia rigida, sintetizzando l'intera curva Precision-Recall per una singola query in un indicatore scalare unitario.

Definizione 1.5.2 (Average Precision)**.** Dato un ordinamento di N documenti per una query avente R documenti rilevanti totali presenti nell'archivio, l'AP è la media delle precisioni istantanee $P@k$ calcolate in corrispondenza di ciascuna posizione k in cui si incontra un documento rilevante (ovvero dove il richiamo subisce un incremento):

$$AP = \frac{1}{R} \sum_{k=1}^{N} P@k \cdot \mathrm{rel}_k = \frac{1}{R} \sum_{k \in \{K_1, K_2, \ldots, K_R\}} P@k \tag{1.16}$$

dove $K_1, K_2, \ldots, K_R$ indicano i ranghi dei documenti rilevanti reperiti.

Proprietà 1.5.1 (Interpretazione Geometrica dell'Average Precision)**.** *L'Average Precision possiede un solido fondamento matematico:*

1. *In corrispondenza di ogni documento rilevante al rango k, il richiamo cresce di un incremento discreto pari a $\Delta R = \frac{1}{R}$. Pertanto, la sommatoria calcola la somma delle aree dei rettangoli $P@k \cdot \Delta R$, rappresentando l'esatta* ***Area Sotto la Curva Precision-Recall*** *(AUC-PR).*

2. *La normalizzazione per R (il totale assoluto dei documenti rilevanti nell'intera collezione, e non solo quelli estratti) impone che tutti i rilevanti non compresi nei primi N ranghi concorrano implicitamente con precisione pari a 0, penalizzando severamente i sistemi a basso richiamo.*

La Figura 1.4 illustra graficamente la sequenza di computazione dell'AP.

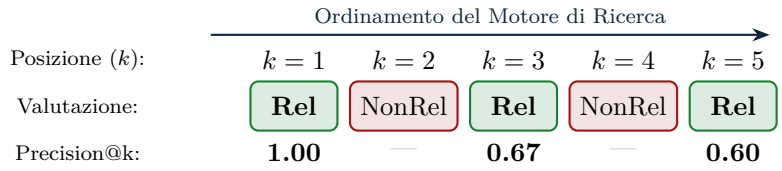


Figura 1.4: Calcolo passo-passo di Average Precision per un ordinamento di 5 elementi con $R = 3$ documenti rilevanti totali.

Esempio 1.5.2 (Studio Comparativo tra Due Ordinamenti Alternativi)**.** Si consideri una query avente $R = 6$ documenti rilevanti nella collezione. Due motori alternativi producono i seguenti primi 10 documenti:

- **Ranking #1 (Rilevanti concentrati in cima):**

$$\langle \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{NonRel}, \text{Rel} \rangle$$

- **Ranking #2 (Rilevanti ritardati e dispersi):**

$$\langle \text{NonRel}, \text{Rel}, \text{NonRel}, \text{NonRel}, \text{Rel}, \text{Rel}, \text{Rel}, \text{NonRel}, \text{Rel}, \text{Rel} \rangle$$

La Tabella 1.3 traccia l'evoluzione di Precisione e Richiamo rango per rango.

	Ranking #1			**Ranking #2**		
Rango (k)	**Esito**	**Recall**	**Precision**	**Esito**	**Recall**	**Precision**
1	**Rel**	0.17	**1.00**	NonRel	0.00	—
2	NonRel	0.17	—	**Rel**	0.17	**0.50**
3	**Rel**	0.33	**0.67**	NonRel	0.17	—
4	**Rel**	0.50	**0.75**	NonRel	0.17	—
5	**Rel**	0.67	**0.80**	**Rel**	0.33	**0.40**
6	**Rel**	0.83	**0.83**	**Rel**	0.50	**0.50**
7	NonRel	0.83	—	**Rel**	0.67	**0.57**
8	NonRel	0.83	—	NonRel	0.67	—
9	NonRel	0.83	—	**Rel**	0.83	**0.56**
10	**Rel**	1.00	**0.60**	**Rel**	1.00	**0.60**
Average Precision (AP)			**0.78**			**0.52**

Tabella 1.3: Confronto di Average Precision: Ranking #1 premia la reattività posizionale anticipando i rilevanti.

I rispettivi punteggi analitici valgono:

$$AP(\text{Ranking } \#1) = \frac{1.00 + 0.67 + 0.75 + 0.80 + 0.83 + 0.60}{6} = \frac{4.65}{6} \approx \mathbf{0.78} \tag{1.17}$$

$$AP(\text{Ranking } \#2) = \frac{0.50 + 0.40 + 0.50 + 0.57 + 0.56 + 0.60}{6} = \frac{3.13}{6} \approx \mathbf{0.52} \tag{1.18}$$

Sebbene entrambi i motori recuperino tutti e 6 i documenti rilevanti entro il decimo rango ($R = 100\%$), Ranking #1 distanzia nettamente il concorrente (0.78 contro 0.52), premiando il risparmio di sforzo visivo garantito all'utente.

1.5.4 Mean Average Precision (MAP)

Quando si valuta l'efficacia di un algoritmo sull'intera suite di query benchmark $\mathcal{Q}$, si adotta la **Mean Average Precision** (MAP):

Definizione 1.5.3 (Mean Average Precision)**.** Media aritmetica dei valori di Average Precision calcolati su tutte le query del benchmark:

$$MAP = \frac{1}{|\mathcal{Q}|} \sum_{j=1}^{|\mathcal{Q}|} AP(q_j) \tag{1.19}$$

Proprietà 1.5.2 (Macro-Averaging e Tutela della Coda Lunga)**.** *Il calcolo di MAP costituisce una procedura formale di* ***Macro-Averaging****: a ciascuna query viene assegnato esattamente lo stesso peso pari a* $1/|\mathcal{Q}|$*.*

- *Se si impiegasse il Micro-Averaging (sommare tutti i contributi di precisione dell'intero benchmark dividendo per il totale cumulativo dei documenti rilevanti esistenti), il punteggio globale verrebbe cannibalizzato da pochissime query ad altissima densità documentale (es.* `"sport"` *o* `"notizie"`*, con decine di migliaia di rilevanti).*
- *Il Macro-Averaging preserva la parità di rappresentanza: l'insuccesso su una query specialistica di coda lunga (tail query) incide sul punteggio finale con la medesima gravità del fallimento su una query comune.*

Esempio 1.5.3 (Calcolo di MAP su Due Query Distinte)**.** Si consideri un testbed con due bisogni informativi q_1 e q_2:

- **Query** q_1 **(**$R_1 = 5$ **rilevanti totali):** Reperiti 5 rilevanti ai ranghi $\{1, 3, 6, 9, 10\}$ con precisioni istantanee 1.00, 0.67, 0.50, 0.44, 0.50:

$$AP(q_1) = \frac{1.00 + 0.67 + 0.50 + 0.44 + 0.50}{5} = \frac{3.11}{5} \approx \mathbf{0.62} \tag{1.20}$$

- **Query** q_2 **(**$R_2 = 3$ **rilevanti totali):** Reperiti 3 rilevanti ai ranghi $\{2, 5, 7\}$ con precisioni istantanee 0.50, 0.40, 0.43:

$$AP(q_2) = \frac{0.50 + 0.40 + 0.43}{3} = \frac{1.33}{3} \approx \mathbf{0.44} \tag{1.21}$$

Il valore di Mean Average Precision del sistema su questo benchmark è:

$$MAP = \frac{AP(q_1) + AP(q_2)}{2} = \frac{0.62 + 0.44}{2} = \mathbf{0.53} \tag{1.22}$$

1.5.5 Metriche Troncate per il Web e Recommender Systems: AP@k e MAP@k

Nell'architettura del Web search moderno e nei sistemi di raccomandazione, calcolare l'AP sull'intero archivio è impraticabile: il numero totale R di documenti rilevanti sparsi nel Web è ignoto, e all'utente finale non interessa ciò che accade oltre la ventesima posizione.

Si definiscono pertanto le versioni troncate a una soglia prestabilita K (tipicamente $K = 10$ o $K = 20$):

Definizione 1.5.4 (AP@K e MAP@K)**.** Data la profondità di taglio K, l'Average Precision at K per la query q_i è:

$$AP@K(q_i) = \frac{1}{K_i} \sum_{k=1}^{K} P@k(q_i) \cdot \text{rel}(q_i, k) \tag{1.23}$$

dove il fattore di normalizzazione K_i corrisponde al numero effettivo di documenti rilevanti reperibili entro la soglia:

$$K_i = \min(R_i, K) \quad \left(\text{oppure } K_i = \sum_{k=1}^{K} \text{rel}(q_i, k)\right) \tag{1.24}$$

La metrica globale **MAP@K** su $\mathcal{Q}$ corrisponde alla media aritmetica:

$$MAP@K(\mathcal{Q}) = \frac{1}{|\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} AP@K(q_i) \tag{1.25}$$

Normalizzando per $\min(R_i, K)$ anziché per il valore rigido K, una query con soli 2 rilevanti totali che li collochi ai primi due posti consegue $AP@10 = 1.0$, eliminando la distorsione strutturale di $P@10$.

1.5.6 Mean Reciprocal Rank (MRR)

In contesti applicativi quali:

- *Known-Item Search:* L'utente cerca una pagina o un file specifico di cui conosce l'esistenza;
- *Query Navigazionali:* Ricerca dell'accesso o della homepage di un servizio (ad es. `"portale studenti unipi"`);
- *Factoid Question Answering:* Domande che ammettono una singola risposta esatta,

l'utente non ha alcun interesse a esaminare dieci documenti rilevanti, ma desidera localizzare unicamente la **prima risposta corretta**. Lo sforzo cognitivo e il tempo speso dall'utente sono inversamente proporzionali alla posizione di rango del primo risultato utile:

Definizione 1.5.5 (Reciprocal Rank e MRR)**.** Sia $\text{rank}_1(q)$ la posizione ordinale del primo documento rilevante restituito per la query q. Il **Reciprocal Rank** (RR) è definito come:

$$RR(q) = \frac{1}{\text{rank}_1(q)} \tag{1.26}$$

Il **Mean Reciprocal Rank** (MRR) è la media aritmetica di RR sull'insieme delle query benchmark:

$$MRR = \frac{1}{|\mathcal{Q}|} \sum_{i=1}^{|\mathcal{Q}|} \frac{1}{\text{rank}_1(q_i)} \tag{1.27}$$

Se il primo documento rilevante si trova al rango 1, il punteggio vale 1.0; se scende al rango 2 diviene 0.5; se scende al rango 10 crolla a 0.1. Qualora la graduatoria non contenga alcun documento utile entro la finestra considerata, si pone convenzionalmente $RR = 0$.

1.6 Oltre la Rilevanza Binaria: Rilevanza Graduata e NDCG

Nelle applicazioni Web e commerciali, l'assunzione che un documento sia categoricamente rilevante (1) o non rilevante (0) si rivela eccessivamente semplicistica.

Si consideri a titolo esemplificativo la query reale:

```
Query:  "information retrieval unipi"
```

I risultati reperibili presentano sfumature qualitative nettamente distinte:

1. **DidaWiki del Corso (Docente, CFU, programma didattico, dispense ufficiali):** Risponde integralmente e perfettamente all'intento dell'utente $\implies$ **Altamente Rilevante** ($r = 3$).

2. **Pagina Valutami UniPi (Scheda descrittiva e requisiti istituzionali d'esame):** Fonte autorevole e attinente, ma priva di materiale didattico operativo $\implies$ **Rilevante** ($r = 2$).

3. **Guida Generale del Corso di Laurea Magistrale:** Cita brevemente il corso nel piano di studi $\implies$ **Marginalmente Rilevante** ($r = 1$).

4. **Articoli di cronaca generale o pagine esterne slegate:** $\implies$ **Irrilevante** ($r = 0$).

Tutti i primi tre documenti presentano un legame tematico con la query, ma l'utente percepisce una soddisfazione radicalmente diversa. Per modellare questa granularità, la letteratura adotta scale discrete di giudizio a più livelli (tipicamente $\mathcal{S} = \{0, 1, 2, 3\}$ o $\{0, 1, 2, 3, 4\}$).

1.6.1 I Fondamenti del Discounted Cumulative Gain (DCG)

La formulazione del **Discounted Cumulative Gain** (DCG), introdotta da Järvelin e Kekäläinen [8], poggia su due assunzioni comportamentali cardine:

1. I documenti altamente rilevanti arrecano all'utente un'utilità informativa (*gain*) enormemente superiore rispetto a documenti solo marginalmente pertinenti.

2. Minore è la posizione ordinale (rango) di un documento nella SERP, minore è la probabilità che l'utente lo esamini; pertanto, il guadagno apportato deve essere progressivamente scontato (*discounted*) verso il basso con il crescere del rango.

Definizione 1.6.1 (Cumulative Gain a Rango p (CG_p))**.** Dato il vettore delle rilevanze graduate $\mathbf{r} = [r_1, r_2, \ldots, r_p]$ fino al rango di taglio p, il Cumulative Gain è la somma lineare dei singoli guadagni:

$$CG_p = \sum_{i=1}^{p} r_i \tag{1.28}$$

Limite strutturale: CG_p è del tutto insensibile all'ordinamento posizionale: scambiare un documento con $r = 3$ tra il primo e il decimo posto non altera minimamente il valore di CG_p.

1.6.2 Formulazione Standard vs. Formulazione Esponenziale

Per modellare il decadimento dell'attenzione visiva dell'utente, il DCG introduce un fattore di sconto posizionale:

Definizione 1.6.2 (DCG Standard (Järvelin e Kekäläinen))**.** Applica uno sconto logaritmico in base al rango i:

$$DCG_p = r_1 + \sum_{i=2}^{p} \frac{r_i}{\log_2(i)} \tag{1.29}$$

Proprietà 1.6.1 (Significato Fisico dello Sconto Logaritmico)**.** *Il divisore* $\log_2(i)$ *modella la progressione dello sguardo lungo la pagina:*

- *Al rango* $i = 4$*, il fattore vale* $\frac{1}{\log_2(4)} = \frac{1}{2}$*: il valore del documento si dimezza rispetto alla prima posizione.*
- *Al rango* $i = 8$*, il fattore scende a* $\frac{1}{\log_2(8)} = \frac{1}{3}$*.*
- *Al rango* $i = 16$*, il fattore si riduce a* $\frac{1}{\log_2(16)} = \frac{1}{4}$*.*

Nei motori di ricerca commerciali Web (Google, Microsoft Bing) si adotta prevalentemente la **formulazione esponenziale**:

$$DCG_p^{\text{exp}} = \sum_{i=1}^{p} \frac{2^{r_i} - 1}{\log_2(i+1)} \tag{1.30}$$

Differenze Chiave tra le Formulazioni di DCG

1. **Gestione del primo rango:** Nell'Equazione (1.29) il termine r_1 è scorporato per evitare $\log_2(1) = 0$ a denominatore, facendo partire la sommatoria da $i = 2$. Nell'Equazione (1.30) l'indice parte direttamente da $i = 1$ adottando $\log_2(i+1)$ ($\log_2(1+1) = \log_2(2) = 1$).
2. **Guadagno non lineare:** Il numeratore $2^{r_i} - 1$ dilata esponenzialmente il peso dei documenti eccellenti: un documento con $r = 3$ conferisce un guadagno $2^3 - 1 = 7$, mentre un documento con $r = 1$ vale solo $2^1 - 1 = 1$. Questo premia in modo aggressivo il recupero dei documenti a massima pertinenza ai primissimi ranghi.

1.6.3 Normalized Discounted Cumulative Gain (NDCG)

Poiché query diverse presentano un numero variabile di documenti rilevanti (e con differenti gradi di pertinenza massima), il valore assoluto di DCG_p non è confrontabile né mediabile tra query eterogenee.

Si definisce l'**Ideal DCG** ($IDCG_p$) come il valore di DCG calcolato sulla graduatoria ideale (*perfect ranking*), ottenuta ordinando tutti i documenti della collezione in senso decrescente di rilevanza:

Definizione 1.6.3 (Normalized Discounted Cumulative Gain)**.**

$$NDCG_p = \frac{DCG_p}{IDCG_p} \quad \in [0, 1] \tag{1.31}$$

dove:

$$IDCG_p = r_1^* + \sum_{i=2}^{p} \frac{r_i^*}{\log_2(i)} \quad \text{con } r_1^* \geq r_2^* \geq \cdots \geq r_p^* \tag{1.32}$$

La normalizzazione garantisce che un ordinamento perfetto consegua $NDCG = 1.0$ per qualsiasi query, rendendo la metrica idonea alla media macroscopica su ampie suite di benchmark.

1.6.4 Esempio Numerico Completo: Traccia a 10 Documenti

Si consideri la seguente graduatoria di 10 documenti, valutata su scala $r \in \{0, 1, 2, 3\}$:

$$\mathbf{r} = [3,\ 2,\ 3,\ 0,\ 0,\ 1,\ 2,\ 2,\ 3,\ 0] \tag{1.33}$$

Ordinando i valori di rilevanza in senso non crescente si ottiene la graduatoria ideale:

$$\mathbf{r}^* = [3,\ 3,\ 3,\ 2,\ 2,\ 2,\ 1,\ 0,\ 0,\ 0] \tag{1.34}$$

La Tabella 1.4 riporta l'evoluzione analitica dettagliata di ogni singolo termine posizionale secondo la formulazione standard.

i	r_i	Divisore Sconto	Guadagno Scontato	$DCG@i$	r_i^*	Guadagno Ideale	$IDCG@i$	$NDCG@i$
1	3	1.000	3.00	**3.00**	3	3.00	**3.00**	**1.00**
2	2	$\log_2(2) = 1.000$	2.00	**5.00**	3	3.00	**6.00**	**0.83**
3	3	$\log_2(3) \approx 1.585$	1.89	**6.89**	3	1.89	**7.89**	**0.87**
4	0	$\log_2(4) = 2.000$	0.00	**6.89**	2	1.00	**8.89**	**0.76**
5	0	$\log_2(5) \approx 2.322$	0.00	**6.89**	2	0.86	**9.75**	**0.71**
6	1	$\log_2(6) \approx 2.585$	0.39	**7.28**	2	0.77	**10.52**	**0.69**
7	2	$\log_2(7) \approx 2.807$	0.71	**7.99**	1	0.36	**10.88**	**0.73**
8	2	$\log_2(8) = 3.000$	0.67	**8.66**	0	0.00	**10.88**	**0.80**
9	3	$\log_2(9) \approx 3.170$	0.95	**9.61**	0	0.00	**10.88**	**0.88**
10	0	$\log_2(10) \approx 3.322$	0.00	**9.61**	0	0.00	**10.88**	**0.88**

Tabella 1.4: Computazione analitica passo-passo di $DCG@i$, $IDCG@i$ e $NDCG@i$ per i 10 documenti dell'esempio canonico.

1.6.5 Confronto tra Funzioni di Ranking Alternative

Per comprendere la sensibilità di NDCG alle inversioni di rango, si analizzi un caso di studio con 4 documenti $\{d_1, d_2, d_3, d_4\}$ aventi gradi di rilevanza:

$$r(d_1) = 0, \quad r(d_2) = 1, \quad r(d_3) = 2, \quad r(d_4) = 2 \tag{1.35}$$

Si pongano a confronto due funzioni di ranking concorrenti con la graduatoria di riferimento (*Ground Truth*):

- **Ground Truth (GT):** Ordinamento $\langle d_4, d_3, d_2, d_1 \rangle$ con vettore $\mathbf{r}^* = [2, 2, 1, 0]$:

$$\begin{aligned} IDCG_4 &= 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} + 0 \\ &= 2 + 2 + \frac{1}{1.5850} \approx 4 + 0.6309 = 4.6309 \\ NDCG_{GT} &= \frac{4.6309}{4.6309} = \mathbf{1.00} \end{aligned} \tag{1.36}$$

- **Ranking Function 1 (RF_1):** Ordinamento $\langle d_3, d_4, d_2, d_1 \rangle$ con vettore $\mathbf{r} = [2, 2, 1, 0]$. Sebbene d_3 e d_4 siano scambiati tra loro, poiché possiedono esattamente lo stesso grado di rilevanza ($r = 2$), l'inversione non genera alcuna penalizzazione:

$$\begin{aligned} DCG_{RF_1} &= 2 + \frac{2}{\log_2(2)} + \frac{1}{\log_2(3)} = 4.6309 \\ NDCG_{RF_1} &= \frac{4.6309}{4.6309} = \mathbf{1.00} \end{aligned} \tag{1.37}$$

- **Ranking Function 2 (RF_2):** Ordinamento $\langle d_3, d_2, d_4, d_1 \rangle$ con vettore $\mathbf{r} = [2, 1, 2, 0]$. In questo ordinamento, il documento subottimale d_2 ($r = 1$) scavalca indebitamente d_4 ($r = 2$) alla seconda posizione:

$$\begin{aligned} DCG_{RF_2} &= 2 + \frac{1}{\log_2(2)} + \frac{2}{\log_2(3)} + 0 \\ &= 2 + 1 + \frac{2}{1.5850} \approx 3 + 1.2618 = 4.2618 \\ NDCG_{RF_2} &= \frac{DCG_{RF_2}}{IDCG_4} = \frac{4.2618}{4.6309} \approx \mathbf{0.9203} \end{aligned} \tag{1.38}$$

Questo esempio dimostra con rigore la proprietà fondamentale di NDCG: la funzione punisce proporzionalmente le discrepanze qualitative, distinguendo gli scambi neutri tra elementi paritari dalle inversioni penalizzanti tra documenti di rilevanza eterogenea.

1.6.6 Quadro Sinottico Comparativo delle Metriche di Valutazione

La Tabella 1.5 riassume sinotticamente l'intero panorama delle metriche esaminate nel capitolo, evidenziandone la natura matematica, il focus operativo e gli ambiti d'uso raccomandati.

Metrica	Rilevanza	Focus Principale	Punto di Forza	Limite Strutturale
Precision (P)	Binaria	Purezza insieme	Intuitiva, definita su insiemi	Ignora richiamo e ranking
Recall (R)	Binaria	Completezza estrazione	Misura copertura assoluta	Ignora rumore (FP) e ranking
F_1**-Score**	Binaria	Equilibrio armonico	Punisce forti asimmetrie	Ignora la graduatoria
F_β**-Score**	Binaria	Ponderazione flessibile	Regolabile per dominio	Ignora la graduatoria
P@k	Binaria	Qualità top-k	Rispecchia la prima SERP	Non media bene se R varia
Recall@k	Binaria	Frazione R nei top-k	Valuta richiamo locale	Dipende fortemente da k
AP	Binaria	Area curva PR singola query	Sintesi globale dell'ordinamento	Complessa da calcolare su Web
MAP	Binaria	Macro-media su query suite	Tutela query di coda lunga	Assume rilevanza binaria
MAP@K	Binaria	Top-K su Web e RecSys	Elimina distorsione di $P@k$	Tronca la coda dei risultati
MRR	Binaria	Primo risultato utile	Ideale per factoid/known-item	Ignora tutti i risultati successivi
CG	Graduata	Somma guadagni cumulativi	Supporta scale multi-livello	Cieca rispetto all'ordinamento
DCG	Graduata	Sconto posizionale logaritmico	Modella attenzione visiva reale	Non normalizzato tra query
NDCG	Graduata	Normalizzazione $DCG/IDCG$	Standard assoluto ranking Web	Sensibile alla scelta del log/gain

Tabella 1.5: Quadro sinottico comparativo delle metriche di valutazione per l'Information Retrieval.

Capitolo 2

Efficienza Algoritmica sulle Architetture CPU Moderne

2.1 Il Ruolo dell'Efficienza nell'Information Retrieval

Lo sviluppo di sistemi di reperimento dell'informazione su scala globale si articola lungo due dimensioni complementari e in costante tensione tra loro [1, 2]:

1. **Efficacia (*Effectiveness*):** massimizza la qualità semantica dei risultati restituiti e la soddisfazione dell'utente (*User Satisfaction*), adoperando modelli di ranking sofisticati, reti neurali e tecniche avanzate di Natural Language Processing tipicamente sviluppate in ambienti ad alto livello di astrazione (Python, PyTorch).

2. **Efficienza (*Efficiency*):** minimizza i costi computazionali, i consumi energetici e l'infrastruttura hardware necessaria per servire milioni di interrogazioni al secondo, ottimizzando il codice fino all'ultimo ciclo di clock della CPU e al singolo bit di memoria occupato.

Chi progetta modelli ad alto livello non può ignorare i vincoli fisici dell'hardware sottostante per non creare soluzioni irrealizzabili in produzione; parallelamente, chi ottimizza a basso livello deve comprendere la semantica dei dati per preservare l'efficacia del recupero.

2.1.1 I Tre Pilastri dell'Efficienza nei Sistemi IR

L'efficienza algoritmica nei moderni sistemi di reperimento dell'informazione non è un mero esercizio di micro-ottimizzazione, ma un requisito abilitante che si fonda su tre pilastri:

1. **Esperienza Utente (*User Experience*):** Nei motori di ricerca Web contemporanei, la risposta a una query deve essere erogata entro una frazione minima di secondo (latenze tipiche inferiori a 50–100 ms). Ritardi percepibili inducono frustrazione e spingono l'utente a migrare verso servizi concorrenti. Inoltre, ridurre la latenza di computazione consente di eseguire algoritmi di ranking e filtraggio più complessi ed efficaci *a parità di budget temporale prestabilito* (*Time Budget*).

2. **Scalabilità Operativa (*Scalability*):** Le collezioni Web e aziendali contano decine di miliardi di documenti e gestiscono picchi di centinaia di migliaia di query al secondo. Il fabbisogno infrastrutturale di un motore di ricerca è approssimabile dalla relazione:

$$\text{Numero di Macchine} = \left\lceil \frac{\text{Volume Totale dei Dati e delle Query}}{\text{Capacità Operativa del Singolo Nodo}} \right\rceil \tag{2.1}$$

   Un algoritmo scalabile richiede una crescita lineare dei nodi al raddoppiare dei dati; un algoritmo inefficiente rischia invece una crescita superlineare insostenibile. Poiché i dati operativi devono risiedere in RAM ed essere replicati su molteplici macchine per garantire la tolleranza ai guasti (*fault tolerance*), qualsiasi inefficienza di memoria o di calcolo viene amplificata dal fattore di replica dell'infrastruttura.

3. **Sostenibilità Ambientale ed Economica (*Environmental Sustainability*):** I moderni data center assorbono quantitativi colossali di energia elettrica, sia per alimentare i processori ad altissima densità sia per i complessi impianti di dissipazione termica (*cooling systems*). Un risparmio marginale del 5% nell'utilizzo della CPU si traduce in milioni di kilowattora risparmiati all'anno e in un abbattimento diretto dell'impronta di carbonio (*carbon footprint*).

2.2 La Gerarchia di Memoria e i Principi di Località

Nei sistemi di elaborazione contemporanei, le prestazioni reali sono dominate dai tempi di accesso alla memoria piuttosto che dalla mera velocità aritmetica

dell'ALU: si parla storicamente di **Memory Wall** [9, 10].

2.2.1 Il Divario di Latenza tra CPU e DRAM

Mentre il modello teorico astratto della macchina RAM assume un costo di accesso uniforme e costante $\mathcal{O}(1)$ per qualsiasi indirizzo, la fisica del silicio impone un divario abissale tra la velocità di commutazione dei registri interni del processore e i tempi di risposta della memoria dinamica (DRAM). Per mitigare tale asimmetria, l'architettura dei calcolatori interpone una gerarchia di memorie cache via via più veloci, piccole e costose (Tabella 2.1).

Livello Hardware	**Latenza Tipica (ns)**	**Cicli CPU Equivalenti**	**Capacità Tipica**
Istruzione CPU (Registri / ALU)	0.5–1.0 ns	$\approx$ 1–2	$<$ 1 KB (interni al core)
L1I Cache (Instruction)	0.5 ns	$\approx$ 1 ciclo	32 KB per core
L1D Cache (Data)	0.5 ns	$\approx$ 1 ciclo	32 KB per core
L2 Cache (Dati + Istruzioni)	1.5 ns	$\approx$ 3–4 cicli	256 KB–1 MB per core
L3 Cache (LLC, Condivisa)	5.0 ns	$\approx$ 12–15 cicli	16–64 MB per package
RAM Principale (DRAM)	80.0 ns	$\approx$ 160–200+ cicli	64–512 GB
Storage Secondario (SSD NVMe / Disco)	500 000 ns (0.5 ms)	oltre un milione	8–30 TB

Tabella 2.1: Confronto quantitativo di latenze, cicli di clock e capacità nella gerarchia di memoria di un moderno processore server x86_64 [9, 10].

Definizione 2.2.1 (Algoritmo *Memory-Bound* vs. *CPU-Bound*)**.** Un algoritmo si definisce **memory-bound** quando il suo tempo di esecuzione totale è dominato dall'attesa del trasferimento dati dalla DRAM (stallo della CPU durante i cache miss), rendendo trascurabile il numero di operazioni matematiche eseguite. Si definisce **CPU-bound** quando tutti i dati necessari risiedono nelle cache ad alta velocità (L1/L2) e il fattore limitante è la capacità di calcolo o di throughput delle istruzioni del processore.

2.2.2 Architettura Fisica sul Chip e Disaccoppiamento L1I / L1D

Per evitare conflitti strutturali sul bus di memoria durante l'esecuzione parallela, la cache di primo livello è fisicamente sdoppiata:

- **L1I (*Instruction Cache*):** contiene unicamente il codice macchina prelevato dall'unità di *instruction fetch*.
- **L1D (*Data Cache*):** serve le richieste di lettura e scrittura degli operandi (`load` e `store`) emesse dalle ALU.

La cache L2 è anch'essa integrata all'interno del singolo core (fornendo un buffer intermedio a bassa latenza), mentre la cache L3 (denominata *Last Level*

Cache, LLC) risiede sul die del chip ma è condivisa dinamicamente tra tutti i core fisici del socket.

2.2.3 La Cache Line come Unità Atomica di Trasferimento

Proprietà 2.2.1 (Unità di Trasferimento Hardware: $B = 64$ Byte)**.** *I moderni processori non leggono né scrivono mai singoli byte o singole variabili isolate dalla memoria principale: l'unità atomica di trasferimento tra DRAM e cache è la* ***cache line****, la cui dimensione universale nelle architetture x86_64 ed ARM è pari a* $B = 64$ *byte (corrispondente a 8 interi a 64 bit).*

Quando un'istruzione accede a un indirizzo di memoria A, l'hardware carica obbligatoriamente l'intero blocco allineato:

$$\left[\left\lfloor \frac{A}{64} \right\rfloor \times 64, \quad \left\lfloor \frac{A}{64} \right\rfloor \times 64 + 63\right] \tag{2.2}$$

Se l'algoritmo legge un singolo intero a 64 bit (8 byte) ed effettua un salto casuale verso un'altra area della memoria, i restanti 56 byte (pari all'87.5% del traffico di bus assorbito) vengono irrimediabilmente sprecati.

2.2.4 Località Temporale, Spaziale e Hardware Prefetching

Un algoritmo sfrutta la gerarchia di memoria se e solo se esibisce una delle due proprietà di località [9]:

- **Località Temporale:** Quando una locazione di memoria viene acceduta, è altamente probabile che la stessa locazione venga richiesta nuovamente a breve termine. Le cache sfruttano questo principio implementando politiche di rimpiazzo (varianti di **LRU**, *Least Recently Used*) che mantengono in memoria i blocchi più recentemente referenziati.
- **Località Spaziale:** Quando una locazione viene acceduta, è altamente probabile che locazioni adiacenti vengano accedute nell'immediato futuro. Leggere sequenzialmente elementi contigui ammortizza il costo di caricamento della cache line su tutti i dati presenti nel blocco.

I processori integrano circuiti dedicati, detti **Hardware Prefetcher** [9, 10], che analizzano a runtime il flusso degli indirizzi richiesti per anticipare i trasferimenti di memoria:

- **Stream Prefetcher:** Rileva accessi puramente lineari $(A, A+64, A+128, \dots)$ e ordina preventivamente alla DRAM il caricamento delle linee successive nelle cache L1/L2, mascherando completamente la latenza di caricamento.
- **Stride Prefetcher:** Intercetta accessi a passo costante $(A, A+s, A+2s, \dots)$, caricando in anticipo le linee distanziate di s byte.
- **Pattern Casuali (*Pointer Chasing*):** Di fronte a salti casuali e non deterministici, nessun circuito predittivo è in grado di anticipare l'indirizzo bersaglio; la CPU è costretta ad arrestarsi a ogni iterazione, subendo per intero la latenza di 80 ns della RAM.

2.3 Caso di Studio: Scansione Sequenziale contro *Pointer Chasing*

Per quantificare sperimentalmente l'impatto della località e del prefetching, si esamina il seguente esperimento canonico. Si consideri un vettore `v` di dimensione N contenente una **permutazione casuale con un singolo ciclo**:

Definizione 2.3.1 (Permutazione con un Singolo Ciclo)**.** Una permutazione π dell'insieme $\{0, 1, \dots, N-1\}$ possiede un singolo ciclo se e solo se, partendo dall'indice iniziale 0, la catena ricorsiva $k_{t+1} = \pi(k_t)$ visita ogni singola cella del vettore esattamente una volta prima di ritornare a 0:

$$0 \to \pi(0) \to \pi(\pi(0)) \to \cdots \to \pi^{N-1}(0) \to 0 \tag{2.3}$$

senza generare sottocicli disgiunti o componenti isolate.

Si pongono a confronto due funzioni Rust che sommano tutti gli elementi di `v`:

Listing 2.1: Scansione sequenziale con località spaziale e prefetching.

```
pub fn sum(v: &[usize]) -> usize {
    let mut sum = 0;
    for &value in v.iter() {
        sum += value;
    }
    sum
}
```

Listing 2.2: Scansione disordinata per salti casuali (*pointer chasing*).

```rust
pub fn sum_by_jumping(v: &[usize]) -> usize {
    let mut sum = 0;
    let mut k = 0;
    for _ in 0..v.len() {
        sum += v[k];
        k = v[k]; // il valore letto determina l'
           indirizzo del passo successivo!
    }
    sum
}
```

Nel modello computazionale teorico RAM entrambe le routine eseguono esattamente N addizioni e N accessi a vettore, presentando identica complessità asintotica $\mathcal{O}(N)$. Sulla CPU reale, misurando il tempo medio per singolo elemento al crescere della taglia N, emergono divergenze impressionanti (Tabella 2.2).

Elementi (N)	Dimensione	Livello Cache Ospitante	`sum()`	`sum_by_jumping()`	Rallentamento
256	2 KiB	Cache L1 (32 KB)	0.1 ns	1.3 ns	13×
8 192	64 KiB	Cache L2 (256 KB)	0.1 ns	2.5 ns	25×
262 144	2 MiB	Cache L3 (16 MB)	0.2 ns	13.4 ns	67×
16 777 216	128 MiB	Memoria RAM (DRAM)	0.4 ns	84.9 ns	212×

Tabella 2.2: Benchmark comparativo del tempo per elemento al variare della taglia del vettore [9].

2.3.1 Strategie di Ottimizzazione nel Pointer Chasing

Esempio 2.3.1 (Challenge 2: Sfruttare la Cache Line nel *Pointer Chasing*)**.** Come ammortizzare l'attesa di 85 ns della RAM effettuando comunque salti disordinati? Se raggruppiamo gli elementi in blocchi da 64 byte (8 interi `usize`), possiamo saltare casualmente da un blocco all'altro, ma accumulare tutti gli 8 valori della linea prima di passare al blocco successivo:

```rust
pub fn sum_by_jumping_cache_lines(v: &[usize]) -> usize
   {
    let mut sum = 0;
    let mut k = 0;
    let num_blocks = v.len() / 8;
    for _ in 0..num_blocks {
        let base = k * 8;
        // La prima lettura carica forzatamente tutti i
           64 byte in L1
```

```
8           sum += v[base] + v[base+1] + v[base+2] + v[base
                +3]
9               + v[base+4] + v[base+5] + v[base+6];
10          k = v[base+7] % num_blocks; // puntatore al
                prossimo blocco casuale
11      }
12      sum
13  }
```

Il cache miss di 85 ns viene pagato una sola volta per blocco e suddiviso su 8 addizioni: il costo scende a $\approx \frac{85\,\text{ns}}{8} \approx 10.6\,\text{ns}$ per elemento, con un'accelerazione di circa $8\times$ rispetto al salto singolo.

Esempio 2.3.2 (Challenge 1: Eliminare il Lieve Rallentamento di `sum()` su Grandi Dati)**.** In Tabella 2.2, `sum()` passa da 0.1 ns a 0.4 ns quando $N = 16\text{M}$ perché l'hardware prefetcher, pur saturando la banda del bus, non riesce ad alimentare l'ALU a pieno regime (il tempo di calcolo di una somma è inferiore al tempo di trasferimento dalla DRAM). Se arricchiamo il corpo del ciclo con operazioni aritmetiche più dense su ciascun elemento (ad esempio molteplici moltiplicazioni, polinomi o hashing), il tempo computazionale supera la latenza di caricamento ($T_{\text{comp}} \gg T_{\text{load}}$): il sistema diviene interamente **CPU-bound** e la curva dei tempi di `sum()` rimane perfettamente costante al variare di N.

2.4 Parallel Computing e Forme di Parallelismo nella CPU

Per incrementare le prestazioni computazionali, l'architettura dei calcolatori moderni non si limita a innalzare la frequenza di clock, ma sfrutta molteplici forme di **Parallel Computing** [9, 10].

Parallelismo a Livello di Server: Il Paradigma *Embarrassingly Parallel*

Nei motori di ricerca su larga scala, la gestione simultanea di migliaia di query non richiede complesse sincronizzazioni tra i core per una singola interrogazione: si adotta il **parallelismo triviale** (*embarrassingly parallel*), assegnando ciascuna query a un thread isolato su un core dedicato. Non essendovi condivisione di memoria o contese di lock, si azzerano gli overhead di coordinamento e i problemi di coerenza della cache.

All'interno del **singolo core**, la CPU sfrutta simultaneamente tre livelli di parallelismo microscopico:

1. **Pipelining:** Sovrappone l'esecuzione temporale di istruzioni contigue, suddividendo il ciclo di vita dell'istruzione in stadi indipendenti.
2. **Processori Superscalari:** Integrano molteplici unità di calcolo parallele (*Execution Ports*) all'interno del medesimo core per elaborare più istruzioni indipendenti nello stesso ciclo di clock ($IPC > 1$).
3. **SIMD (*Single Instruction, Multiple Data*):** Esegue la medesima operazione su un intero vettore di dati paralleli mediante registri dedicati di ampiezza fino a 512 bit.

2.4.1 Anatomia della Pipeline a Cinque Stadi

Il pipelining scompone l'esecuzione di ogni istruzione macchina in una catena di montaggio sequenziale. Nel modello didattico classico si distinguono 5 stadi (Figura 2.1):

1. **IF (*Instruction Fetch*):** Prelievo dell'istruzione binaria dalla cache L1I tramite il registro *Program Counter* (PC).
2. **ID (*Instruction Decode*):** Decodifica dell'istruzione in μ-ops e lettura degli operandi dai registri architetturali.
3. **EX (*Execute*):** Esecuzione del calcolo nelle unità aritmetico-logiche (ALU) o calcolo dell'indirizzo effettivo di memoria.
4. **MEM (*Memory Access*):** Accesso alla cache L1D per istruzioni di caricamento (`load`) o scrittura (`store`).
5. **WB (*Write-Back*):** Scrittura del risultato definitivo nei registri di destinazione e ritiro (*retirement*) dell'istruzione.

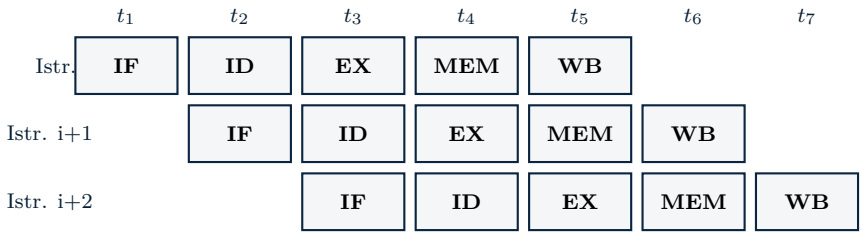


Figura 2.1: Esecuzione sovrapposta nella pipeline a 5 stadi: a ogni ciclo di clock t, ciascuno stadio hardware elabora una fase di un'istruzione differente.

2.4.2 Latenza, Throughput e Profondità della Pipeline

Sia k il numero di stadi della pipeline e τ il periodo del ciclo di clock.

- La **Latenza** $L = k \cdot \tau$ è il tempo impiegato da una singola istruzione per attraversare l'intera catena di elaborazione.
- Il **Throughput** a regime $W = 1/\tau$ misura il numero di istruzioni ritirate per unità di tempo: idealmente, a ogni ciclo di clock un'istruzione completa lo stadio di WB.
- Il periodo minimo di clock è imposto dal ritardo dello stadio circuitale più lento:

$$\tau \geq \max_{1 \leq j \leq k} \left(t_{\text{stadio}_j} \right) \tag{2.4}$$

Per aumentare la frequenza di clock (ridurre τ ed elevare i gigahertz), i progettisti aumentano la profondità k della pipeline (portandola tipicamente a 15–20 stadi nei processori x86_64 moderni come Intel Skylake o AMD Zen), suddividendo ogni operazione in micro-fasi circuitali estremamente brevi. Il prezzo da pagare per pipeline molto profonde risiede nella grave dilatazione della penalità di svuotamento (*pipeline flush*) in caso di errore di predizione dei salti.

2.5 Esecuzione Superscalare e Interleaving Aritmetico

Un processore si definisce **superscalare** quando ha la capacità di emettere, eseguire e ritirare più di un'istruzione per ciclo di clock all'interno del medesimo core ($IPC > 1$, *Instructions Per Cycle*) [9].

I core ad alte prestazioni integrano multiple unità funzionali parallele collegate a una serie di porte di esecuzione indipendenti (*Execution Ports*):

- Multiple ALU aritmetico-logiche per operazioni su interi e logica binaria;
- Moltiplicatori e divisori hardware dedicati;
- Porte di generazione degli indirizzi e unità di caricamento/scrittura (`load`/`store`);
- Unità vettoriali SIMD per calcolo in virgola mobile e matriciale.

2.5.1 L'Esperimento dell'*Interleaving* Aritmetico

Si consideri un set di benchmark sperimentale in cui si accumulano riduzioni su un vettore di interi a 64 bit `&[u64]`:

Listing 2.3: Funzioni di benchmark: incremento delle operazioni indipendenti nel ciclo.

```
pub fn do_stuff(v: &[u64]) -> u64 {
    let mut prod = 1;
    for &value in v.iter() {
        prod *= value;
    }
    prod
}

pub fn do_more_stuff_2(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        prod *= value;
        sum += value;
        xor ^= value;
    }
    (prod, sum, xor)
}

#[inline]
pub fn do_more_stuff_3(v: &[u64]) -> (u64, u64, u64, u64
    ) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    let mut sum2 = 0;
    for &value in v.iter() {
        prod *= value;
        sum += value;
        xor ^= value;
        sum2 += value;
    }
    (prod, sum, xor, sum2)
}
```

I risultati sperimentali mostrano un fenomeno controintuitivo (Tabella 2.3):

Funzione	Operazioni Eseguite per Iterazione	Tempo Medio per Iterazione
`do_stuff()`	1 moltiplicazione	0.4–1.1 ns
`do_more_stuff_2()`	3 operazioni (moltiplicazione, somma, XOR)	**0.4** ns
`do_more_stuff_3()`	4 operazioni (moltiplicazione, 2 somme, XOR)	**0.5** ns

Tabella 2.3: Tempi per iterazione: eseguire tre o quattro calcoli indipendenti richiede lo stesso tempo di uno solo grazie all'architettura superscalare.

Spiegazione fisica: Le variabili di accumulo `prod`, `sum` e `xor` sono completamente disaccoppiate tra loro. La CPU superscalare rileva l'assenza di dipendenze e invia contemporaneamente la moltiplicazione a una porta di calcolo complessa, la somma a una prima porta ALU e lo XOR a una seconda porta ALU. Le tre operazioni vengono completate **in parallelo nello stesso identico ciclo di clock**: il lavoro computazionale triplica, ma il tempo di esecuzione resta invariato (0.4 ns).

2.6 Pipeline Hazards: Tipologie e Risoluzione

Un **Pipeline Hazard** è una condizione architetturale che impedisce all'istruzione successiva di iniziare la propria esecuzione nel ciclo di clock immediatamente seguente, costringendo il processore a inserire bolle di stallo (*pipeline bubbles* o *stalls*) che degradano il throughput [9, 10].

Si distinguono tre categorie fondamentali di hazard:

2.6.1 Hazard Strutturali (*Structural Hazards*)

Si verificano quando due o più istruzioni pronte per l'esecuzione richiedono simultaneamente la medesima risorsa hardware fisica (ad esempio una specifica porta ALU o un'unità di divisione in virgola mobile non completamente pipelined). Rappresentano limiti fisici invalicabili del silicio, non risolvibili se non dilatando l'hardware.

2.6.2 Data Hazards (*Dipendenze di Dato, RAW*)

Si verificano quando un'istruzione necessita del valore prodotto da un'istruzione precedente che non ha ancora completato il proprio stadio di esecuzione o di scrittura nei registri (*Read-After-Write*, RAW).

Si analizzi la funzione `do_more_stuff_4()`, che introduce una rigida catena seriale di dipendenze:

Listing 2.4: Catena di dipendenze seriali: ogni operazione dipende dal risultato della precedente.

```
pub fn do_more_stuff_4(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        sum += value;        // Calcola 'sum'
        prod *= sum;         // ATTENDE 'sum'!
        xor ^= prod;         // ATTENDE 'prod'!
        sum += xor;          // ATTENDE 'xor'!
    }
    (prod, sum, xor)
}
```

Il Crollo di Throughput causato dai Data Hazard

Nella funzione `do_more_stuff_4()`, la moltiplicazione non può iniziare finché la somma non è pronta; lo XOR non può iniziare prima del completamento della moltiplicazione; e la somma finale deve attendere lo XOR. La pipeline non può sovrapporre i calcoli: il tempo per iterazione balza da **0.4** ns a **1.7** ns (un rallentamento di oltre 4×).
Soluzione Algoritmica: Ristrutturare l'algoritmo spezzando le catene critiche di dipendenza o applicando l'*interleaving* con operazioni disaccoppiate.

2.6.3 Control Hazards (*Dipendenze di Controllo*)

Si verificano in corrispondenza di istruzioni di salto condizionato (`if`, salti assembly `JNE`, `JLE`), in cui il processore non può determinare quale blocco di codice caricare finché la condizione logica non viene calcolata nello stadio EX. Di fronte a questo dilemma, la CPU è costretta a speculare.

2.7 Branch Prediction ed Esecuzione Speculativa

Per evitare che la pipeline si arresti a ogni salto condizionato, i processori integrano un componente ad altissima complessità: il **Branch Predictor** [9, 10].

Il predittore prova a indovinare quale ramo verrà intrapreso prima ancora che la condizione sia valutata:

- **Esecuzione Speculativa:** La CPU preleva ed esegue provvisoriamente le istruzioni del ramo predetto.
- **Predizione Corretta:** L'esecuzione procede a piena velocità senza alcuno stallo.
- **Errore di Predizione (*Branch Misprediction*):** Se la predizione si rivela errata, la CPU deve cancellare tutti i risultati provvisori calcolati ed eseguire il **flush completo della pipeline**, ricaricando le istruzioni corrette dal punto di salto.

Proprietà 2.7.1 (Costo del Branch Misprediction)**.** *La penalità in cicli di clock di un errore di predizione è pari al numero di stadi della pipeline tra il fetch (IF) e la verifica dell'istruzione nello stadio di esecuzione (EX), tipicamente* ***15–20 cicli di clock sprecati*** *per ogni singolo errore!*

2.7.1 L'Esperimento dei Dati Ordinati e Alternati

Si analizzi la funzione `do_more_stuff_5()`, che inserisce un salto condizionato all'interno del ciclo:

Listing 2.5: Presenza di salto condizionato all'interno del loop.

```
pub fn do_more_stuff_5(v: &[u64]) -> (u64, u64, u64) {
    let mut prod = 1;
    let mut sum = 0;
    let mut xor = 0;
    for &value in v.iter() {
        if value > 50 { // Salto condizionale!
            sum += value;
            prod *= sum;
            xor ^= prod;
            sum += xor;
        }
    }
    (prod, sum, xor)
}
```

Eseguendo la funzione su un vettore contenente valori estratti uniformemente in $[0, 99]$ sotto tre diverse disposizioni, emergono i risultati di Tabella 2.4.

Disposizione dei Dati in v	Tempo per Iterazione	Comportamento del Branch Predictor
`do_more_stuff_4()` (nessun `if`)	1.7 ns	Assenza di salti condizionati
`do_more_stuff_5()` (dati casuali)	**3.5** ns	Ramo impredicibile: fallimento nel $\approx 50\%$ dei casi con continui flush
`do_more_stuff_5()` (dati ordinati)	**1.2** ns	Predizione perfetta: solo 1 misprediction al punto di transizione > 50
`do_more_stuff_5()` (dati alternati)	**1.1** ns	Pattern periodico riconosciuto dai registri di shift storico: 0% misprediction

Tabella 2.4: Tempo medio per iterazione di `do_more_stuff_5()` in funzione della predicibilità dei dati [10].

- Con **dati casuali**, il salto è analogo al lancio di una moneta: la CPU sbaglia la metà delle volte, pagando costantemente la penalità di 15–20 cicli di flush ($\Longrightarrow$ 3.5 ns).
- Con **dati ordinati**, il predittore assume che il salto non sia preso per tutta la prima metà e sia sempre preso per la seconda: commette un solo errore nell'intero vettore, impiegando solo 1.2 ns (meno della versione senza condizione, poiché metà degli elementi viene saltata a costo zero!).
- Con **dati alternati** (un valore > 50, uno ≤ 50), i predittori hardware integrano tabelle storiche dei pattern a due livelli (*Branch History Shift Registers*): identificata l'alternanza periodica, il circuito anticipa la scelta con precisione assoluta ($\Longrightarrow$ 1.1 ns).

2.7.2 Tecniche *Branchless* e Condizione del Ciclo `for`

Quando i dati non sono ordinabili né prevedibili, l'unica soluzione per evitare i disastrosi stalli di misprediction consiste nell'eliminare fisicamente il salto condizionato (**Branchless Programming**), trasformando la selezione logica in un'espressione aritmetica:

```
1 // Versione branchless: converte il predicato booleano
     in 1 oppure 0
2 let mask = (value > 50) as u64;
3 sum += value * mask;
```

Il compilatore traduce l'espressione in istruzioni condizionali prive di salto (quali `CMOV` o istruzioni vettoriali mascherate): il processore esegue sempre il calcolo, ma la pipeline procede in modo lineare e privo di flush.

Nota 2.7.1 (Perché il Salto di Fine Ciclo `for` non Danneggia le Prestazioni)**.** Ogni ciclo `for` termina con un'istruzione di salto condizionato per ritornare all'inizio. Questo salto non rallenta la CPU perché è intrinsecamente sbilanciato: su N iterazioni il salto viene preso $N - 1$ volte consecutive ed è non

preso solo all'uscita finale. Il branch predictor apprende istantaneamente la regolarità, commettendo un unico errore su milioni di passi.

2.8 Vettorizzazione SIMD e la Risoluzione del Mistero di L1 Cache

La terza forma cardine di parallelismo interno alla CPU è rappresentata dal paradigma **SIMD** (*Single Instruction, Multiple Data*) [9, 10].

2.8.1 Registri Vettoriali e Famiglie di Estensioni x86

Le moderne CPU supportano estensioni del set di istruzioni progettate per operare su registri speciali ad altissima capacità:

- **MMX / SSE (Streaming SIMD Extensions):** Registri `XMM` a 128 bit.
- **AVX / AVX2 (Advanced Vector Extensions):** Registri `YMM` a 256 bit.
- **AVX-512:** Registri `ZMM` a 512 bit (in grado di alloggiare contemporaneamente 64 interi a 8 bit o 8 numeri a virgola mobile a 64 bit).

I registri vettoriali vengono idealmente suddivisi in corsie parallele (*vector lanes*): una singola istruzione macchina (ad esempio `vpaddq`) comanda l'esecuzione simultanea della medesima operazione algebrica su tutti gli elementi delle corsie parallele, moltiplicando linearmente il throughput computazionale (Figura 2.2).

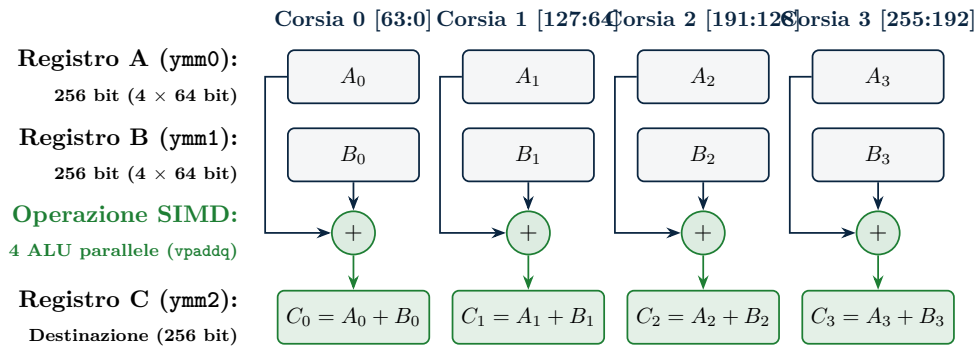


Figura 2.2: Schema concettuale di operazione SIMD vettoriale a 256 bit: quattro addizioni a 64 bit eseguite simultaneamente in un unico ciclo di clock.

2.8.2 Auto-Vettorizzazione del Compilatore contro Intrinsics

Lo sfruttamento di SIMD nel software segue due strade:

1. **Auto-vettorizzazione da parte del Compilatore:** Compilatori ottimizzanti moderni (LLVM/Rust, GCC, Clang con flag `-O3`) analizzano autonomamente i cicli for privi di effetti collaterali e traducono le sequenze scalari in istruzioni vettoriali SIMD.
    - *Vantaggi:* Totale trasparenza, nessun intervento manuale sul codice sorgente, portabilità cross-platform immediata.
    - *Limiti:* Il compilatore è conservativo; se il ciclo contiene condizioni complesse, accessi indiretti o rischio di aliasing di puntatori, l'auto-vettorizzazione fallisce.
2. **Intrinsics Hardware di Basso Livello:** Funzioni speciali C/Rust (ad es. libreria `core::arch::x86_64`) che mappano biunivocamente su specifiche istruzioni assembly SIMD (quali `_mm256_loadu_si256`, `_mm256_add_epi64`).
    - *Vantaggi:* Controllo totale sulle strutture dati e possibilità di ottimizzare algoritmi complessi non identificati dal compilatore.
    - *Limiti:* Richiede una profonda padronanza micro-architetturale e rende il codice rigidamente vincolato allo specifico set di istruzioni della CPU bersaglio.

2.8.3 La Rivelazione Finale: Perché `sum()` valeva 0.1 ns in L1 Cache?

A conclusione della trattazione sull'efficienza della CPU, si ricollega l'intero percorso didattico all'esperimento iniziale.

La Risoluzione dell'Enigma: L'Analisi con Compiler Explorer

Nella Sezione 2.3, per $N = 256$ elementi (vettore di appena 2 KiB, interamente contenuto nella velocissima cache L1D):

`sum()` impiegava **0.1** ns contro i **1.3** ns di `sum_by_jumping()` (13× di differenza!)

Poiché entrambi i vettori risiedevano interamente in cache L1, il divario **non poteva dipendere dai cache miss della RAM**! Qual era dunque la causa fisica di tale differenza abissale?
Ispezionando l'assembly generato tramite **Compiler Explorer** (Godbolt), emerge la verità:

1. **`sum()` è stata auto-vettorizzata con AVX dal compilatore!** Il ciclo non compie addizioni scalari elemento per elemento: carica vettori a 256 bit e calcola **quattro somme da 64 bit simultaneamente in un singolo ciclo di clock** (`vpaddq`), srotola il loop (*loop unrolling*) ed esegue una sola somma orizzontale di riduzione al termine.

2. **`sum_by_jumping()` è strutturalmente non vettorizzabile!** La dipendenza di dato stretta $k_{t+1} = v[k_t]$ impone che l'indirizzo della lettura successiva dipenda dal valore caricato dalla memoria nell'iterazione corrente. Il compilatore è impossibilitato a vettorizzare: la CPU esegue un'unica somma scalare seriale per volta, subendo inoltre la latenza di dipendenza RAW.

Questo risultato sancisce il principio metodologico fondamentale di questo capitolo:

Proprietà 2.8.1 (Principio di Regime Operativo)**.**

- *Quando i dati eccedono le cache e risiedono nella memoria principale DRAM, il programma è **memory-bound**: le prestazioni sono dominate dalla località spaziale, dalla dimensione della cache line (64 byte) e dall'efficacia dell'hardware prefetcher.*

- *Quando i dati sono compressi o ridotti al punto da risiedere interamente nelle cache L1/L2, il programma diventa **CPU-bound**: le prestazioni sono dominate dall'assenza di stalli della pipeline, dalla predicibilità dei salti e dalla capacità di sfruttare la vettorizzazione SIMD.*

2.9 Quadro Sinottico e Implicazioni per l'Information Retrieval

La Tabella 2.5 riassume schematicamente tutti i principi hardware esaminati e illustra le dirette implicazioni per la progettazione degli algoritmi e delle strutture dati nei motori di ricerca moderni (temi che verranno sviluppati nei successivi Capitoli 6, 7 e 8).

Principio Hardware	**Meccanismo / Vincolo Fisico**	**Impatto nella Progettazione di Sistemi IR**
Gerarchia di Memoria	Divario RAM vs CPU (80 ns vs 0.5 ns)	Compressione degli indici invertiti per farli risiedere in RAM
Cache Line (64 B)	Blocco atomico minimo di trasferimento	Layout a blocchi contigui (*Array of Structures* vs *SoA*)
Hardware Prefetcher	Anticipazione di stream contigui o a passo fisso	Scansione sequenziale delle posting list anziché alberi sparsi
Pipelining	Sovrapposizione di istruzioni a stadi sequenziali	Funzioni interne snelle senza chiamate a funzione indirette
Superscalarità	Molteplici porte di esecuzione parallele ($IPC > 1$)	Interleaving di accumuli indipendenti nel calcolo di BM25
Data Hazards	Dipendenze seriali strette (RAW)	Spezzamento delle catene di accumulo tramite registri multipli
Branch Prediction	Svuotamento della pipeline se il salto fallisce	Codice branchless nelle intersezioni AND e nel pruning WAND
Vettorizzazione SIMD	Calcolo simultaneo su registri fino a 512 bit	Decodifica vettoriale di liste compresse (SIMD-BP128, PForDelta)

Tabella 2.5: Quadro sinottico: dai vincoli micro-architetturali della CPU alle scelte ingegneristiche in Information Retrieval.

Capitolo 3

Proprietà del Linguaggio Naturale e Preprocessing del Testo

3.1 Proprietà Fondamentali del Linguaggio Naturale

L'utilizzo di un linguaggio strutturato complesso rappresenta uno dei tratti distintivi più profondi della cognizione umana [1, 11]. Nei sistemi di Information Retrieval, il testo non strutturato costituisce il mezzo primario con cui gli utenti formulano le proprie esigenze informative e con cui sono redatti i documenti della collezione. Tuttavia, a differenza dei dati memorizzati nei database relazionali, il linguaggio naturale non obbedisce a schemi rigidi prefissati e presenta caratteristiche intrinseche che rendono la sua elaborazione automatica una sfida ingegneristica e scientifica di primaria grandezza.

3.1.1 Migliaia di Simboli, Lessico e Vocabolario

A differenza dei sistemi formali o dei linguaggi di programmazione (caratterizzati da poche decine di parole chiave riservate), le lingue naturali umane poggiano su vocabolari sterminati comprendenti decine o centinaia di migliaia di simboli discreti (lemmi, forme flesse, gerghi specialistici e neologismi).

La crescita del vocabolario in una collezione documentale segue andamenti sublineari ma non asintotici (modellati dalla legge empirica di Heaps, $|\mathcal{V}| = kN^{\beta}$), evidenziando come l'universo delle parole distinte continui a espandersi indefinitamente all'aumentare dei documenti acquisiti.

3.1.2 La Legge di Zipf e il Principio del Minimo Sforzo

La distribuzione statistica della frequenza dei termini all'interno di un corpus documentale segue con straordinaria accuratezza la **Legge di Zipf** [1].

Definizione 3.1.1 (Legge di Zipf)**.** Sia data una collezione testuale e sia ordinato il vocabolario in senso decrescente di frequenza d'uso, assegnando a ciascun termine un rango $r \in \{1, 2, \ldots, |\mathcal{V}|\}$ (dove la parola più frequente possiede $r = 1$, la seconda $r = 2$, e così via). La frequenza empirica f del termine è inversamente proporzionale al suo rango di frequenza:

$$f \propto \frac{1}{r} \implies f(r) \approx \frac{c}{r} \tag{3.1}$$

dove c è una costante caratteristica del corpus considerata.

Passando alla scala logaritmica, l'Equazione (3.1) definisce una retta con pendenza -1:

$$\log f(r) \approx \log c - \log r \tag{3.2}$$

Questa distribuzione a legge di potenza genera una concentrazione sproporzionata delle masse di frequenza:

- Un nucleo minuscolo di parole ad altissima ricorrenza copre la maggioranza assoluta delle occorrenze testuali totali: ad esempio, nella collezione benchmark storica Reuters-21578, appena **313 parole generano oltre la metà delle 500 000 occorrenze complessive**!
- La parola più frequente in una lingua (ad es. l'articolo `"the"` in inglese o `"il"` in italiano) ricorre con frequenza doppia rispetto alla seconda, tripla rispetto alla terza e dieci volte superiore rispetto alla decima.

Proprietà 3.1.1 (Il Principio del Minimo Sforzo (*Principle of Least Effort*))**.** *Linguisti e scienziati cognitivi spiegano la genesi della legge di Zipf come un punto di equilibrio tra due forze contrapposte:*

1. ***Economia del Parlante (Speaker's Effort):*** *Chi emette il messaggio tende a minimizzare lo sforzo cognitivo e articolatorio, prediligendo un vocabolario ristretto composto da parole brevi, generiche e frequenti.*
2. ***Economia dell'Ascoltatore (Hearer's Effort):*** *Chi riceve il messaggio desidera minimizzare l'ambiguità interpretativa, preferendo un vocabolario vasto e specialistico, in cui ciascuna parola possiede un significato univoco e puntuale.*

La legge di Zipf rappresenta il compromesso evolutivo tra queste due esigenze.

Dalla dinamica di Zipf derivano ulteriori regolarità statistiche rilevanti per l'IR:

- *Relazione inversa tra frequenza e lunghezza:* Le parole più frequenti sono sistematicamente le più corte (`a`, `in`, `to`, `is`), mentre i termini tecnici a bassa frequenza presentano lunghezze morfologiche nettamente superiori.
- *Polisemia e frequenza:* Il numero di significati distinti m di una parola è approssimativamente proporzionale all'inverso della sua radice di frequenza ($m \propto \frac{1}{\sqrt{f}}$): le parole comuni tendono a essere polisemiche, mentre i termini rari sono altamente specifici.

3.1.3 Parole Rare, Hapax Legomena e Stopwords

L'analisi della coda lunga della distribuzione evidenzia due classi polari di termini con implicazioni opposte per i motori di ricerca:

Parole Rare e *Hapax Legomena*. I termini che compaiono una sola volta nell'intera collezione documentale prendono il nome filologico di **Hapax Legomena**. Essi costituiscono tipicamente tra il 40% e il 60% di tutte le parole distinte del vocabolario $\mathcal{V}$. Una parola può risultare rara per svariati motivi:

1. Termine autenticamente specialistico o scientifico (ad es. un composto chimico o una patologia rara);
2. Errore di battitura o refuso casuale di digitalizzazione (*typo*);
3. Identificatore sintetico univoco legato al documento (codice alfanumerico, matricola, URL o slug di un articolo).

Eliminare le parole rare dall'indice consente di abbattere lo spazio di memoria occupato dal vocabolario e di velocizzare il query processing; tuttavia, in un motore IR i termini rari sono spesso quelli dotati del più alto potere discriminante, poiché un utente che ricerca un codice specifico desidera esattamente quel documento.

Stopwords e Ponderazione Semantica. All'estremo opposto si collocano le parole vuote o funzionali (**Stopwords**): articoli, preposizioni, congiunzioni e particelle pronominali (`the`, `of`, `a`, `an`, `to`, `in`) che ricorrono ubiquitariamente in qualsiasi documento.

```
Testo grezzo:  "the president of the united states of america"
      Testo filtrato:  "president united states america"
```

Poiché compaiono in quasi tutti i testi, le stopword apportano un contributo semantico trascurabile alla discriminazione del contenuto. Tradizionalmente venivano escluse a priori dall'indicizzazione; tuttavia, la rimozione cieca può distruggere l'intento di ricerca in celebri casi limite:

- Ricerche di frasi esatte o titoli: `"to be or not to be"`, `"The Who"`, `"Vitamin A"`.
- Classificazione del sentiment: ad esempio, la stopword list storica di MySQL include vocaboli come `appreciate`, `serious` e `unfortunately`, che risultano invece fondamentali per comprendere la polarità affettiva di una recensione.

3.1.4 Sintassi Complessa e Semantica Composizionale

La comprensione del testo impone di analizzare come le parole si aggregano per generare significato:

- **Semantica Composizionale (*Compositional Semantics*):** Nelle strutture regolari del linguaggio, il significato globale di una proposizione è una funzione matematica composita del significato delle singole parole che la costituiscono e delle regole sintattiche che le legano. Esempi: *to buy a car*, *leggere un libro*.
- **Collocazioni (*Collocations*):** Coppie o gruppi di parole che compaiono con frequenza sensibilmente superiore rispetto al caso, con vincoli stilistici stabiliti dall'uso consolidato della lingua. Esempi: *make the bed* (e non *do the bed*), *saltare la lezione*.
- **Espressioni Idiomatiche (*Idioms*):** Frasi fatte non composizionali, il cui significato complessivo non può essere dedotto dalla somma dei singoli elementi costituenti. Esempi: *break a leg* (in bocca al lupo), *dalla padella nella brace*. Un sistema basato sul significato letterale delle singole parole fallisce inevitabilmente di fronte alle espressioni idiomatiche.

3.1.5 Sorgenti di Ambiguità nel Linguaggio Umano

Il linguaggio naturale è intrinsecamente ambiguo su molteplici livelli:

1. **Ambiguità Morfologica e di Categoria Grammaticale (*Part-of-Speech Ambiguity*):** La medesima forma grafica può svolgere funzioni grammaticali completamente differenti in contesti diversi. Si consideri il termine `beat`:

   - *Sostantivo:* `"Listen to this cool beat."`
   - *Verbo:* `"I will beat you at checkers!"`

2. **Ambiguità Semantica e Polisemia (*Word Sense Ambiguity*):** Una singola parola può riferirsi a concetti ontologici disgiunti a seconda dell'ambito disciplinare. Si consideri il vocabolo `interest`:

   - *Coinvolgimento cognitivo / curiosità:* "There is widespread interest on this scientific topic."
   - *Valore finanziario / tasso di rendimento:* `"The central bank raised the interest rate by 50 basis points."`

3. **Ambiguità Sintattica e Attaccamento Preposizionale (*Syntactic Ambiguity*):** L'ordine delle parole può generare molteplici alberi di derivazione sintattica validi:

   `"I saw the man with the telescope."`

   L'interlocutore ha osservato un uomo per mezzo di un telescopio, oppure ha avvistato un uomo che portava con sé un telescopio?

3.1.6 Conoscenza Condivisa del Mondo (*World Knowledge*)

La comunicazione verbale non esplicita mai tutti i passaggi logici: essa si fonda sull'assunto che chi ascolta condivida una vasta enciclopedia informale sul funzionamento della realtà fisica e sociale (*Shared World Knowledge*):

- `"I'm faster than Lewis Hamilton."`
  $\Longrightarrow$ Più veloce a fare cosa? Se stiamo correndo a piedi è plausibile; se stiamo gareggiando in automobile si tratta di un'iperbole sarcastica.

- "Alexis and Kate are sisters." vs. "Alexis and Kate are mothers."
  $\Longrightarrow$ La prima proposizione stabilisce una **relazione di parentela simmetrica** tra i due individui; la seconda attribuisce a ciascuna una medesima proprietà individuale (*essere genitrice*), ma non istituisce alcuna relazione biologica diretta tra loro!

- `"My mother is younger than me."`
  $\Longrightarrow$ Dal punto di vista della sintassi formale la frase è ineccepibile; tuttavia, la conoscenza del mondo biologico ci avverte istantaneamente che l'asserzione è impossibile.

- `"I was on a bike with a helmet."` vs. `"I was on a bike with an electric engine."`
  $\Longrightarrow$ L'ambiguità sintattica di superficie dell'attaccamento preposizionale (`with`) è identica, ma la nostra conoscenza fisica ci dice che il casco è indossato sulla testa del ciclista, mentre il motore elettrico è parte integrante del telaio del veicolo.

3.1.7 Dinamicità, Evoluzione Temporale e il Concetto di AI-Completeness

A differenza dei linguaggi artificiali, il linguaggio naturale umano:

- **È appreso da zero lungo l'intero arco vitale (*Learned along life*):** Non viene codificato per via genetica o cablato staticamente, ma acquisito per immersione contestuale.

- **Evolve continuamente (*Temporal Dynamicity*):** Il lessico si adatta alle rivoluzioni tecnologiche e ai mutamenti sociali generando nuovi vocaboli, metafore e slittamenti semantici:

  `airplane, quark, automobile, genocide, Tesla, I googled it,`
  `it's in the cloud, crowdfunded, to navigate,`
  `super-spreader, wear a mask`

Natural Language Understanding (NLU) e il Problema AI-Complete

La disciplina che mira a estrarre la semantica autentica dal testo prende il nome di **Natural Language Understanding** (NLU). In informatica teorica e intelligenza artificiale, la comprensione esaustiva del linguaggio naturale è considerata un problema **AI-Complete** (o *AI-hard*): per comprendere appieno una qualsiasi conversazione aperta in linguaggio naturale senza commettere errori di interpretazione contestuale, una macchina dovrebbe possedere un'intelligenza generale pari o superiore a quella umana, essendo in grado di percepire il mondo, formulare ragionamenti causali, comprendere le emozioni e integrare il senso comune.

3.2 Relazioni con Altri Tipi di Dati e Architettura nei Motori Web

Nel Web moderno l'Information Retrieval non opera sul testo in isolamento, ma orchestra l'elaborazione di molteplici tipologie informative eterogenee (Figura 3.1).



Figura 3.1: Integrazione multimodale delle sorgenti informative in un moderno motore di ricerca Web.

1. **Testo Non Strutturato vs. Dati Tabulari (Database Relazionali):** I database gestiscono dati rigidamente tipizzati e organizzati in tabelle (schemi relazionali, vincoli di chiave, algebra relazionale SQL). L'IR colma il divario elaborando testo libero privo di schema, in cui l'informazione è distribuita nella semantica delle frasi.

2. **Grafi di Collegamento e Knowledge Graph:** Il Web è innervato da grafi ipertestuali (*Web Link Graph*). I motori integrano il testo delle pagine con la topologia della rete tramite algoritmi quali il PageRank [1], e collegano le entità a basi di conoscenza strutturate (*Knowledge Graphs*).

3. **Come si collegano insieme in un Motore di Ricerca Web?**

   - *Anchor Text:* Il testo associato ai collegamenti ipertestuali entranti (*incoming links*) che puntano a una pagina viene indicizzato come parte integrante della pagina di destinazione, offrendo una sintesi esterna oggettiva del suo contenuto.
   - *Arricchimento Multimediale:* Immagini e video vengono indicizzati combinando il testo circostante, gli attributi alternativi (`alt`), i metadati `schema.org` e gli embedding vettoriali estratti da modelli visivi multimodali.

Definizione 3.2.1 (Concetto di *Processing Pipeline*)**.** Una **Processing Pipeline** è una sequenza deterministica di trasformazioni a stadi che riceve in ingresso flussi di byte non strutturati (pagine Web, PDF, feed RSS) e li converte progressivamente in unità semantiche discrete, normalizzate e indicizzabili.

3.3 Text Processing in Python: Dalla Rete ai Dati Puliti

La pipeline applicativa di elaborazione del testo grezzo si articola in tre macro-fasi operative implementate in Python [11]:

1. Download e acquisizione del contenuto Web grezzo tramite `urllib`;
2. Estrazione dei tag e pulizia dell'albero HTML con `BeautifulSoup`;
3. Normalizzazione linguistica avanzata con la suite `NLTK` (*Natural Language Toolkit*).

3.3.1 Scaricare Testo dal Web con `urllib`

Il modulo `urllib.request` della libreria standard di Python implementa le primitive per l'interazione client HTTP con i server Web:

Listing 3.1: Scaricamento di una pagina Web con `urllib.request`.

```
from urllib import request

url = "https://example.com/sample.html"
response = request.urlopen(url)
# Decodifica dello stream di byte grezzi in stringa UTF-8
html_raw = response.read().decode('utf-8')
print(html_raw[:120])
# Output tipico: '<!DOCTYPE html>\n<html lang="en-US">\n  <head>...'
```

3.3.2 Estrazione e Pulizia del Testo HTML tramite BeautifulSoup

Il documento HTML restituito dalla rete contiene frammenti di codice JavaScript, fogli di stile CSS, tag di formattazione e commenti che non costituiscono

contenuto testuale informativo. La libreria `BeautifulSoup` crea un albero sintattico DOM (*Document Object Model*) navigabile, consentendo di estrarre selettivamente il solo testo visibile:

Listing 3.2: Estrazione del testo pulito tramite `BeautifulSoup`.

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_raw, 'html.parser')
# Rimozione automatica dei tag di markup e isolamento del testo
clean_text = soup.get_text()
print(clean_text[:150])
```

3.3.3 NLTK: Tokenizzazione e Sentence Splitting

La libreria open source **NLTK** (*Natural Language Toolkit*) [11] costituisce il punto di riferimento accademico per il Natural Language Processing in ambiente Python.

Tokenizzazione delle Parole (*Word Tokenization*). Consiste nello scomporre un testo continuo in una sequenza di unità lessicali atomiche (*tokens*). Sebbene nei linguaggi occidentali lo spazio bianco funga da separatore naturale, una semplice suddivisione tramite `str.split()` fallisce di fronte alla punteggiatura, agli apostrofi e alle contrazioni:

Listing 3.3: Tokenizzazione con NLTK.

```
import nltk
from nltk.tokenize import word_tokenize

text = "Natural Language Processing isn't trivial in the U.S.A.!"
tokens = word_tokenize(text)
print(tokens)
# Output: ['Natural', 'Language', 'Processing', 'is', "n't", 'trivial', 'in', 'the', 'U.S.A.', '!']
```

L'algoritmo separa correttamente la contrazione `isn't` nei due token morfosintattici `is` e `n't`, isola il punto esclamativo finale e preserva intatto l'acronimo `U.S.A.`.

Segmentazione delle Frasi (*Sentence Splitting*). Dividere un documento nei suoi singoli periodi costituisce un compito non banale a causa

della polisemia del punto fermo: esso può indicare la conclusione di una frase, un'abbreviazione onorifica (*Dr. Smith*), un acronimo (*e.g.*, *etc.*) o un separatore decimale numerico (3.14). La funzione `sent_tokenize()` implementa un classificatore addestrato non supervisionato (algoritmo Punkt) per disambiguare i confini di frase:

Listing 3.4: Sentence splitting con `sent_tokenize`.

```
from nltk.tokenize import sent_tokenize

text = "Dr. Smith graduated from UniPi in 2024. He now
   works in IR."
sentences = sent_tokenize(text)
print(sentences)
# Output: ['Dr. Smith graduated from UniPi in 2024.', '
   He now works in IR.']
```

Esplorazione Quantitativa: Oggetti `Text` e `FreqDist`. NLTK mette a disposizione due strutture dati ad alte prestazioni per esplorare la statistica lessicale di un corpus:

- `FreqDist`: calcola la distribuzione di frequenza assoluta di ciascun termine (fondamento statistico per le leggi di Zipf e TF-IDF);
- `Text`: abilita funzioni di concordanza (*concordance*, che mostra tutte le occorrenze di una parola circondata dal proprio contesto visivo) e di estrazione di collocazioni frequenti.

3.3.4 Filtraggio delle Stopword in Python

NLTK include liste di stopword compilate manualmente per oltre venti lingue:

Listing 3.5: Rimozione delle stopword con NLTK.

```
from nltk.corpus import stopwords

stop_words_en = set(stopwords.words('english'))
tokens = ['the', 'president', 'of', 'the', 'united', '
   states']
filtered = [w for w in tokens if w.lower() not in
   stop_words_en]
print(filtered)
# Output: ['president', 'united', 'states']
```

3.3.5 Normalizzazione Morfologica: Stemming vs. Lemmatizzazione

Nei testi reali, le parole compaiono in molteplici varianti flesse (singolare/plurale, coniugazioni verbali, generi). L'IR adotta due strategie alternative per ricondurre le varianti alla medesima radice canonica:

Caratteristica	**Stemming (Algoritmo di Porter)**	**Lemmatizzazione (WordNet)**
Approccio	Euristico: troncamento brutale di prefissi/suffissi	Analisi morfosintattica formale con dizionario
Risultato	Spesso radici inesistenti nel vocabolario	Lemma reale appartenente alla lingua
Contesto POS	Ignorato completamente	Richiesto (*Part-of-Speech tag*)
Prestazioni	Computazionalmente istantaneo e leggero	Più costoso (lookup nel lessico WordNet)

Tabella 3.1: Confronto architetturale tra Stemming e Lemmatizzazione [11].

Listing 3.6: Stemming con `PorterStemmer` contro Lemmatizzazione con `WordNetLemmatizer`.

```
from nltk.stem.porter import PorterStemmer
from nltk.stem.wordnet import WordNetLemmatizer

stemmer = PorterStemmer()
lemmatizer = WordNetLemmatizer()

# Caso 1: Sostantivo plurale regolare
print(stemmer.stem('cars'))        # Output: 'car'
print(lemmatizer.lemmatize('cars'))# Output: 'car'

# Caso 2: Forma verbale irregolare ('was')
print(stemmer.stem('was'))                  # Output: 'wa
    ' (radice spuria inesistente!)
print(lemmatizer.lemmatize('was'))          # Output: 'wa
    ' (default: assume sostantivo)
print(lemmatizer.lemmatize('was', pos='v'))# Output: 'be
    ' (ricostruzione perfetta del lemma!)
```

3.4 Quadro Sinottico della Pipeline di Text Processing

La Tabella 3.2 sintetizza il flusso logico della processing pipeline, correlando gli stadi applicativi alle tecnologie software analizzate.

Stadio della Pipeline	Input di Elaborazione	Modulo / Algoritmo Python	Output Prodotto
1. Acquisizione Rete	Indirizzo URL target	`urllib.request`	Flusso di byte HTML grezzo decodificato in UTF-8
2. Rimozione Markup	Albero sintattico HTML	`bs4.BeautifulSoup`	Testo pulito privo di tag, script e fogli CSS
3. Sentence Splitting	Testo continuo non strutturato	`nltk.sent_tokenize`	Lista di singoli periodi logicamente separati
4. Tokenizzazione	Stringhe di frase	`nltk.word_tokenize`	Lista di token lessicali discreti e simboli
5. Rimozione Stopword	Token grezzi	`nltk.corpus.stopwords`	Lista di termini a elevata informativa semantica
6. Stemming / Lemmat.	Parole flesse	`PorterStemmer` / `WordNet`	Radici morfologiche unificate o lemmi canonici

Tabella 3.2: Quadro sinottico della pipeline di acquisizione e preprocessing del testo in Python.

Dal Testo Pulito alla Rappresentazione per il Retrieval

Con la normalizzazione morfologica si conclude la fase di pulizia e pre-elaborazione del testo grezzo. Nel Capitolo 4 si affronteranno le operazioni sulle collezioni di documenti e i modelli formali di rappresentazione testuale (Bag of Words, N-grammi di parole e caratteri, vettorizzazione one-hot), per poi analizzare il modello di reperimento Booleano tradizionale e i suoi limiti intrinseci.

Capitolo 4

Rappresentazioni del Testo, Modello Booleano ed Evoluzione del Retrieval

4.1 Operazioni Fondamentali su Collezioni di Testo

Nelle collezioni documentali reali (costituite da milioni di pagine Web o documenti eterogenei), la manipolazione del testo non può prescindere da tre operazioni computazionali ad alto impatto:

1. **Stemming:** Troncamento euristico dei suffissi e prefissi morfologici per ricondurre parole derivate alla medesima radice (es. *"connected"*, *"connecting"*, *"connection"* $\rightarrow$ *"connect"*). Riduce drasticamente la dimensionalità del vocabolario, ma può introdurre conflitti di *over-stemming* (fusione scorretta di parole semanticamente distinte) o *under-stemming* (mancata unificazione).

2. **Lemmatizzazione:** Analisi morfosintattica completa basata su dizionari grammaticali e riconoscimento del contesto (*Part-of-Speech tag*), volta a ricondurre ciascuna forma flessa al suo lemma autentico (es. *"was"*, *"is"* $\rightarrow$ *"be"*). Garantisce rigore semantico a fronte di un costo computazionale superiore.

3. **Rimozione delle Stopwords:** Filtraggio dei termini ad altissima frequenza e scarso contenuto informativo discriminante (articoli, preposizioni, congiunzioni). Riduce la dimensione fisica dell'indice invertito fino al 30–40% e accelera l'elaborazione delle interrogazioni congiuntive.

4.2 Rappresentazioni del Testo: Dal Testo alle Feature

Dalla sequenza temporale di token ripuliti, il sistema IR deve costruire una rappresentazione computazionale adatta a confrontare documenti e interrogazioni.

4.2.1 Il Modello Bag of Words (BoW): Esempi, Vantaggi e Svantaggi

Nel modello **Bag of Words** (Sacco di Parole), un documento viene modellato prescindendo totalmente dalla sintassi, dalla punteggiatura e dall'ordine relativo con cui i termini si susseguono, trattenendo unicamente l'insieme (*set*) o il multinsieme (*multiset*) dei vocaboli presenti [1]:

Listing 4.1: Costruzione del Bag of Words in Python.

```
from nltk.tokenize import word_tokenize

text = 'the president of the united states of america'
tokens = word_tokenize(text)
bow_set = set(tokens)
print(f"Numero token: {len(tokens)} -> Dimensione BoW: {
    len(bow_set)}")
# Output: Numero token: 8 -> Dimensione BoW: 6
# bow_set: {'america', 'of', 'president', 'states', 'the
    ', 'united'}
```

Vantaggi e Svantaggi del Bag of Words (Pros and Cons):

- **Vantaggi (Pros):**
  1. *Semplicità Computazionale ed Efficienza:* Riduce la complessità del linguaggio a operazioni insiemistiche o vettoriali elementari;
  2. *Bassa Impronta di Memoria:* Consente l'indicizzazione scalabile tramite posting list disaccoppiate dalla posizione;
  3. *Efficacia Statistica:* Cattura la focalizzazione tematica primaria del documento nella maggioranza dei compiti di ricerca per parole chiave.
- **Svantaggi (Cons):**

1. *Perdita Totale dell'Ordine Sintattico e Semantico:* Due frasi con significati diametralmente opposti risultano identiche se condividono gli stessi termini:

   $t_1 =$ `set(word_tokenize('I won, and thus you lose.'))`
   $t_2 =$ `set(word_tokenize('I lose, and thus you won.'))`

   In Python `t1 == t2` restituisce `True`, cancellando la contrapposizione semantica tra vittoria e sconfitta.

2. *Perdita della Frequenza nei Set Puri:* Nella formulazione booleana, la parola ripetuta dieci volte non si distingue da quella presente una volta sola.

4.2.2 Word N-Grammi: Preservazione dell'Ordine Locale, Pro e Contro

Per preservare le relazioni di prossimità e le locuzioni idiomatiche senza dover ricorrere a complessi analizzatori sintattici ad albero, l'IR estrae sequenze ordinate di n parole adiacenti, denominate **Word N-Grammi**:

- $n = 1$: Unigrammi (equivalente al vocabolario canonico delle singole parole);
- $n = 2$: Bigrammi di parole (es. *"united states"*, *"machine learning"*);
- $n = 3$: Trigrammi di parole (es. *"natural language processing"*).

Listing 4.2: Preservazione dell'ordine locale tramite bigrammi di parole in Python.

```
import nltk

t1_bigrams = set(nltk.ngrams(word_tokenize('I won, and
    thus you lose.'), 2))
t2_bigrams = set(nltk.ngrams(word_tokenize('I lose, and
    thus you won.'), 2))
print(t1_bigrams == t2_bigrams) # Output: False!
```

Il testo t_1 genera le feature `('won', 'and')` e `('you', 'lose')`, mentre t_2 produce `('lose', 'and')` e `('you', 'won')`: la discriminazione semantica viene ripristinata con successo.

Vantaggi e Svantaggi dei Word N-Grammi (Pros and Cons):

- **Vantaggi (Pros):** Preservano il contesto locale, discriminano concetti composti e locuzioni fisse, eliminano l'ambiguità delle parole polisemiche.
- **Svantaggi (Cons):** Comportano un'esplosione combinatoria del vocabolario (fino a $\mathcal{O}(|\mathcal{V}|^n)$), determinando matrici enormemente più sparse e problemi di sovradattamento statistico per $n \geq 3$.

4.2.3 Character N-Grammi: Tolleranza ai Refusi, Pro e Contro

I **Character N-Grammi** scompongono il testo in sequenze scorrevoli di n caratteri continui a livello subword, attraversando i confini delle parole. Questa rappresentazione è cruciale per conferire al sistema tolleranza rispetto a refusi di digitazione (*typos*) e flessioni morfologiche:

Listing 4.3: Tolleranza agli errori di digitazione con trigrammi di caratteri in Python.

```
# Confronto tra parola corretta ('rainbow') e refuso ('
   rainbaw')
t1_char = set(nltk.ngrams('rainbow', 3))
t2_char = set(nltk.ngrams('rainbaw', 3))

print("Trigrammi rainbow:", t1_char)
# {'r-a-i', 'a-i-n', 'i-n-b', 'n-b-o', 'b-o-w'}
print("Intersezione:", t1_char.intersection(t2_char))
# Output: {'r-a-i', 'a-i-n', 'i-n-b'} (3 trigrammi
   condivisi su 5!)
```

Nonostante l'errore di battitura invalidi il matching esatto della parola, la condivisione di 3 trigrammi su 5 consente al motore di quantificare la vicinanza ortografica e attivare il modulo dei suggerimenti (*"Forse cercavi: rainbow"*).

Vantaggi e Svantaggi dei Character N-Grammi (Pros and Cons):

- **Vantaggi (Pros):** Tolleranza ai refusi e rumore OCR; gestione nativa di parole fuori vocabolario (OOV); indipendenza da regole morfologiche in lingue agglutinanti (es. tedesco, turco, finlandese).
- **Svantaggi (Cons):** Perdita della semantica lessicale di alto livello; generazione di indici di grandi dimensioni con un numero altissimo di posting list corte.

4.3 Perché Serve una Rappresentazione Documentale: Verso il Vector Space Model

L'estrazione delle feature testuali solleva una domanda teorica fondamentale: **perché un motore di ricerca non può operare direttamente sulle stringhe grezze e necessita di una rappresentazione formale del documento?**

1. **Efficienza di Calcolo:** La scansione sequenziale di stringhe su collezioni di milioni di testi (*grep*) richiede tempi lineari insostenibili ($\mathcal{O}(|D| \cdot |q|)$), incompatibili con il vincolo di latenza interattiva sub-secondo.

2. **Spazio Matematico Comune:** Query e documenti appartengono originariamente a domini eterogenei (una query è una stringa breve e telegrafica di 2–3 parole; un documento è un testo strutturato di migliaia di parole). È indispensabile proiettare entrambe le entità all'interno di uno spazio geometrico condiviso.

3. **Base Computazionale per lo Scoring:** Una rappresentazione algebrica permette di definire funzioni continue di similarità basate su prodotti scalari e proiezioni vettoriali.

4.3.1 Binary One-Hot Encoding e la Sparsità Vettoriale

Dato l'universo delle feature lessicali distinte $\mathcal{F} = \{f_1, f_2, \ldots, f_{|\mathcal{F}|}\}$, ciascuna feature f_j viene mappata sull'asse ortogonale j-esimo mediante un vettore **One-Hot**:

$$\mathbf{e}_j = [0, \ldots, 0, \underbrace{1}_{j\text{-esima coordinata}}, 0, \ldots, 0]^T \in \{0,1\}^{|\mathcal{F}|}. \tag{4.1}$$

Un generico documento d viene conseguentemente rappresentato come combinazione lineare ponderata dei suoi vettori di feature:

$$\mathbf{v}(d) = \sum_{f_j \in d} w_{f_j,d}\, \mathbf{e}_j. \tag{4.2}$$

In virtù della vastità del lessico ($|\mathcal{F}| \geq 10^5$–10^6) e della brevità relativa del singolo testo, oltre il **99.9**% delle componenti del vettore documentale è rigorosamente pari a zero: la gestione efficiente di tale estrema sparsità costituisce il fondamento dell'Information Retrieval moderno.

4.4 Il Modello Booleano Tradizionale (Boolean Retrieval)

Il primo e più longevo modello formale di reperimento documentale è il **Modello Booleano**, fondato sulla teoria proposizionale classica e sull'algebra degli insiemi [1].

Sintassi e Criterio di Soddisfacibilità. Le query sono espressioni logiche costituite da termini uniti tramite gli operatori booleani fondamentali: `AND` (intersezione insiemistica $\cap$), `OR` (unione insiemistica $\cup$) e `NOT` (complemento insiemistico $\setminus$). Il criterio di selezione è deterministico e proposizionale:

$$\text{Matching}(q, d) = d \models q \implies \{0, 1\}. \tag{4.3}$$

Un documento appartiene all'insieme dei risultati se e solo se valuta a `True` la formula logica dell'interrogazione; in caso contrario, esso viene escluso.

Vantaggi e Svantaggi del Modello Booleano (Pros and Cons):

- **Vantaggi (Pros):**
  1. *Semantica Precisa e Prevedibile:* Gli operatori logici non ammettono ambiguità: un utente esperto sa esattamente perché un documento è stato incluso o scartato;
  2. *Efficienza Implementativa:* Si mappa direttamente su operazioni logiche bitwise a livello hardware sui vettori di incidenza o su fusioni ordinate delle posting list;
  3. *Ideale per Esperti di Dominio:* Costituisce tuttora lo standard nei sistemi informativi giuridici, nei repertori brevettuali e nella ricerca bibliografica accademica (PubMed).
- **Svantaggi e Limiti Strutturali (Cons):**
  1. *Assenza di Punteggio di Rilevanza (Graduatoria):* Il modello restituisce un insieme piatto e non ordinato di documenti conformi. Non vi è modo di identificare quale documento risponda meglio al bisogno informativo;
  2. *Incapacità di Riconoscere Match Parziali:* Se una query richiede tre termini in `AND` e un documento ne contiene due in modo brillante ma manca del terzo, esso viene scartato con punteggio zero;

3. *Complessità per l'Utente Comune:* Gli utenti ordinari del Web formulano query in linguaggio naturale libero (*free-text*) e non possiedono le competenze per strutturare espressioni logiche nidificate con parentesi;
4. *La Patologia del "Feast or Famine" (Banchetto o Carestia):* L'interazione con il modello booleano conduce quasi sistematicamente a due estremi disfunzionali:
   - **Carestia (*Famine*):** L'operatore `AND` restringe rigidamente i requisiti logici; un sinonimo imprevisto porta frequentemente a restituire 0 risultati;
   - **Banchetto (*Feast*):** L'operatore `OR` allarga eccessivamente la selezione, restituendo decine di migliaia di documenti senza alcun ordine di priorità qualitativa.

4.5 Come Estendere il Modello Booleano: Verso il Ranked Retrieval

Per superare la patologia del *Feast or Famine*, la disciplina dell'IR ha introdotto il concetto di **Ranked Retrieval**: abbandonare il giudizio binario proposizionale a favore di un **punteggio scalare continuo** $\text{Score}(q, d) \in \mathbb{R}$, ordinando i documenti per stima decrescente di pertinenza (Tabella 4.1).

Proprietà	Boolean Retrieval	Ranked Retrieval
Criterio di Matching	Esatto, proposizionale e binario ($d \models q \implies \{0,1\}$).	Continuo e scalare su numeri reali: $\text{Score}(q,d) \in \mathbb{R}$.
Formulazione Query	Espressioni logiche booleane vincolanti (`AND`, `OR`, `NOT`).	Linguaggio naturale libero (*free-text query*).
Output Fornito	Insieme piatto non ordinato di documenti conformi.	Graduatoria ordinata in modo decrescente per score stimato.
Esperienza Utente	Sindrome patologica di *Feast or Famine* (Banchetto o Carestia).	Restituzione istantanea e ordinata dei migliori k risultati (top-k, tipicamente $k \approx 10$).
Destinatari Tipici	Documentalisti professionisti, bibliotecari, giuristi, brevettisti.	Utenti generici del Web.

Tabella 4.1: Confronto analitico tra il modello Booleano classico e il modello Ranked Retrieval [1].

4.5.1 Landscape della Letteratura: Le Grandi Ere dell'Information Retrieval

L'evoluzione dei modelli di calcolo del punteggio continuo ha attraversato quattro grandi ere concettuali ed architetturali [1, 2]:

1. **Era Statistica (Anni '60 – Metà anni '90):** La rilevanza viene computata fondandosi puramente sulla statistica lessicale locale nei testi (*Term Frequency*, tf) e globale nel corpus (*Inverse Document Frequency*, idf), formalizzata nel Vector Space Model e nei primi modelli probabilistici. Vulnerabile al *keyword stuffing* sui dati Web aperti.

2. **Era Strutturale e PageRank (1998):** Con l'esplosione del Web, Brin e Page introducono l'analisi topologica dell'ipertesto (Web Graph). I link in ingresso vengono interpretati come voti di autorevolezza democratica, neutralizzando le manomissioni testuali locali.

3. **Era del Machine Learning e Learning-to-Rank (Anni 2000):** La complessità industriale impone di combinare centinaia di segnali

eterogenei (rilevanza lessicale BM25, PageRank, freschezza temporale, click-through rate, dwell time) tramite modelli supervisionati addestrati su graduatorie reali.

4. **Era Neurale e dei Modelli Densi (2018 – Oggi):** L'avvento dei Transformer e dei Large Language Model abilita il superamento del vocabolario lessicale discreto attraverso spazi di incorporamento denso (*dense retrieval* bi-encoder e cross-encoder) o modelli sparsi appresi (SPLADE).

4.6 Quadro Sinottico Comparativo

La Tabella 4.2 riassume le caratteristiche distintive delle rappresentazioni testuali e del modello di reperimento booleano esaminate in questo capitolo.

Rappresentazione / Modello	Unità Lessicale Fondamentale	Vantaggio Principale	Criticità / Limite
Bag of Words (BoW)	Unigrammi di termini ($n = 1$)	Semplicità ed efficienza lineare	Perdita totale della sintassi e dell'ordine
Word N-Grammi	Sequenze di n parole adiacenti	Preservazione del contesto locale	Esplosione combinatoria del vocabolario
Character N-Grammi	Frammenti subword di n caratteri	Tolleranza ai refusi e alle forme OOV	Perdita del significato lessicale integro
Binary One-Hot Vector	Vettore ortogonale in $\mathbb{R}^{\lvert\mathcal{F}\rvert}$	Formalizzazione geometrica rigorosa	Estrema sparsità ($> 99.9\%$ coordinate nulle)
Boolean Retrieval	Proposizioni logiche (`AND, OR, NOT`)	Trasparenza e controllo formale assoluto	Nessuna graduatoria (*Feast or Famine*)

Tabella 4.2: Quadro sinottico comparativo delle rappresentazioni testuali e del modello booleano.

Dalla Logica Proposizionale alla Geometria dello Spazio Vettoriale

Accertata l'insostenibilità del recupero binario booleano per compiti di ricerca general-purpose, il successivo Capitolo 5 affronta la formulazione matematica di **come** assegnare un punteggio continuo alle coppie query-documento, partendo dalle matrici di conteggio, dal coefficiente di Jaccard, dalla ponderazione TF-IDF, fino alla formalizzazione geometrica del Vector Space Model, al fallimento della distanza euclidea e alla similarità cosenica con la notazione SMART.

Capitolo 5

Vector Space Model, Ponderazione TF-IDF e Similarità Coseno

5.1 Dall'Estensione del Booleano alla Matrice di Incidenza Binaria

Per assegnare un punteggio a un documento, il primo passo evolutivo consiste nel superare la scansione testuale sequenziale strutturando la collezione in una rappresentazione matriciale esplicita [1].

Definizione 5.1.1 (Matrice di Incidenza Binaria)**.** Dato un vocabolario di termini unici $\mathcal{V} = \{t_1, t_2, \ldots, t_M\}$ e un corpus di N documenti $\mathcal{D} = \{d_1, d_2, \ldots, d_N\}$, la **Matrice di Incidenza Termine-Documento** $M \in \{0,1\}^{M \times N}$ è definita come:

$$M_{t,d} = \begin{cases} 1 & \text{se il termine } t \text{ compare nel documento } d, \\ 0 & \text{altrimenti.} \end{cases} \tag{5.1}$$

In tale matrice, ciascuna riga rappresenta il vettore di presenza di una parola su tutti i documenti, mentre ciascuna colonna rappresenta il profilo lessicale binario del singolo testo in $\{0,1\}^{|\mathcal{V}|}$. Le interrogazioni booleane corrispondono a veloci operazioni logiche bitwise (`AND`, `OR`, `NOT`) tra le righe.

5.2 Misure di Similarità Set-Based: Il Coefficiente di Jaccard

Un primo tentativo intuitivo di calcolare un punteggio continuo senza ricorrere a complessi spazi geometrici consiste nell'applicare il coefficiente di sovrapposizione insiemistica di Jaccard [1]:

Definizione 5.2.1 (Coefficiente di Jaccard)**.** Dati due insiemi di parole A e B (ad esempio i vocaboli distinti di una query q e di un documento d), la somiglianza di Jaccard è definita come:

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|}. \tag{5.2}$$

Il coefficiente è strettamente confinato nell'intervallo continuo $[0, 1]$, dove $J(A, A) = 1$ e $J(A, B) = 0 \iff A \cap B = \emptyset$.

5.2.1 Esempio Guidato: Il Paradosso della Lunghezza Documentale

Si consideri l'esempio didattico canonico:

- **Query** q**:** *"ides of march"*
  $Q = \{\text{ides}, \text{of}, \text{march}\}, \quad |Q| = 3.$
- **Documento** d_1**:** *"caesar died in march"*
  $D_1 = \{\text{caesar}, \text{died}, \text{in}, \text{march}\}, \quad |D_1| = 4.$
- **Documento** d_2**:** *"the long march"*
  $D_2 = \{\text{the}, \text{long}, \text{march}\}, \quad |D_2| = 3.$

Entrambi i testi condividono con la query un'unica parola comune (`march`):

$$Q \cap D_1 = \{\text{march}\} \implies |Q \cap D_1| = 1, \quad J(q, d_1) = \frac{1}{3 + 4 - 1} = \frac{1}{6} \approx \mathbf{0.166}, \tag{5.3}$$

$$Q \cap D_2 = \{\text{march}\} \implies |Q \cap D_2| = 1, \quad J(q, d_2) = \frac{1}{3 + 3 - 1} = \frac{1}{5} = \mathbf{0.200}. \tag{5.4}$$

Il Paradosso di Rilevanza del Coefficiente di Jaccard

Poiché $J(q, d_2) = 0.200 > J(q, d_1) = 0.166$, l'algoritmo dichiara d_2 più rilevante di d_1. Tuttavia, sotto il profilo tematico, d_1 descrive l'assassinio di Cesare (avvenuto storicamente alle idi di marzo), mentre d_2 tratta un evento completamente estraneo (la Lunga Marcia cinese). Il documento d_2 vince unicamente perché è più breve, subendo una minore espansione del denominatore dell'unione!

Vantaggi e Svantaggi del Coefficiente di Jaccard (Pros and Cons):

- **Vantaggi (Pros):** Estremamente intuitivo; computazionalmente immediato; restituisce un punteggio normalizzato continuo in $[0, 1]$.
- **Svantaggi e Limiti (Cons):**
  1. *Ignora la frequenza locale (tf):* un termine che ricorre cinquanta volte riceve lo stesso peso di uno presente una volta sola;
  2. *Ignora la rarità globale (df):* parole comuni e prive di valore informativo (es. *"of"*) pesano esattamente quanto termini discriminanti (es. *"ides"*);
  3. *Penalizza severamente i documenti lunghi ed esaustivi:* testi ricchi di dettagli gonfiano il denominatore $|A \cup B|$, venendo ingiustamente penalizzati rispetto a testi sintetici.

5.3 Frequenza dei Termini e Rilevanza: Term Frequency e Matrice di Conteggio

Per superare i limiti degli insiemi puri, la rappresentazione documentale evolve riconoscendo che la **frequenza di occorrenza** di una parola riflette la focalizzazione tematica del testo.

5.3.1 Matrice di Conteggio Termine-Documento (*Count Matrix*)

Si definisce la matrice dei conteggi grezzi $C \in \mathbb{N}^{|\mathcal{V}| \times N}$, dove ogni cella contiene la frequenza locale:

$$C_{t,d} = \mathrm{tf}_{t,d} = \text{numero di occorrenze del termine } t \text{ nel documento } d. \quad (5.5)$$

La Tabella 5.1 riporta il celebre esempio shakespeariano analizzato nel testo di riferimento [1].

Termine	Antony & Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth
Antony	157	73	0	0	0	0
Brutus	4	157	0	1	0	0
Caesar	232	227	0	2	1	1
Calpurnia	0	10	0	0	0	0
Cleopatra	57	0	0	0	0	0
mercy	2	0	3	5	5	1
worser	2	0	1	1	1	0

Tabella 5.1: Matrice di conteggio dei termini (*Count Matrix*) su una selezione di opere di Shakespeare [1].

5.3.2 Rendimenti Marginali Decrescenti e Ponderazione Log-TF

La rilevanza non cresce linearmente con la frequenza grezza: un testo con 10 occorrenze di una parola chiave non è dieci volte più rilevante di uno con 1 sola occorrenza. La prima comparsa attesta il tema trattato; le successive ne confermano la centralità con un apporto informativo marginale decrescente. Per modellare tale principio, la letteratura adotta la trasformazione sublineare logaritmica:

Definizione 5.3.1 (Ponderazione Log-Frequency (wf))**.**

$$wf_{t,d} = \begin{cases} 1 + \log_{10}(\mathrm{tf}_{t,d}) & \text{se } \mathrm{tf}_{t,d} > 0, \\ 0 & \text{altrimenti.} \end{cases} \tag{5.6}$$

L'offset unitario $+1$ garantisce la continuità della metrica per $\mathrm{tf} = 1$ ($wf = 1 + \log_{10}(1) = 1.0$), scalando con dolcezza: $\mathrm{tf} = 10 \implies wf = 2.0$, $\mathrm{tf} = 100 \implies wf = 3.0$, $\mathrm{tf} = 1\,000 \implies wf = 4.0$.

5.4 Frequenza Documentale e Rarità: DF vs. CF e Inverse Document Frequency (IDF)

La sola frequenza locale tf non basta: parole ubiquitarie nel corpus possiedono un valore discriminante nullo, mentre parole rare identificano con precisione l'argomento cercato.

5.4.1 Collection Frequency (cf_t) contro Document Frequency (df_t)

È essenziale distinguere formalmente tra due metriche di frequenza globale:

- **Collection Frequency (cf_t):** Numero totale di occorrenze del termine t sull'intera collezione, contando tutte le ripetizioni;
- **Document Frequency (df_t):** Numero di documenti distinti che contengono almeno una volta il termine t.

Esempio 5.4.1 (Perché df è preferibile a cf nell'Information Retrieval)**.** Si consideri un corpus di 10^6 testi e due termini con identica cf $= 10\,000$:

- Termine A (*"insurance"*): compare 10 000 volte distribuito uniformemente in 10 000 documenti distinti ($df = 10\,000$);
- Termine B (*"tryptophan"*): compare 10 000 volte concentrato quasi integralmente in 5 trattati di biochimica ($df = 5$).

Sebbene abbiano la stessa cf, il Termine B è immensamente più discriminante: la sua presenza in un documento circoscrive la ricerca a una frazione microscopica della collezione. La Document Frequency df_t misura l'informatività in modo assai più affidabile della frequenza grezza nella collezione.

5.4.2 Formula Matematica dell'Inverse Document Frequency (IDF)

Poiché l'utilità informativa è inversamente proporzionale alla frequenza documentale df_t, si definisce:

Definizione 5.4.1 (Inverse Document Frequency)**.** Dato un corpus di N documenti, l'IDF del termine t è:

$$\mathrm{idf}_t = \log_{10}\left(\frac{N}{df_t}\right). \tag{5.7}$$

Effetto dell'IDF sull'Ordinamento: Query a Singolo Termine vs. Multi-Termine.

- **Query Monotermine ($q = \{t\}$):** L'IDF è una costante moltiplicativa identica per tutti i documenti contenenti t. Essa non altera l'ordinamento relativo dei risultati (che dipende solo da tf), ma agisce solo come fattore di scala globale;

- **Query Multi-Termine ($q = \{t_1, t_2, \ldots, t_m\}$):** L'IDF gioca un ruolo decisivo: ribilancia la rilevanza assegnando un peso preponderante alle parole rare e specifiche (es. *"Arachnocampa"*) rispetto alle parole diffuse (es. *"caves"*).

Termine	**Document Frequency (df_t)**	**idf$_t$ = $\log_{10}(10^6/df_t)$**
`calpurnia`	1	6.00
`animal`	100	4.00
`sunday`	1 000	3.00
`fly`	10 000	2.00
`under`	100 000	1.00
`the`	1 000 000	0.00

Tabella 5.2: Valori di IDF calcolati su una collezione di $N = 1\,000\,000$ documenti [1].

5.5 Lo Schema di Ponderazione TF-IDF Completo

Moltiplicando la frequenza locale logaritmica per la rarità globale si ottiene la celebre ponderazione **TF-IDF**:

$$w_{t,d} = \left(1 + \log_{10}(\mathrm{tf}_{t,d})\right) \cdot \log_{10}\left(\frac{N}{df_t}\right). \tag{5.8}$$

5.5.1 Esercizio Numerico Svolto alla Lavagna

Si consideri una collezione di $N = 806\,791$ documenti e i conteggi grezzi per quattro termini:

Termine	tf$_{\text{raw}}$	df_t	Log-TF = $1 + \log_{10}(\mathrm{tf})$	IDF = $\log_{10}(N/df_t)$
`auto`	19	3 665	$1 + \log_{10}(19) = \mathbf{2.28}$	$\log_{10}(806\,791/3\,665) = \mathbf{2.34}$
`best`	14	20 288	$1 + \log_{10}(14) = \mathbf{2.15}$	$\log_{10}(806\,791/20\,288) = \mathbf{1.60}$
`car`	27	10 282	$1 + \log_{10}(27) = \mathbf{2.43}$	$\log_{10}(806\,791/10\,282) = \mathbf{1.89}$
`insurance`	34	560	$1 + \log_{10}(34) = \mathbf{2.53}$	$\log_{10}(806\,791/560) = \mathbf{3.16}$

Tabella 5.3: Dati dell'esercizio svolto alla lavagna per il calcolo dei pesi TF-IDF.

I pesi composti finali risultano:

$$w(\texttt{auto}) = 2.28 \cdot 2.34 = \mathbf{5.34}, \tag{5.9}$$

$$w(\texttt{best}) = 2.15 \cdot 1.60 = \mathbf{3.44}, \tag{5.10}$$

$$w(\texttt{car}) = 2.43 \cdot 1.89 = \mathbf{4.59}, \tag{5.11}$$

$$w(\texttt{insurance}) = 2.53 \cdot 3.16 = \mathbf{8.00}. \tag{5.12}$$

Sebbene `car` abbia una frequenza locale grezza doppia rispetto a `best`, il termine più determinante per la pertinenza è `insurance`, la cui eccezionale rarità nel corpus ne eleva drasticamente il peso finale (8.00).

5.6 Documenti e Query come Vettori N-dimensionali nello Spazio Vettoriale

Nel **Vector Space Model** (VSM), introdotto da Gerard Salton [1], l'intera collezione documentale e le interrogazioni dell'utente vengono modellate come punti e vettori geometrici immersi in uno spazio vettoriale continuo a dimensionalità finita:

$$\text{Dimensione dello Spazio} = |\mathcal{V}|, \tag{5.13}$$

dove ogni asse ortogonale corrisponde a un termine unico del vocabolario $\mathcal{V}$. Sia la query q sia il documento d sono rappresentati come vettori multidimensionali di pesi continui:

$$\vec{v}(q) \in \mathbb{R}^{|\mathcal{V}|}, \qquad \vec{v}(d) \in \mathbb{R}^{|\mathcal{V}|}. \tag{5.14}$$

Proprietà 5.6.1 (Estrema Sparsità Vettoriale)**.** *Nelle collezioni reali, la cardinalità $|\mathcal{V}|$ raggiunge facilmente 10^6 vocaboli. Un tipico documento Web contiene poche centinaia di parole distinte: oltre il* $\mathbf{99.9}\%$ *delle componenti del vettore è pari a zero. Sia la memorizzazione fisica sia i prodotti algebrici poggiano su strutture per vettori sparsi (sparse vectors) e liste invertite (inverted index).*

5.7 Come Rispondere a una Query nello Spazio Vettoriale: Distanza L_2 (Pro e Contro)

La metrica geometrica più immediata per quantificare la prossimità tra due vettori nello spazio cartesiano è la distanza euclidea (L_2):

$$\text{dist}_{L_2}(\vec{q}, \vec{d}) = \|\vec{q} - \vec{d}\|_2 = \sqrt{\sum_{i=1}^{|\mathcal{V}|} (q_i - d_i)^2}. \tag{5.15}$$

Vantaggi e Svantaggi della Distanza Euclidea (Pros and Cons):

- **Vantaggi (Pros):** Metrica rigorosa e intuitiva nello spazio reale continuo; soddisfa tutte le proprietà di distanza metrica (disuguaglianza triangolare, simmetria, non negatività).
- **Svantaggi e Limiti Critici (Cons):** Risente fortemente della lunghezza assoluta del documento anziché del suo contenuto semantico.

5.7.1 L'Esperimento Mentale del Documento Concatenato ($d' = d \circ d$)

Si consideri un documento ideale d, rilevante rispetto alla query q. Si costruisca un documento duplicato d' concatenando due copie integrali di d: $d' = d \circ d$.

- Il profilo tematico, l'argomento e la densità semantica di d' sono **rigorosamente identici** a quelli di d;
- Tuttavia, le frequenze raddoppiano: $\text{tf}_{t,d'} = 2 \cdot \text{tf}_{t,d}$, comportando approssimativamente il raddoppio delle coordinate: $\vec{v}(d') \approx 2 \cdot \vec{v}(d)$.

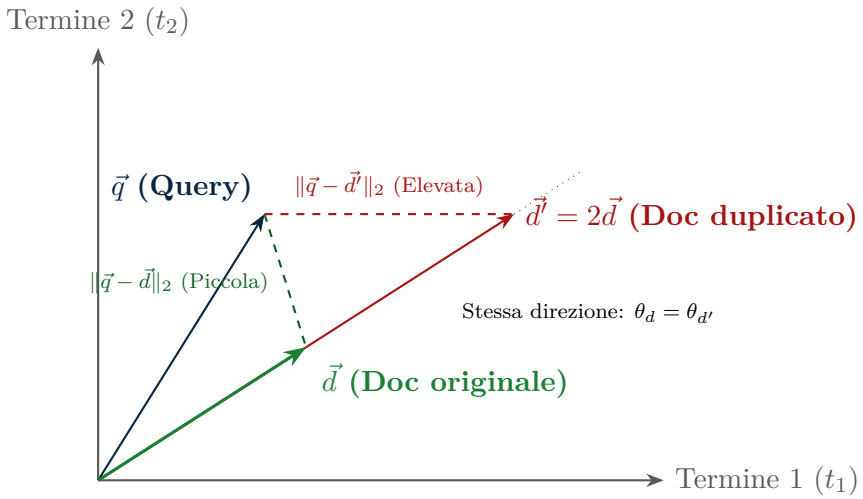


Figura 5.1: Fallimento della distanza euclidea nello Spazio Vettoriale: il documento duplicato $\vec{d'}$ giace sulla medesima retta di $\vec{d}$, ma la distanza euclidea da $\vec{q}$ cresce artificiosamente a causa della magnitudo.

Come illustrato graficamente nella Figura 5.1:

$$\|\vec{q} - \vec{d'}\|_2 \gg \|\vec{q} - \vec{d}\|_2. \tag{5.16}$$

La distanza euclidea penalizza ingiustamente il documento più esaustivo a causa della sua lunghezza fisica (magnitudo del vettore).

5.8 Dagli Angoli alla Cosine Similarity e Normalizzazione L2

Poiché la magnitudo del vettore distorce la stima, l'intuizione geometrica fondamentale dell'IR consiste nel quantificare la pertinenza misurando **l'angolo θ compreso tra i vettori**, anziché la loro distanza metrica:

- Il documento originale $\vec{d}$ e il duplicato $\vec{d'}$ possiedono esattamente la medesima direzione: formano con la query $\vec{q}$ lo stesso identico angolo θ;
- Più l'angolo si avvicina a 0°, più i vettori condividono le proporzioni lessicali;
- Poiché il coseno decresce monotonicamente tra 0° e 90° (da 1 a 0), esso funge da metrica ideale di rilevanza.

5.8.1 Derivazione Analitica della Cosine Similarity

Dal teorema del prodotto scalare:

$$\vec{q} \cdot \vec{d} = \|\vec{q}\|_2 \cdot \|\vec{d}\|_2 \cdot \cos\theta, \tag{5.17}$$

da cui si ricava la formula canonica della **Cosine Similarity**:

$$\text{Score}_{\cos}(q, d) = \cos\theta = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\|_2 \|\vec{d}\|_2} = \frac{\sum_{t \in q \cap d} w_{t,q} \cdot w_{t,d}}{\sqrt{\sum_{t \in q} w_{t,q}^2} \cdot \sqrt{\sum_{t \in d} w_{t,d}^2}}. \tag{5.18}$$

5.8.2 Normalizzazione L_2 e Proiezione sull'Ipersfera Unitaria

La formula del coseno equivale al prodotto scalare tra vettori preventivamente riscalati a norma unitaria (**Normalizzazione L_2**):

$$\vec{v}_{\text{norm}} = \frac{\vec{v}}{\|\vec{v}\|_2} = \frac{\vec{v}}{\sqrt{\sum_i v_i^2}} \implies \|\vec{v}_{\text{norm}}\|_2 = 1. \tag{5.19}$$

Sotto il profilo geometrico, la normalizzazione proietta ogni punto sulla superficie dell'ipersfera unitaria a $|\mathcal{V}|$ dimensioni, azzerando le differenze dovute alla lunghezza fisica dei testi.

5.8.3 Architettura di Calcolo: Disaccoppiamento Offline vs. Online

Nei motori in produzione, il denominatore della formula (5.18) non viene ricalcolato a runtime:

- **Fase Offline (Indicizzazione):** Per ogni documento d, la norma $\|\vec{d}\|_2$ (o i vettori già normalizzati) viene calcolata una sola volta e memorizzata nell'indice;
- **Fase Online (Query Processing):** All'arrivo della query, il sistema calcola la norma della sola query $\|\vec{q}\|_2$ ed esegue il prodotto scalare rapido $\vec{q}_{\text{norm}} \cdot \vec{d}_{\text{norm}}$ unicamente sui documenti che condividono termini con q.

5.9 Esercizio Pratico Completo: Confronto Stilistico tra Tre Romanzi

Per osservare il funzionamento completo del modello su dati reali, si consideri l'analisi didattica di Manning et al. [1] su tre celebri capolavori della letteratura inglese:

- D_1: *Sense and Sensibility* (Jane Austen, 1811);
- D_2: *Pride and Prejudice* (Jane Austen, 1813);
- D_3: *Wuthering Heights* (Emily Brontë, 1847);

focalizzata su tre termini-chiave del vocabolario: `affection`, `jealous` e `gossip`.

Termine	D_1 **(Sense & Sensibility)**	D_2 **(Pride & Prejudice)**	D_3 **(Wuthering Heights)**
`affection`	115	58	20
`jealous`	10	7	11
`gossip`	2	0	6

Tabella 5.4: Frequenze grezze assolute (tf_{raw}) nei tre romanzi.

Passo 1: Ponderazione Log-TF $(1 + \log_{10}(tf))$:

- *Sense and Sensibility (D_1):*

$$\begin{aligned} w_{\texttt{affection},D_1} &= 1 + \log_{10}(115) \approx 1 + 2.0607 = \mathbf{3.06}, \\ w_{\texttt{jealous},D_1} &= 1 + \log_{10}(10) = 1 + 1.0000 = \mathbf{2.00}, \\ w_{\texttt{gossip},D_1} &= 1 + \log_{10}(2) \approx 1 + 0.3010 = \mathbf{1.30}. \end{aligned}$$

- *Pride and Prejudice (D_2):*

$$\begin{aligned} w_{\texttt{affection},D_2} &= 1 + \log_{10}(58) \approx 1 + 1.7634 = \mathbf{2.76}, \\ w_{\texttt{jealous},D_2} &= 1 + \log_{10}(7) \approx 1 + 0.8451 = \mathbf{1.85}, \\ w_{\texttt{gossip},D_2} &= \mathbf{0.00}. \end{aligned}$$

- *Wuthering Heights (D_3):*

$$\begin{aligned} w_{\texttt{affection},D_3} &= 1 + \log_{10}(20) \approx 1 + 1.3010 = \mathbf{2.30}, \\ w_{\texttt{jealous},D_3} &= 1 + \log_{10}(11) \approx 1 + 1.0414 = \mathbf{2.04}, \\ w_{\texttt{gossip},D_3} &= 1 + \log_{10}(6) \approx 1 + 0.7782 = \mathbf{1.78}. \end{aligned}$$

Passo 2: Calcolo delle Norme Euclidee L_2:

$$\begin{aligned} \|\vec{d_1}\|_2 &= \sqrt{3.06^2 + 2.00^2 + 1.30^2} \\ &= \sqrt{9.3636 + 4.0000 + 1.6900} = \sqrt{15.0536} \approx \mathbf{3.88}, \qquad (5.20) \\ \|\vec{d_2}\|_2 &= \sqrt{2.76^2 + 1.85^2 + 0.00^2} \\ &= \sqrt{7.6176 + 3.4225 + 0.0000} = \sqrt{11.0401} \approx \mathbf{3.32}, \qquad (5.21) \\ \|\vec{d_3}\|_2 &= \sqrt{2.30^2 + 2.04^2 + 1.78^2} \\ &= \sqrt{5.2900 + 4.1616 + 3.1684} = \sqrt{12.6200} \approx \mathbf{3.55}. \qquad (5.22) \end{aligned}$$

Dividendo ciascuna coordinata per la rispettiva norma si ricavano i vettori normalizzati:

Termine	D_1 **(Sense & Sensibility)**	D_2 **(Pride & Prejudice)**	D_3 **(Wuthering Heights)**
`affection`	$3.06/3.88 = \mathbf{0.789}$	$2.76/3.32 = \mathbf{0.831}$	$2.30/3.55 = \mathbf{0.648}$
`jealous`	$2.00/3.88 = \mathbf{0.515}$	$1.85/3.32 = \mathbf{0.557}$	$2.04/3.55 = \mathbf{0.575}$
`gossip`	$1.30/3.88 = \mathbf{0.335}$	$0.00/3.32 = \mathbf{0.000}$	$1.78/3.55 = \mathbf{0.501}$

Tabella 5.5: Vettori di peso normalizzati con norma euclidea unitaria ($\|\vec{v}_{\text{norm}}\|_2 = 1$).

Passo 3: Similarità Coseno tra le Opere:

$$\begin{aligned}\cos(D_1, D_2) &= (0.789 \cdot 0.831) + (0.515 \cdot 0.557) + (0.335 \cdot 0.000) \\ &= 0.6557 + 0.2869 + 0 = \mathbf{0.9426} \approx \mathbf{0.94}, \end{aligned} \tag{5.23}$$

$$\begin{aligned}\cos(D_1, D_3) &= (0.789 \cdot 0.648) + (0.515 \cdot 0.575) + (0.335 \cdot 0.501) \\ &= 0.5113 + 0.2961 + 0.1678 = \mathbf{0.9752} \approx \mathbf{0.97}, \end{aligned} \tag{5.24}$$

$$\begin{aligned}\cos(D_2, D_3) &= (0.831 \cdot 0.648) + (0.557 \cdot 0.575) + (0.000 \cdot 0.501) \\ &= 0.5385 + 0.3203 + 0 = \mathbf{0.8588} \approx \mathbf{0.86}. \end{aligned} \tag{5.25}$$

I risultati confermano la straordinaria affinità stilistica dei romanzi di Jane Austen ($\cos \approx 0.94$), differenziandoli chiaramente dalla drammaticità gotica di Emily Brontë.

5.10 Varianti di Ponderazione e la Notazione SMART

Nei sistemi applicativi, le componenti di Term Frequency (t), Document Frequency (d) e Normalizzazione (n) ammettono molteplici varianti algebriche, codificate nello standard **SMART** (Gerard Salton, Cornell University):

$$\textbf{ddd.qqq} \tag{5.26}$$

dove i primi tre caratteri descrivono la pesatura del **documento** e gli ultimi tre quella della **query**.

Term Frequency (t)		**Document Frequency (d)**		**Normalizzazione (n)**	
`n` (none)	tf	`n` (none)	1	`n` (none)	1
`l` (log)	$1 + \ln(\text{tf})$	`t` (idf)	$\ln(N/df)$	`c` (cosine)	$1/\sqrt{\sum w_i^2}$
`a` (aug)	$0.5 + \frac{0.5\text{tf}}{\max(\text{tf})}$	`p` (prob)	$\max(0, \ln \frac{N-df}{df})$	`u` (pivoted)	$1/(u + a \cdot \text{len})$
`b` (bool)	$\mathbb{I}(\text{tf} > 0)$			`b` (byte)	$1/\text{char_len}^{\alpha}$

Tabella 5.6: Tassonomia delle varianti di ponderazione nella notazione SMART [1].

Il Protocollo Industriale Standard: `lnc.ltc`. Nella configurazione standard:

- **Documento (`lnc`):** Log-TF (`l`), nessuna IDF (`n`, poiché l'IDF del termine è identica per tutti i documenti e risulterebbe ridondante moltiplicarla due volte), normalizzazione coseno (`c`);

- **Query (`ltc`):** Log-TF (`l`), IDF inclusa (`t`), normalizzazione coseno (`c`).

5.10.1 Esercizio Numerico Completo: Configurazione `lnc.ltc`

Si analizzi il seguente esercizio applicativo:

- **Documento** d**:** *"car insurance auto insurance"*
- **Query** q**:** *"best car insurance"*
- $N = 1\,000\,000$. Frequenze documentali:
  - `auto`: $df = 5\,000 \implies idf = \log_{10}(10^6/5000) = \log_{10}(200) \approx 2.3$
  - `best`: $df = 50\,000 \implies idf = \log_{10}(10^6/50000) = \log_{10}(20) \approx 1.3$
  - `car`: $df = 10\,000 \implies idf = \log_{10}(10^6/10000) = \log_{10}(100) = 2.0$
  - `insurance`: $df = 1\,000 \implies idf = \log_{10}(10^6/1000) = 3.0$

Passo 1: Elaborazione del Vettore Query (`ltc`). La Tabella 5.7 riporta il calcolo del vettore di query pesato e normalizzato:

Termine	tf **raw**	tf**-wt** $(1+\log tf)$	df	idf	**Peso grezzo** $(tf \cdot idf)$	**Peso norm.** $(w/\|\vec{q}\|)$
auto	0	0.0	5 000	2.3	$0.0 \times 2.3 = 0.0$	**0.00**
best	1	1.0	50 000	1.3	$1.0 \times 1.3 = 1.3$	$1.3/3.84 \approx$ **0.34**
car	1	1.0	10 000	2.0	$1.0 \times 2.0 = 2.0$	$2.0/3.84 \approx$ **0.52**
insurance	1	1.0	1 000	3.0	$1.0 \times 3.0 = 3.0$	$3.0/3.84 \approx$ **0.78**

Tabella 5.7: Computazione del vettore di query secondo lo schema `ltc`.

Norma euclidea del vettore query:

$$\|\vec{q}\|_2 = \sqrt{0^2 + 1.3^2 + 2.0^2 + 3.0^2} = \sqrt{1.69 + 4.0 + 9.0} = \sqrt{14.69} \approx \mathbf{3.84}. \tag{5.27}$$

Passo 2: Elaborazione del Vettore Documento (`lnc`). La Tabella 5.8 riporta il calcolo del vettore documentale (senza fattore IDF):

Termine	tf **raw**	tf**-wt** $(1+\log tf)$	idf **(omesso per** `n`**)**	**Peso grezzo**	**Peso norm.** $(w/\|\vec{d}\|)$
auto	1	1.0	—	1.0	$1.0/1.92 \approx$ **0.52**
best	0	0.0	—	0.0	**0.00**
car	1	1.0	—	1.0	$1.0/1.92 \approx$ **0.52**
insurance	2	$1 + \log_{10}(2) \approx 1.30$	—	1.3	$1.3/1.92 \approx$ **0.68**

Tabella 5.8: Computazione del vettore di documento secondo lo schema `lnc`.

Norma euclidea del vettore documento:

$$\|\vec{d}\|_2 = \sqrt{1.0^2 + 0^2 + 1.0^2 + 1.30^2} = \sqrt{1.0 + 0 + 1.0 + 1.69} = \sqrt{3.69} \approx \mathbf{1.92}. \tag{5.28}$$

Passo 3: Calcolo del Punteggio Finale di Ranking. Il punteggio di similarità è dato dal prodotto scalare tra i due vettori normalizzati:

$$\begin{aligned} \text{Score}(q, d) &= \vec{q}_{\text{norm}} \cdot \vec{d}_{\text{norm}} \\ &= (0.00 \cdot 0.52) + (0.34 \cdot 0.00) + (0.52 \cdot 0.52) + (0.78 \cdot 0.68) \\ &= 0 + 0 + 0.2704 + 0.5304 = \mathbf{0.8008} \approx \mathbf{0.80}. \end{aligned} \tag{5.29}$$

5.11 Domande di Verifica e Concetti Chiave

1. **Perché il coefficiente di Jaccard assegna talvolta un punteggio superiore a documenti meno pertinenti?** Opera su insiemi puri, trascurando la frequenza locale dei termini (tf) e la rarità globale nella collezione (idf). Inoltre penalizza eccessivamente i documenti lunghi ed esaustivi a causa dell'espansione dell'unione al denominatore.

2. **Perché non si adotta la distanza euclidea nello Spazio Vettoriale per ordinare i documenti?** La distanza euclidea risente della magnitudo: un documento duplicato con identico contenuto ha una norma doppia, risultando geometricamente lontanissimo dalla query pur avendo angolo zero ($\theta = 0°$).

3. **Che cosa rappresenta geometricamente la normalizzazione L_2?** Rappresenta la proiezione radiale dei vettori sulla superficie dell'ipersfera unitaria, neutralizzando l'effetto della lunghezza del documento e riducendo il calcolo del coseno a un semplice prodotto scalare.

4. **Qual è l'effetto dell'IDF su una query monoremica?** Nullo sul ranking relativo: il fattore IDF agisce come una costante moltiplicativa identica per tutti i documenti contenenti la parola, lasciando invariato l'ordinamento.

5.12 Mappa Concettuale Riassuntiva e Verso il Query Processing

La Figura 5.2 offre una visione sinottica dell'intero flusso di trasformazione: dall'acquisizione della risorsa Web grezza fino alla rappresentazione vettoriale e al calcolo della similarità cosenica.

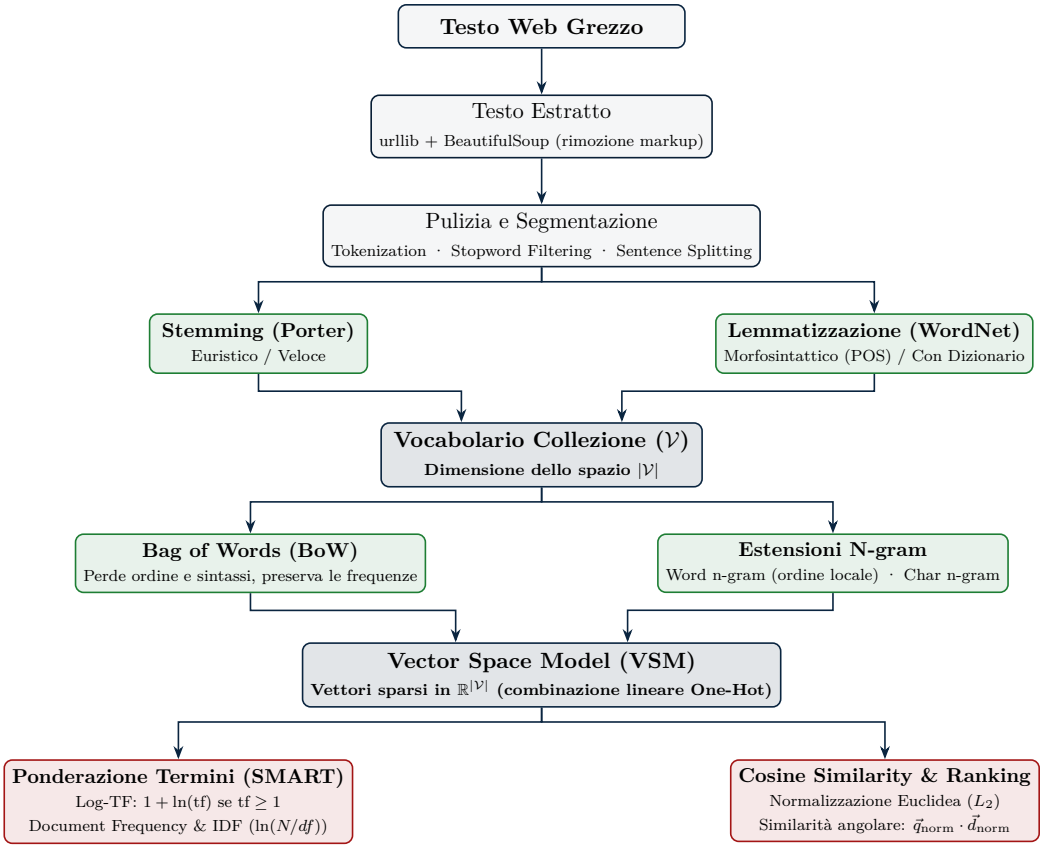


Figura 5.2: Mappa concettuale integrata della pipeline di IR: dall'acquisizione e pre-elaborazione del testo alla rappresentazione geometrica nello Spazio Vettoriale e alla stima della similarità cosenica.

Dallo Scoring Matematico all'Architettura di Sistema

Con la formalizzazione dello Spazio Vettoriale, degli schemi SMART e della similarità cosenica si conclude la trattazione teorica dei modelli di reperimento lineari. Nei successivi capitoli sull'indicizzazione (Capitolo 6, Capitolo 7 e Capitolo 8) si affronta l'esecuzione efficiente delle interrogazioni mediante indici invertiti, mentre l'evoluzione probabilistica (Okapi BM25 e BM25F) e i modelli non lineari di ranking basati su machine learning vengono approfonditi nel Capitolo 9 e nel Capitolo 10.

Capitolo 6

Architettura dei Motori di Ricerca e Indicizzazione

6.1 Architettura di Sistema e Paradigma a Due Stadi

Un motore di ricerca su scala Web deve conciliare requisiti architetturali apparentemente inconciliabili: memorizzare e scansionare collezioni testuali non strutturate di dimensioni sterminate garantendo al contempo una latenza di risposta rigorosamente interattiva ($\leq$ 50–100 ms) per ciascuna interrogazione utente [12]. Per soddisfare tali vincoli sotto volumi elevatissimi di traffico concorrente (migliaia di query al secondo, o QPS), il flusso computazionale viene rigidamente disaccoppiato tra processi asincroni massivi ad alto throughput (**fase offline**) e processi sincroni a latenza vincolata (**fase online**).

6.1.1 Architettura Distribuita del Motore di Ricerca

Nessun singolo server fisico possiede una capacità di memoria o di calcolo sufficiente per gestire l'intero Web. Come formalizzato da Tonellotto et al. [12], un moderno motore di ricerca adotta un'infrastruttura di calcolo distribuita organizzata secondo uno schema **document-partitioned** (partizionamento orizzontale della collezione):

- L'immensa collezione documentale $\mathcal{D}$ ($N \sim 10^9$–10^{11} documenti) viene suddivisa in S frammenti indipendenti denominati **shards** (o partizioni).
- Ciascun server di indice (*index server*) ospita un sottoinsieme di documenti e mantiene il relativo indice invertito locale.

- Un coordinatore centrale, denominato **Query Broker**, riceve l'interrogazione dell'utente, la distribuisce in parallelo a tutti gli index shards, raccoglie i risultati parziali prodotti da ciascun nodo ed esegue la fusione dei candidati (*Results Merging*) per estrarre la graduatoria globale.

6.1.2 Pipeline Offline: Indicizzazione e Compressione

La pipeline offline opera in background e a intervalli periodici sulla collezione documentale attraverso tre macro-fasi:

1. **Text Processing e Parsing:** I documenti raccolti tramite crawler vengono ripuliti dai marcatori HTML/XML, tokenizzati, normalizzati e depurati da caratteri non informativi secondo le pipeline esaminate nel Capitolo 3.

2. **Generazione del Vocabolario e dell'Indice:** Viene estratto il lessico $\mathcal{V} = \{t_1, \ldots, t_{|\mathcal{V}|}\}$ e costruita la struttura associativa che mappa ciascun termine alle relative occorrenze nei documenti.

3. **Compressione Numerica Lossless:** Poiché l'overhead dimensionale dell'indice grezzo può eguagliare o superare la dimensione del testo originario, vengono applicati algoritmi di compressione per interi (*d*-gaps, codici a byte variabile, Elias-Fano, PForDelta) volti a massimizzare la residenza dei dati nei livelli alti della gerarchia di memoria (RAM e cache CPU, come analizzato nel Capitolo 2).

6.1.3 Pipeline Online e Paradigma di Retrieval a Due Stadi

L'elaborazione delle interrogazioni online adotta una rigida **Two-Stage Retrieval Pipeline** (o architettura a imbuto), volta a massimizzare l'efficienza senza sacrificare l'accuratezza semantica:

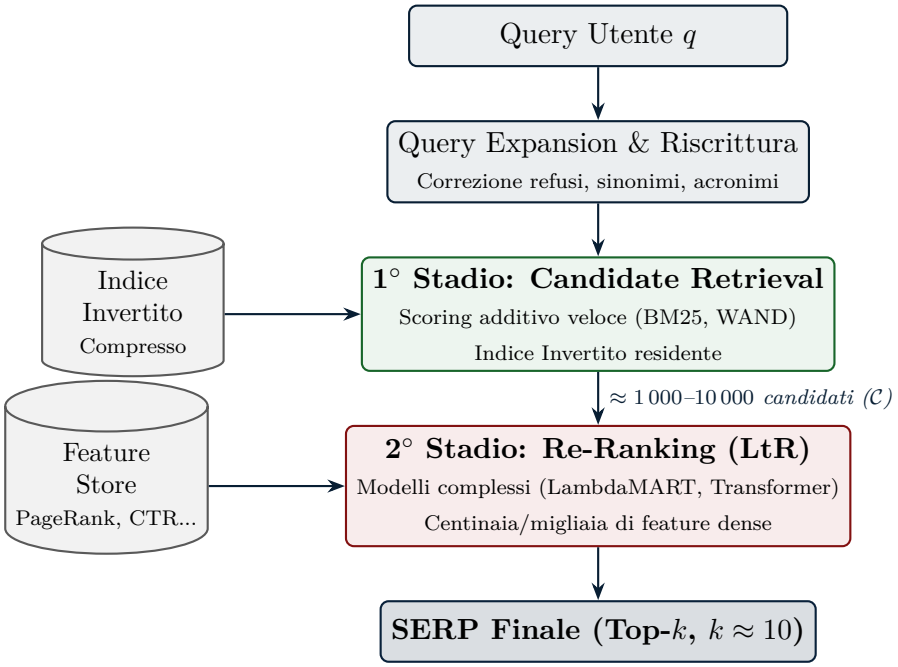


Figura 6.1: Architettura della Two-Stage Retrieval Pipeline: filtraggio massivo ad alta efficienza nel primo stadio ed estrazione dei candidati, seguito da riordinamento fine supervisionato nel secondo stadio [12].

1. **Primo Stadio (Candidate Retrieval / Top-k):** opera sull'intera collezione ($N \sim 10^9$–10^{11} documenti). Impiega algoritmi di dynamic pruning altamente ottimizzati (es. WAND o MaxScore, trattati nel Capitolo 8) su funzioni di scoring additive e scalari (Okapi BM25). L'obiettivo primario è l'altissimo throughput con tempo di risposta sub-millisecondo per shard, estraendo un insieme compatto di candidati:

$$\mathcal{C} \subset \mathcal{D}, \quad |\mathcal{C}| \approx 1\,000\text{–}10\,000 \text{ documenti.} \tag{6.1}$$

2. **Secondo Stadio (Re-Ranking / Heavy Scoring):** opera esclusivamente sull'insieme $\mathcal{C}$. Interroga un archivio di metadati (*Document Features Repository*) per recuperare centinaia di segnali eterogenei (qualità del dominio, PageRank, freschezza temporale, click-through rate storico, prossimità spaziale, affinità densa neurale). Tali feature alimentano modelli di Machine Learning supervisionati (*Learning-to-Rank*, approfonditi nel Capitolo 9) per produrre la graduatoria finale Top-k visualizzata all'utente ($k \approx 10$–20).

6.2 Modelli di Scoring: Prodotto Scalare e Connessione con il Dense Retrieval

L'elaborazione delle interrogazioni richiede una metrica algebrica solida per graduare i documenti in base alla loro aderenza al bisogno informativo. Questa sezione riesamina i limiti del modello booleano e del coefficiente di Jaccard, dimostrando come il prodotto scalare su vettori pesati costituisca la base unificante sia del retrieval sparso classico sia del moderno dense retrieval neurale.

6.2.1 Richiamo dei Limiti del Modello Booleano e di Jaccard

Nei sistemi di prima generazione, il recupero dei documenti poggiava sul modello booleano esatto. Come formalizzato nella Sezione 4.4, tale approccio soffre della grave anomalia del *Feast or Famine*: una congiunzione stringente (`AND`) rischia di svuotare l'insieme soluzione, mentre una disgiunzione (`OR`) restituisce centinaia di migliaia di risultati non graduati. Analogamente, l'applicazione ingenua del coefficiente insiemistico di Jaccard (analizzato nella Sezione 5.2) fallisce poiché penalizza arbitrariamente i documenti lunghi e non discrimina tra parole comuni e parole altamente informative.

6.2.2 Il Prodotto Scalare come Misura Unificante di Rilevanza

Per superare tali limitazioni, lo Spazio Vettoriale (VSM) e i moderni schemi di ponderazione (formalizzati nel Capitolo 5) proiettano documenti e query in vettori pesati, trasformando la misura di pertinenza nel calcolo di un **prodotto scalare**:

$$\text{Score}(q, d) = \mathbf{q} \cdot \mathbf{d} = \sum_{t \in \mathcal{V}} w(t, q) \cdot w(t, d). \tag{6.2}$$

Poiché per una query standard i termini non menzionati hanno peso $w(t, q) = 0$, la sommatoria (6.2) si restringe ai soli termini condivisi:

$$\text{Score}(q, d) = \sum_{t \in q \cap d} w(t, q) \cdot w(t, d). \tag{6.3}$$

Questa formulazione evidenzia una profonda convergenza concettuale tra l'Information Retrieval lessicale classico e il moderno retrieval neurale:

- **Lexical Sparse Retrieval:** I vettori appartengono allo spazio $\mathbb{R}^{|\mathcal{V}|}$, dove ciascuna dimensione corrisponde a un lessema univoco del vocabolario ($|\mathcal{V}| \sim 10^7$). I vettori sono fortemente sparsi e lo score è calcolato tramite pesi statistici (TF-IDF, Okapi BM25).
- **Dense Neural Retrieval:** Modelli basati su Transformer (es. bi-encoder neurali) mappano testi e query in vettori continui densi di dimensionalità ridotta $\mathbf{v}_q, \mathbf{v}_d \in \mathbb{R}^k$ ($k \approx 768$ o $1\,536$). La rilevanza semantica è determinata dal prodotto scalare o similarità coseno:

$$\text{Score}_{\text{neural}}(q, d) = \mathbf{v}_q \cdot \mathbf{v}_d = \sum_{i=1}^{k} v_{q,i} \cdot v_{d,i}. \tag{6.4}$$

In entrambi i paradigmi, l'operazione fondamentale a runtime consiste nell'aggregazione efficiente di prodotti scalari su collezioni documentali su larga scala.

6.3 Strutture Dati ad Alte Prestazioni: Forward Index vs. Indice Invertito

Per gestire la sparsità estrema dello spazio termini–documenti, l'architettura del motore di ricerca deve abbandonare le matrici dense a favore di strutture di memorizzazione compressa. Questa sezione confronta le due modalità canoniche di organizzazione dei dati (Forward Index e Indice Invertito), motivando la centralità dell'indice invertito per la risoluzione rapida delle interrogazioni online.

6.3.1 La Matrice Termini–Documenti e il Problema della Sparsità

Consideriamo la matrice globale termini–documenti $M \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{D}|}$. I parametri dimensionali di un motore di ricerca web reale evidenziano l'impossibilità fisica di allocare e scansionare tale struttura in forma densa:

- Dimensione del lessico: $|\mathcal{V}| \sim 10^7$ termini unici.
- Dimensione della collezione: $|\mathcal{D}| \sim 10^9$–10^{11} documenti web.
- Numero totale di celle: $|\mathcal{V}| \times |\mathcal{D}| \approx 10^{16}$–$10^{18}$ posizioni.

Poiché un documento medio contiene tra 10^2 e 10^3 vocaboli distinti, la densità di elementi non nulli è infinitesima:

$$\frac{\text{elementi non nulli}}{|\mathcal{V}| \times |\mathcal{D}|} \ll 0.001\%. \tag{6.5}$$

Oltre il 99.99% delle celle contiene il valore zero. Memorizzare la matrice in formato denso comporterebbe la saturazione immediata della memoria secondaria e una complessità temporale di valutazione query proibitiva, pari a $\mathcal{O}(|\mathcal{V}| \cdot |\mathcal{D}|)$, costringendo la CPU a esaminare miliardi di documenti irrilevanti.

6.3.2 Rappresentazione tramite Vettori Sparsi: Forward Index vs. Indice Invertito

Per eliminare gli zeri, la matrice viene memorizzata come un insieme di vettori sparsi composti da coppie (Identificatore, Valore). L'orientamento di serializzazione determina due strutture complementari, sintetizzate nella Tabella 6.1:

- **Forward Index (Column-Wise / Document-Oriented):** memorizza per ciascun documento la lista ordinata dei termini che vi compaiono, ad esempio $d_1 \to [(t_2, w_2), (t_7, w_7)]$.
- **Indice Invertito (Row-Wise / Term-Oriented):** memorizza per ciascun termine la lista ordinata dei documenti in cui ricorre, ad esempio $t_1 \to [(d_2, w_1), (d_5, w_5)]$.

Proprietà	**Forward Index**	**Indice Invertito**
Orientamento	Per colonna (Document-Oriented).	Per riga (Term-Oriented).
Struttura logica	$\text{DocID} \to [(t_a, \text{tf}_a), \dots]$	$t \to [(\text{DocID}_i, \text{tf}_i), \dots]$
Complessità query	$\mathcal{O}(\lvert\mathcal{D}\rvert)$: richiede ispezione di tutti i documenti.	$\mathcal{O}\Big(\sum_{t \in q} \lvert\mathcal{L}(t)\rvert\Big)$: accede solo ai documenti candidati.
Ruolo nel motore	Memorizzazione metadati, feature store, snippet generation.	Cuore computazionale del First-Stage Retrieval (matching rapido).

Tabella 6.1: Confronto sistematico tra Forward Index e Indice Invertito.

6.4 Anatomia, Layout di Memoria e Compressione dell'Indice Invertito

Un indice invertito di livello industriale si articola in due componenti fisiche distinte: il **Dizionario** (o Lessico $\mathcal{V}$) e l'insieme delle **Posting List**.

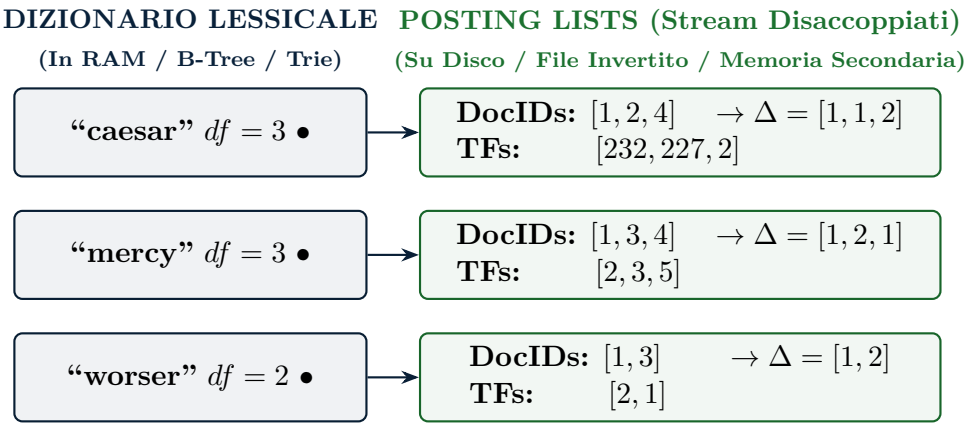


Figura 6.2: Anatomia di un indice invertito con disaccoppiamento fisico tra stream dei DocID (compressi tramite d-gaps) e stream delle frequenze tf.

6.4.1 Il Dizionario (Lexicon)

Il dizionario memorizza il vocabolario $\mathcal{V}$. Per ciascun termine t, registra la stringa lessicale, la Document Frequency df_t (utilizzata per ricavare l'IDF precalcolato), la Collection Frequency F_t e il puntatore di memoria o l'offset su storage secondario che localizza la corrispondente posting list. Dato che $|\mathcal{V}|$ è dell'ordine di milioni di lessemi, il dizionario viene mantenuto interamente residente in **RAM**, indicizzato tramite strutture di accesso rapido come B-Tree, Trie compressi o tabelle hash perfette.

6.4.2 Le Posting List: Disaccoppiamento Fisico degli Stream

Concettualmente, la posting list associata al termine t è una sequenza di tuple:

$$\mathcal{L}(t) = \Big[\langle d_1, \mathrm{tf}_{t,d_1}\rangle, \langle d_2, \mathrm{tf}_{t,d_2}\rangle, \dots\Big]. \tag{6.6}$$

A livello implementativo, memorizzare array continui di strutture composte (es. struct C++ contenenti `uint32_t doc_id` e `uint32_t tf`) è altamente

sub-ottimale. Per massimizzare i fattori di compressione numerica, i motori di ricerca operano il **disaccoppiamento fisico in stream paralleli**:

1. **Stream dei DocID (Monotono Crescente):**

$$\mathbf{D}_t = [\mathrm{DocID}_1, \mathrm{DocID}_2, \ldots, \mathrm{DocID}_m] \quad \text{con } \mathrm{DocID}_i < \mathrm{DocID}_{i+1}. \tag{6.7}$$

   Sfruttando la stretta monotonicità, i valori assoluti vengono convertiti nelle differenze tra identificatori contigui, dette d**-gaps** o delta:

$$\Delta_1 = \mathrm{DocID}_1, \quad \Delta_i = \mathrm{DocID}_i - \mathrm{DocID}_{i-1} \quad (\forall i \geq 2). \tag{6.8}$$

   I delta generati sono interi piccoli (spesso $\Delta \leq 5$ per termini ad alta frequenza), comprimibili ad altissima densità tramite algoritmi dedicati (*Variable Byte*, *Elias-Fano*, *PForDelta*, *Binary Interpolative Coding* [3]).

2. **Stream delle Frequenze (Non Ordinato):**

$$\mathbf{F}_t = [\mathrm{tf}(t, d_1), \mathrm{tf}(t, d_2), \ldots, \mathrm{tf}(t, d_m)]. \tag{6.9}$$

   Le frequenze locali non godono di ordinamento; tuttavia, per la legge di Zipf, la loro distribuzione empirica è fortemente concentrata su valori minimi $(1, 2, 3)$. Esse vengono compresse simmetricamente tramite codici a lunghezza variabile senza richiedere la differenziazione delta.

6.4.3 Trade-Off Architetturale: Interi Compressi vs. Float Precalcolati

Una decisione progettuale critica consiste nello stabilire se memorizzare nell'indice il punteggio finale precalcolato $w(t, d) \in \mathbb{R}$ a virgola mobile oppure l'intero grezzo $\mathrm{tf}(t, d) \in \mathbb{N}$:

Parametro	Punteggio Float Precalcolato	Intero TF Compresso
Occupazione Spazio	32 o 64 bit per posting (`float`/`double`).	Intero compresso (spesso < 8 bit medi per posting).
Comprimibilità	Scarsa o nulla: mantissa float a entropia quasi-casuale.	Elevatissima: concentrazione statistica su delta piccoli.
Flessibilità Ranking	Nulla: vincola l'indice alla specifica formula usata a build-time.	Totale: consente di calcolare qualsiasi funzione (BM25, varianti) a runtime.
Throughput CPU	Nessun calcolo a runtime (lettura diretta).	Pochi cicli ALU per posting per calcolare la formula.

Tabella 6.2: Confronto di sistema tra memorizzazione float e interi compressi.

Risoluzione del Vincolo Memory-Bound

Come analizzato nel Capitolo 2, i sistemi di Information Retrieval su larga scala sono rigorosamente **memory-bound**: il vero fattore limitante delle prestazioni è la banda di trasferimento dalla memoria principale (DRAM) ai registri della CPU (≈ 80 ns di latenza), non il throughput dell'unità aritmetica (ALU, 0.5 ns). Ridurre la dimensione dei posting attraverso la compressione degli interi abbatte il volume di byte trasferiti sul bus di memoria; il modesto costo computazionale richiesto per valutare la formula a runtime (es. BM25) viene ampiamente compensato dal risparmio di banda.

6.5 Tipologie di Query, Indici Posizionali e Riscrittura del Testo

L'indice invertito essenziale a sole frequenze (DocID, tf) non è sufficiente per supportare ricerche complesse che richiedono la contiguità delle parole o la riscrittura semantica delle interrogazioni. Questa sezione approfondisce la struttura dell'indice posizionale per l'elaborazione delle *phrase queries* e illustra le tecniche di *query expansion* e normalizzazione a tempo di esecuzione.

6.5.1 Phrase Queries e la Necessità dell'Indice Posizionale

Nelle ricerche per frase esatta (*Phrase Queries*), la query è racchiusa tra virgolette (es. `"neural network architecture"`). L'utente impone che i termini compaiano nel medesimo documento, nello stesso ordine e in posizioni adiacenti:

$$\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1. \tag{6.10}$$

L'indice invertito standard a sole frequenze (DocID, tf) non permette di validare questo vincolo, avendo scartato la posizione delle parole. Per supportare le phrase queries, la struttura viene estesa a **Indice Posizionale**, memorizzando per ogni posting la lista degli offset ordinali:

$$t \longrightarrow \Big[\langle \text{DocID}_k, \text{tf}_k, [\text{pos}_1, \text{pos}_2, \ldots, \text{pos}_{\text{tf}}]\rangle\Big]. \tag{6.11}$$

Esempio 6.5.1 (Intersezione Posizionale per Phrase Match)**.** Si consideri la query `"neural network"`:

- Posting per `"neural"`: $\cdots \to \langle \text{Doc } 14, \text{tf} = 2, \text{pos} = [4, 21]\rangle \to \ldots$
- Posting per `"network"`: $\cdots \to \langle \text{Doc } 14, \text{tf} = 3, \text{pos} = [5, 89, 102]\rangle \to \ldots$

L'algoritmo esegue l'intersezione su DocID $= 14$ e confronta le liste posizionali: poiché $\text{pos}(\texttt{neural}) = 4$ e $\text{pos}(\texttt{network}) = 5$, la condizione $\text{pos}(t_{i+1}) = \text{pos}(t_i) + 1$ è soddisfatta e il documento viene validato.

Overhead spaziale: L'inclusione delle posizioni incrementa l'impronta complessiva su disco dell'indice invertito di un fattore compreso tra $2\times$ e $3\times$.

6.5.2 Moduli di Query Expansion e il Caso Studio "Babbo Francesco"

I motori di ricerca industriali intercettano la query grezza dell'utente attraverso un modulo di **Query Expansion** e riscrittura, con l'obiettivo di correggere refusi, risolvere acronimi ed espandere sinonimi contestuali.

Tuttavia, espansioni lessicali non supervisionate basate su matrici di co-occorrenza o regole euristiche possono innescare gravi derive semantiche (*semantic drift*), come illustrato dal celebre caso riscontrato nei motori commerciali in lingua italiana:

1. **Query Utente:** L'utente inserisce la query a intento personale/familiare `"babbo francesco"`.

2. **Espansione Sinonimica:** Il modulo associa il termine regionale *babbo* al sinonimo *papà*.

3. **Normalizzazione e Troncamento Accenti:** La pipeline rimuove gli accenti trasformando *papà* in *papa* (omografo non accentato che designa il Pontefice).

4. **Query Riscritta all'Indice:** La disgiunzione risultante inviata al motore diventa `("babbo" OR "papà" OR "papa") AND "francesco"`.

5. **Scoring & Authority Ranking:** L'entità globale *"Papa Francesco"* possiede un'autorevolezza a priori (PageRank) e una co-occorrenza web sterminata rispetto a qualunque testo privato.

6. **Risultato Top-1 Restituito:** La SERP collassa restituendo in prima posizione la voce enciclopedica su *Papa Francesco (Jorge Mario Bergoglio)*, stravolgendo completamente l'intento informativo dell'utente.

6.5.3 Stemming vs. Query Expansion

È fondamentale chiarire la linea di demarcazione tra normalizzazione morfologica ed espansione semantica:

- **Stemming e Lemmatizzazione:** Operano riduzioni sintattiche all'interno della medesima radice etimologica (es. *running*, *runs* $\rightarrow$ *run*, come descritto in Sezione 3.3.5). Non possono collegare vocaboli con radici distinte aventi identico significato (es. *babbo* e *papà*).

- **Approccio Moderno:** I sistemi scalabili contemporanei riducono l'uso di stemming aggressivo in fase di indicizzazione (per preservare la specificità semantica dei termini nell'indice) e gestiscono le varianti flessive e sinonimiche a runtime tramite modelli linguistici neurali di espansione contestuale.

Dall'Indicizzazione al Query Processing

Costruite le strutture dati dell'indice e definite le modalità di rappresentazione delle interrogazioni, il motore deve eseguire il recupero dei documenti rilevanti in tempo reale ($\leq$ 50–100 ms). I modelli teorici e i paradigmi algoritmici per l'attraversamento delle posting list, l'aggregazione dei punteggi, l'ottimizzazione tramite salti rapidi (*Term-at-a-Time* e *Document-at-a-Time*) e la loro realizzazione empirica in Python sono formalizzati e analizzati esaustivamente nel Capitolo 7.

Capitolo 7

Query Processing: Algoritmi di Scansione ed Efficienza di Salto

7.1 Anatomia Statistica dell'Indice Inverso: Dizionario e Posting Lists

L'indice inverso costituisce l'infrastruttura fondamentale su cui poggiano tutte le operazioni di interrogazione nei moderni motori di ricerca. Questa sezione ne esamina l'organizzazione strutturale tra dizionario dei termini e liste di occorrenza, l'invariante di ordinamento dei docID e le statistiche ausiliarie necessarie per abilitare i modelli di scoring pesato.

7.1.1 Definizione e Componenti Architetturali

Come introdotto nella Sezione 6.4, l'**Indice Inverso** è la struttura dati associativa centrale per l'Information Retrieval: mappa ciascun termine del lessico all'insieme ordinato delle sue occorrenze nell'intera collezione. Si compone di due elementi fisici distinti:

1. **Dizionario (Dictionary o Lexicon):** Memorizza l'elenco ordinato di tutti i termini distinti $\mathcal{V}$ della collezione assieme a metadati globali di frequenza e offset di puntamento alla memoria secondaria.

2. **Posting Lists (Liste delle Occorrenze):** Per ciascun termine $t \in \mathcal{V}$, una sequenza ordinata di record (detti *postings*) che identificano i documenti in cui t compare.

Esempio 7.1.1 (Costruzione dell'Indice Inverso Booleano)**.** Si consideri una collezione elementare di $N = 3$ documenti testuali:

- d_1: "*An inverted index is an index storing a mapping from terms to their occurrences in a set of documents.*"
- d_2: "*The inverted index data structure is a central component of a typical search engine indexing algorithm.*"
- d_3: "*The goal is to optimize the speed of the query: find the documents where a set of words occur.*"

Assegnati gli identificatori interi $d_1 \to 1$, $d_2 \to 2$, $d_3 \to 3$, le posting list per alcuni termini del vocabolario risultano:

$$\begin{aligned}
\mathcal{L}(\texttt{"a"}) &= [1, 2, 3] \quad \text{(stopword ad altissima frequenza, tocca tutti i documenti)}, \\
\mathcal{L}(\texttt{"an"}) &= [1], \\
\mathcal{L}(\texttt{"index"}) &= [1, 2], \\
\mathcal{L}(\texttt{"inverted"}) &= [1, 2], \\
\mathcal{L}(\texttt{"query"}) &= [3].
\end{aligned}$$

7.1.2 L'Invariante Fondamentale di Ordinamento dei docID

All'interno di ciascuna posting list, i docID sono **rigorosamente ordinati in senso strettamente crescente**:

$$\forall i < j, \quad \text{docID}_i < \text{docID}_j. \tag{7.1}$$

Questa proprietà strutturale costituisce la precondizione algebrica indispensabile per eseguire l'unione (OR) e l'intersezione (AND) tra liste in tempo lineare o sublineare, evitando scansioni combinatorie esaustive.

7.1.3 Estensione per Modelli di Ranking: Statistiche di Dizionario e Posting

Nel recupero pesato (es. Okapi BM25 o TF-IDF), la semplice presenza binaria del termine non è sufficiente per calcolare la rilevanza. L'indice inverso memorizza statistiche strutturate su due livelli gerarchici:

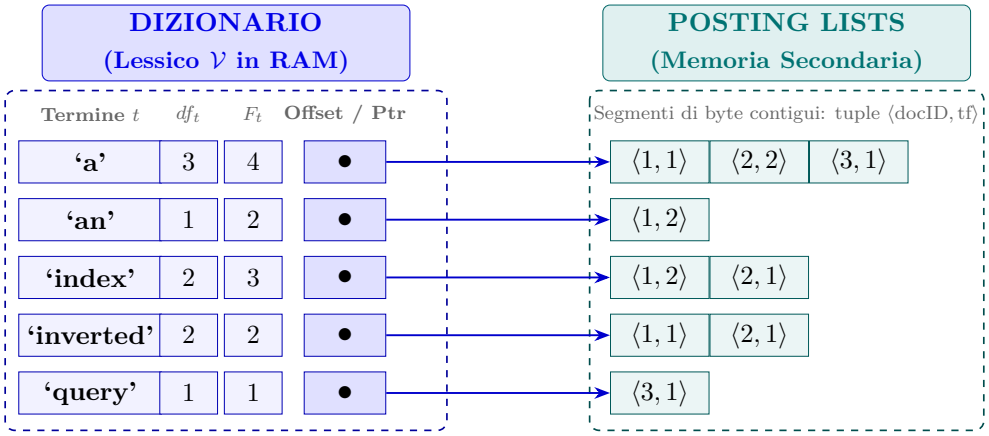


Figura 7.1: Organizzazione gerarchica dell'indice inverso per modelli pesati. A sinistra: il dizionario residente in RAM, ordinato lessicograficamente con statistiche aggregate (df_t, F_t) e campo offset/puntatore d'inizio lista. A destra: le posting list su memoria secondaria, strutturate come array contigui di byte (tuple $\langle \text{docID}, \text{tf} \rangle$) privi di puntatori di concatenazione interna.

1. **Statistiche a livello di Dizionario (per ciascun termine t):**
   - **Document Frequency (df_t):** il numero di documenti distinti contenenti almeno un'occorrenza di t. Corrisponde esattamente alla lunghezza (cardinalità) della posting list $|\mathcal{L}(t)|$.
   - **Collection Frequency (F_t):** il numero complessivo di occorrenze del termine t calcolato sull'intera collezione:
   $$F_t = \sum_{d \in \mathcal{L}(t)} \text{tf}_{t,d}. \tag{7.2}$$
   Ad esempio, per il termine `"a"` si ha $df_{\text{"a"}} = 3$ e $F_{\text{"a"}} = 1+2+1 = 4$. Vale sempre la relazione d'ordine fondamentale:
   $$F_t \geq df_t. \tag{7.3}$$
   L'uguaglianza $F_t = df_t$ si verifica se e solo se ciascun documento rilevante contiene il termine esattamente una volta ($\text{tf}_{t,d} = 1$).
   - **Puntatore / Offset d'Inizio (ptr_t):** l'indirizzo di memoria o l'offset in byte che indica il punto d'inizio della posting list del termine t all'interno del file compresso su memoria secondaria.
2. **Statistiche a livello di Posting List (per ciascun documento d):**
   - Viene memorizzata la **Term Frequency** locale $\text{tf}_{t,d}$, quantificando quante volte il termine compare nello specifico testo di d.

7.1.4 Trade-Off Architetturale: Perché NON Memorizzare lo Score Precalcolato?

Come approfondito nell'analisi di sistema della Sezione 6.4, ci si potrebbe domandare: *perché memorizzare la frequenza intera $tf_{t,d}$ anziché il punteggio finale già precalcolato (ad esempio lo score BM25 come float a 32 bit)?*

La prassi industriale esclude la memorizzazione di punteggi float precomputati per tre ragioni fisiche e progettuali:

1. **Comprimibilità dei Tipi di Dato:** La frequenza locale $\mathrm{tf}_{t,d}$ è un intero positivo strettamente delimitato, concentrato statisticamente su valori minimi $(1, 2, 3)$. Gli interi piccoli si prestano a rapporti di compressione lossless eccezionali (scendendo a pochi bit per posting). Al contrario, un numero float a virgola mobile possiede una mantissa ad alta entropia, pressoché incomprimibile.

2. **Località dell'IDF e Costo del Logaritmo:** Il calcolo dell'Inverse Document Frequency:

$$\mathrm{IDF}(t) = \log\left(\frac{N - df_t + 0.5}{df_t + 0.5}\right) \tag{7.4}$$

   dipende unicamente dalla frequenza di documento df_t e dalla taglia N della collezione: esso è **costante per l'intero termine** e completamente indipendente dal documento d. È computazionalmente vantaggioso calcolare l'IDF **una sola volta online** all'arrivo della query leggendo df_t dal dizionario (residente in RAM), anziché duplicarlo o fonderlo in ciascuno dei milioni di posting su memoria secondaria.

3. **Flessibilità e Modularità del Ranking:** Congelare lo score finale nell'indice vincolerebbe il sistema ai parametri specifici adottati a build-time (ad es. $k_1 = 1.2, b = 0.75$). Modificare i parametri o testare nuove varianti richiederebbe la reindicizzazione integrale del Web. Memorizzare dati grezzi (tf e lunghezze dei documenti) consente di variare liberamente la funzione di scoring a runtime.

7.2 Paradigmi di Scansione: TAAT vs. DAAT

Individuate le posting list associate ai termini di query $q = \{t_1, t_2, \ldots, t_m\}$, il motore deve scansionare i dati ed estrarre i primi k documenti con punteggio più elevato per popolare una coda con priorità (min-heap di dimensione fissa

k). Esistono due paradigmi algoritmici fondamentali: **Term-at-a-Time (TAAT)** e **Document-at-a-Time (DAAT)**.

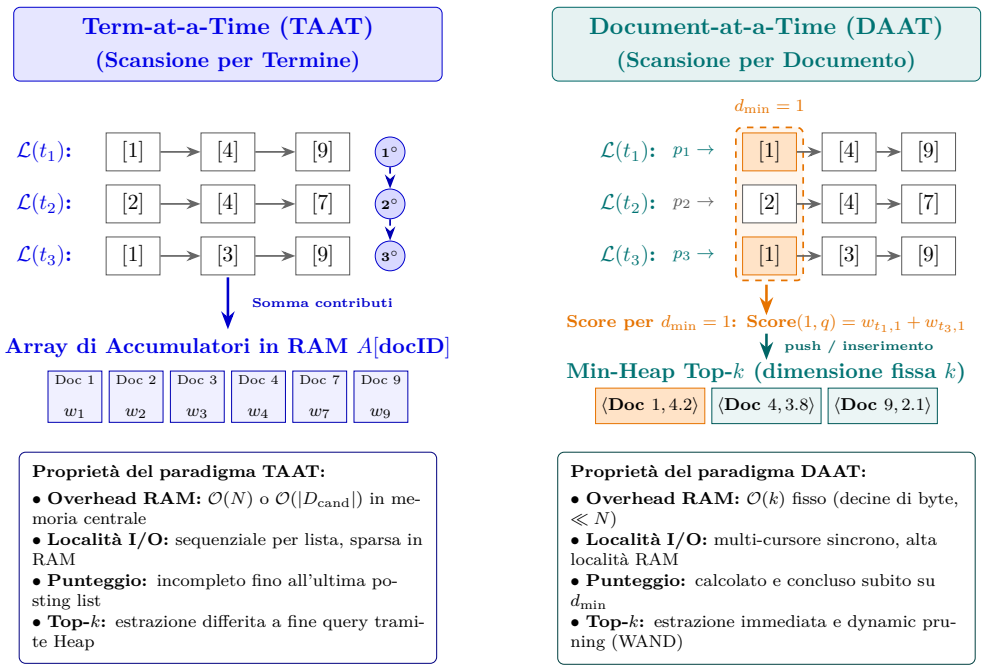


Figura 7.2: Confronto visivo tra i paradigmi di scansione: Term-at-a-Time (scansione orizzontale termine per termine con accumulo parziale $\mathcal{O}(N)$ in RAM) vs. Document-at-a-Time (scansione verticale sincrona per documento $d_{\min}$ con calcolo immediato dello score finale e memorizzazione ristretta al Min-Heap Top-k $\mathcal{O}(k)$).

7.2.1 La Strategia Term-at-a-Time (TAAT)

Nell'approccio TAAT, le posting list vengono elaborate in modo **sequenziale, una alla volta**:

1. Viene allocata una struttura di **accumulatori** $A[\text{DocID}] \in \mathbb{R}$ (tabella hash o array denso indicizzato per DocID).

2. Il motore apre la posting list del primo termine $\mathcal{L}(t_1)$ e itera su tutti i suoi posting, calcolando il punteggio parziale e incrementando l'accumulatore corrispondente.

3. Conclusa $\mathcal{L}(t_1)$, si passa a $\mathcal{L}(t_2)$ sommando i nuovi contributi, procedendo fino all'ultima lista $\mathcal{L}(t_m)$.

4. Al termine, si estraggono i top-k documenti scansionando la struttura degli accumulatori tramite un min-heap.

Algorithm 1: Term-at-a-Time (TAAT)

Dati: Query $q = \{t_1, t_2, \ldots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top-k documenti per punteggio

$A \leftarrow$ InizializzaTabellaAccumulatori() `// Mappa DocID → Score Parziale`
foreach *termine* $t_i \in q$ **do**
  $\mathcal{L}_i \leftarrow$ RecuperaPostingList(t_i)
  $\text{idf}_i \leftarrow$ CalcolaIDF(t_i)
  foreach $\langle d, tf \rangle \in \mathcal{L}_i$ **do**
    $s_{\text{parz}} \leftarrow$ CalcolaContributo(tf, idf_i)
    if $d \in A$ **then**
      $A[d] \leftarrow A[d] + s_{\text{parz}}$
    else
      $A[d] \leftarrow s_{\text{parz}}$
$H \leftarrow$ InizializzaMinHeap(capacità $= k$)
foreach *coppia* $(d, score) \in A$ **do**
  if $H.Dimensione() < k$ **then**
    H.Inserisci(d, score)
  else
    if $score > H.MinimoPunteggio()$ **then**
      H.EstraiMinimo()
      H.Inserisci(d, score)
return H

Analisi prestazionale di TAAT:

- **Vantaggi:** Accesso strettamente sequenziale alla memoria. Ciascuna posting list viene letta come un blocco contiguo, massimizzando l'efficacia del prefetching hardware della CPU e la località di cache (Capitolo 2).
- **Svantaggi:** Footprint di memoria volatile elevatissimo. La tabella degli accumulatori deve contenere l'unione dei documenti di tutte le posting list: con termini ad alta Document Frequency, A può coinvolgere decine di milioni di record. Inoltre, è impossibile effettuare il *dynamic early pruning*, poiché uno score parziale basso potrebbe essere incrementato arbitrariamente dalle liste successive.

7.2.2 La Strategia Document-at-a-Time (DAAT)

Nell'approccio DAAT, le posting list di tutti i termini di query vengono elaborate **simultaneamente e in parallelo**, operando come un merge multi-cursore:

1. Viene posizionato un puntatore p_i all'inizio di ciascuna lista $\mathcal{L}(t_i)$.

2. A ogni iterazione, il sistema determina il minimo identificatore corrente:

$$d_{\min} = \min_{i \in \{1,\dots,m\}} \{p_i.\text{CurrentDocID}()\}. \tag{7.5}$$

3. Vengono identificati tutti i cursori posizionati su $d_{\min}$, calcolando immediatamente il **punteggio finale consolidato** del documento:

$$S(q, d_{\min}) = \sum_{i : p_i.\text{DocID} = d_{\min}} \text{ScoreTerm}(p_i.\text{TF}, t_i). \tag{7.6}$$

4. Il punteggio definitivo viene confrontato all'istante con il Min-Heap di taglia k: se $S(q, d_{\min}) > H.\text{Minimo}()$, il documento entra nella Top-k scalzando il vecchio minimo.

5. Tutti i cursori attestati su $d_{\min}$ avanzano al prossimo elemento.

Algorithm 2: Document-at-a-Time (DAAT)

Dati: Query $q = \{t_1, t_2, \ldots, t_m\}$, Indice Invertito, intero k
Risultato: Min-Heap H contenente i Top-k documenti per punteggio

```
1  cursori ← InizializzaCursoriPerOgniLista(q)
2  H ← InizializzaMinHeap(capacità = k)
3  while EsistonoCursoriAttivi(cursori) do
4      d_min ← min{p.CurrentDocID() | p ∈ cursori ∧ p.ÈAttivo()}
5      score_tot ← 0.0
6      foreach p ∈ cursori do
7          if p.ÈAttivo() ∧ p.CurrentDocID() == d_min then
8              score_tot ←
                 score_tot + CalcolaContributo(p.CurrentTF(), p.Termine())
9              p.Avanza()
       // Il documento è completo: inserimento immediato
          nell'heap
10     if H.Dimensione() < k then
11         H.Inserisci(d_min, score_tot)
12     else
13         if score_tot > H.MinimoPunteggio() then
14             H.EstraiMinimo()
15             H.Inserisci(d_min, score_tot)
16 return H
```

Analisi prestazionale di DAAT:

- **Vantaggi:** Memoria di lavoro trascurabile ($\mathcal{O}(k)$ per il Min-Heap e $\mathcal{O}(m)$ per lo stato dei cursori), senza alcuna tabella intermedia di accumulo. Supporto nativo all'**Early Pruning dinamico**: conoscendo la soglia minima corrente dell'heap $\theta = H$.MinimoPunteggio(), algoritmi avanzati come **WAND** (*Weak AND*) o **Block-Max WAND** possono stimare i limiti superiori (*upper bounds*) di punteggio residuo e saltare a blocchi intere sequenze di posting che matematicamente non possono superare θ, evitando decompressioni superflue.

- **Svantaggi:** Accesso alla memoria frammentato su più stream contigui, con frequenti salti tra puntatori che riducono l'efficacia del caching rispetto alla lettura lineare di TAAT.

7.3 Elaborazione Term-at-a-Time (TAAT) in Dettaglio

Nel paradigma TAAT la computazione procede esaurendo sequenzialmente una lista di occorrenza alla volta, accumulando incrementalmente i punteggi parziali dei documenti intercettati. Questa sezione analizza il flusso operativo di TAAT tramite un esempio tracciato passo-passo, confronta le strutture dati per la memorizzazione degli accumulatori e ne evidenzia i limiti prestazionali in termini di località di cache e assenza di salti.

7.3.1 Tracciamento dell'Esecuzione di TAAT

Si consideri una query composta dai termini `"information"` e `"retrieval"`. Le posting list associate ai due vocaboli sono:

$$\mathcal{L}(\texttt{"information"}) = [1, 5, 8, 11, 13, 20, 35, 40, 42],$$
$$\mathcal{L}(\texttt{"retrieval"}) = [1, 5, 6, 8, 11, 15, 17, 50, 60].$$

Fase 1: Scansione completa della lista `"information"`. Il puntatore scorre l'intera lista $\mathcal{L}(\texttt{"information"})$. Per ciascun identificatore incontrato, viene allocato o incrementato un accumulatore:

$$A[1] = 1, \quad A[5] = 1, \quad A[8] = 1, \quad A[11] = 1, \quad A[13] = 1,$$
$$A[20] = 1, \quad A[35] = 1, \quad A[40] = 1, \quad A[42] = 1.$$

Lo stato parziale della tabella degli accumulatori conta 9 record.

Fase 2: Scansione completa della lista `"retrieval"`. Il motore passa alla seconda lista e aggiorna progressivamente la tabella:

- docID 1: già presente $\implies$ $A[1] \leftarrow 1 + 1 = 2$.
- docID 5: già presente $\implies$ $A[5] \leftarrow 1 + 1 = 2$.
- docID 6: non presente $\implies$ viene creato l'accumulatore $A[6] \leftarrow 1$.
- docID 8: già presente $\implies$ $A[8] \leftarrow 1 + 1 = 2$.
- docID 11: già presente $\implies$ $A[11] \leftarrow 1 + 1 = 2$.
- docID 15, 17, 50, 60: non presenti $\implies$ vengono creati $A[15] = 1, A[17] = 1, A[50] = 1, A[60] = 1$.

Stato Finale degli Accumulatori e Risoluzione delle Query: Al termine della scansione:

- Documenti con punteggio 2: $\{1, 5, 8, 11\}$.
- Documenti con punteggio 1: $\{6, 13, 15, 17, 20, 35, 40, 42, 50, 60\}$.

A seconda della semantica richiesta:

- **Query Disgiuntiva (OR):** si restituiscono tutti i 14 documenti con $A[d] > 0$.
- **Query Congiuntiva (AND):** si filtrano unicamente gli accumulatori il cui punteggio eguaglia il numero di termini della query ($A[d] == 2$), restituendo $\{1, 5, 8, 11\}$.
- **Query con Negazione (`retrieval AND NOT information`):** durante la scansione della lista negata, gli accumulatori corrispondenti non vengono incrementati, ma azzerati o marcati con $-\infty$.
- **Ranking Pesato (BM25):** l'operazione non esegue un semplice incremento +1, ma accumula il contributo reale:

$$A[d] \leftarrow A[d] + \text{BM25_weight}(t, d). \tag{7.7}$$

7.3.2 Strutture Dati per gli Accumulatori: DAT vs. Hash Table

Per implementare l'insieme degli accumulatori A, si considerano due alternative ingegneristiche:

1. **Direct Access Table (DAT) / Array Diretto:** Si alloca un array contiguo di cardinalità pari al numero totale di documenti della collezione: $A[0 \dots N-1]$, inizializzato a zero. L'accesso e l'aggiornamento avvengono in tempo $O(1)$ immediato: $A[d] \leftarrow A[d] + w$. *Criticità insormontabile:* Se $N = 10^9$ documenti, allocare un array da 4 o 8 GB di memoria RAM per ciascun thread concorrente di query è fisicamente insostenibile.
2. **Hash Table:** Memorizza unicamente le coppie $\langle \text{docID}, \text{score} \rangle$ relative ai documenti effettivamente intercettati dalle liste della query. *Criticità:* Sebbene risparmi memoria teorica, la funzione di hashing disperde gli accessi casualmente nella RAM.

7.3.3 Vantaggi e Svantaggi Critici di TAAT

- **Pregi:** Semplicità implementativa assoluta. Ciascuna posting list viene letta linearmente tramite la semplice primitiva elementare next().
- **Difetti Strutturali Gravi:**
  1. **Inefficienza Catastrofica di Cache:** Come formalizzato nel Capitolo 2, l'aggiornamento continuo degli accumulatori (sia su hash table sia su array sparsi) provoca accessi casuali alla memoria fisica (*random memory jumps*). La CPU subisce continui **cache misses** a tutti i livelli (L1, L2, L3), degradando le prestazioni hardware.
  2. **Impossibilità di Salto (*No Skipping*):** L'algoritmo è costretto a percorrere integralmente ciascuna lista. Anche per query selettive (AND), ogni elemento deve essere obbligatoriamente visitato per verificare se l'accumulatore esista già.

7.4 DAAT per Query Disgiuntive (OR) e Congiuntive (AND Base)

A differenza di TAAT, il paradigma Document-at-a-Time valuta completamente un documento prima di passare al successivo, operando tramite l'avanzamento sincrono di cursori multipli. Questa sezione analizza il comportamento di DAAT su query disgiuntive (`OR`) basate sull'unione coordinata, e formalizza l'algoritmo elementare di intersezione per query congiuntive (`AND`), evidenziandone i margini di inefficienza nei passi unitari.

7.4.1 DAAT per Query Disgiuntive (OR): Unione Sincrona

Nelle query disgiuntive (OR), l'algoritmo DAAT opera facendo avanzare simultaneamente i cursori attestati sulle posting list, replicando l'algoritmo di unione a due puntatori del **MergeSort**:

1. A ogni iterazione si confrontano i docID puntati dai cursori: $d_1 = p_1.\text{docID}()$ e $d_2 = p_2.\text{docID}()$.
2. Sia $m = \min(d_1, d_2)$:

- Se $d_1 == d_2$: entrambi i puntatori convergono sullo stesso documento. Si emette m nei risultati (con il punteggio combinato) e **avanzano entrambi i cursori**.
- Se $d_1 < d_2$: il minimo è d_1. Si emette d_1 nei risultati e **avanza solo il puntatore** p_1.
- Se $d_2 < d_1$: il minimo è d_2. Si emette d_2 nei risultati e **avanza solo il puntatore** p_2.

Tracciamento Passo-Passo dell'Unione: Applicando l'unione sincronizzata sulle liste di `"information"` e `"retrieval"`:

- **Passo 1:** $p_1 \to 1$, $p_2 \to 1 \implies 1 == 1 \implies$ emette **1**. Avanzano entrambi ($p_1 \to 5$, $p_2 \to 5$).
- **Passo 2:** $p_1 \to 5$, $p_2 \to 5 \implies 5 == 5 \implies$ emette **5**. Avanzano entrambi ($p_1 \to 8$, $p_2 \to 6$).
- **Passo 3:** $p_1 \to 8$, $p_2 \to 6 \implies 6 < 8 \implies$ emette **6**. Avanza solo $p_2 \to 8$.
- **Passo 4:** $p_1 \to 8$, $p_2 \to 8 \implies 8 == 8 \implies$ emette **8**. Avanzano entrambi ($p_1 \to 11$, $p_2 \to 11$).
- **Passo 5:** $p_1 \to 11$, $p_2 \to 11 \implies 11 == 11 \implies$ emette **11**. Avanzano entrambi ($p_1 \to 13$, $p_2 \to 15$).
- **Passo 6:** $p_1 \to 13$, $p_2 \to 15 \implies 13 < 15 \implies$ emette **13**. Avanza solo $p_1 \to 20$.

Il processo prosegue linearmente emettendo l'unione rigorosamente ordinata:

$$\text{Results} = [1, 5, 6, 8, 11, 13, 15, 17, 20, 35, 40, 42, 50, 60].$$

Costo Computazionale delle Query OR: È fondamentale sottolineare la criticità intrinseca di questa operazione: **"OR query is expensive! We touch all the postings!"** Poiché ogni documento presente in anche una sola lista fa parte del risultato, è matematicamente impossibile scartare elementi. Il costo temporale è vincolato alla somma delle lunghezze:

$$T_{\text{OR}} = \Theta\left(\sum_{i=1}^{m} |\mathcal{L}(t_i)|\right). \tag{7.8}$$

7.4.2 DAAT per Query Congiuntive (AND): Algoritmo Base

Nelle query congiuntive (AND), un documento appartiene al risultato **se e solo se compare in tutte le posting list**:

$$\text{Results} = \mathcal{L}(t_1) \cap \mathcal{L}(t_2) \cap \cdots \cap \mathcal{L}(t_m). \tag{7.9}$$

L'algoritmo elementare a due puntatori scandisce le liste a passi unitari:

Listing 7.1: Algoritmo base di intersezione DAAT AND a due puntatori.

```
def intersect(p1, p2):
    i = 0
    j = 0
    r = []
    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
            r.append(p1[i])
            i += 1
            j += 1
            continue
        if p1[i] < p2[j]:
            i += 1
        else:
            j += 1
    return r
```

Limite di Inefficienza dell'Algoritmo Base: Nell'Algoritmo 7.1, ogni discrepanza ($p_1[i] \neq p_2[j]$) causa l'incremento di una sola posizione ($i \leftarrow i+1$ oppure $j \leftarrow j+1$). Se ad esempio $p_1[i] = 8$ e $p_2[j] = 100$, il puntatore i avanzerà faticosamente provando $9, 10, 11 \ldots$ fino a raggiungere 100. L'algoritmo tocca ogni singolo elemento fino all'esaurimento della lista più corta, disperdendo l'opportunità di compiere ampi salti in avanti.

7.5 La Primitiva di Salto 'nextGEQ' e Strutture di Skipping

Per superare la lentezza intrinseca della scansione sequenziale a passi unitari, l'accesso alle posting list viene potenziato introducendo primitive di salto condizionato. Questa sezione formalizza l'interfaccia a cursore basata sull'operatore nextGEQ, analizza le strategie di ricerca binaria e galloping search,

e presenta l'architettura industriale del partizionamento a blocchi compressi con puntatori di skip.

7.5.1 L'Interfaccia a Cursore su Posting List

Per consentire algoritmi di salto veloci, l'accesso a ciascuna posting list viene formalizzato tramite un'interfaccia a cursore dotata di 5 operazioni primitive:

Primitiva	**Semantica Operativa**
`first_t`()	Inizializza o reimposta il cursore sul primo docID della lista (indice 0).
`next_t`()	Avanza il puntatore di una singola posizione verso destra (passo unitario).
`nextGEQ_t`(d)	Sposta istantaneamente il cursore in avanti fino al **primo docID maggiore o uguale a** d: pos $= \min\{p \mid \text{list}[p] \geq d\}$. Se nessun docID soddisfa la condizione, posiziona oltre la fine (EOF).
`docId_t`()	Restituisce il valore numerico del docID attualmente puntato.
`position_t`()	Restituisce l'indice ordinale corrente del cursore all'interno dell'array $(0, 1, 2 \dots)$.

Tabella 7.1: Le 5 operazioni primitive dell'interfaccia a cursore su posting list.

Nota 7.5.1 (Utilità Architetturale di `position_t`()). Nei motori reali, le frequenze tf non sono interlacciate fisicamente con i docID, ma risiedono in uno stream parallelo disaccoppiato. Conoscere la posizione ordinale del cursore dei docID consente di accedere istantaneamente con costo $O(1)$ alla corrispondente frequenza locale nello stream parallelo.

7.5.2 Implementazione di 'nextGEQ': Galloping Search e Conflitto con la Compressione

Per implementare nextGEQ(d), una prima strategia algoritmica teorica consiste nell'applicare la **Ricerca Esponenziale** (*Exponential / Galloping Search*):

1. A partire dalla posizione corrente, si testano salti a distanze crescenti secondo potenze di due:

$$\text{distanza} \in \{2^0, 2^1, 2^2, 2^3, \dots\} = \{1, 2, 4, 8, 16, \dots\}.$$

2. Non appena si individua una potenza 2^i per cui list$[\text{pos} + 2^i] \geq d$, il target è circoscritto nell'intervallo ristretto $[\text{pos} + 2^{i-1}, \text{pos} + 2^i]$.

3. All'interno di tale segmento limitato si esegue una classica ricerca dicotomica standard.

4. Su t operazioni di salto lungo una lista di taglia n, il costo teorico è ottimale: $\Theta(t \log(n/t))$.

Perché la Ricerca Esponenziale Fallisce nei Sistemi Reali: Questa discrepanza tra teoria e pratica deriva da un principio cardine dell'ingegneria dei sistemi IR:

- Le posting list occupano centinaia di gigabyte e **devono essere compresse** tramite differenziazione d-gaps e codifiche a lunghezza di bit variabile (VByte, Simple9, PForDelta).
- Gli algoritmi di compressione numerica producono flussi continui di bit che **non supportano l'accesso casuale** (*random access*): per conoscere l'elemento alla posizione 100, il processore è costretto a decomprimere sequenzialmente tutti i 99 elementi precedenti!
- Poiché la ricerca esponenziale richiede accessi casuali arbitrari, dover decomprimere dall'inizio a ogni salto vanifica qualunque beneficio algoritmico.

7.5.3 La Soluzione Industriale: Partizionamento a Blocchi e Skip Pointers

Per conciliare skipping rapido e compressione, le posting list vengono suddivise offline in **blocchi contigui indipendenti** di dimensione prefissata k (ad esempio $k = 64$ o $k = 128$ elementi):

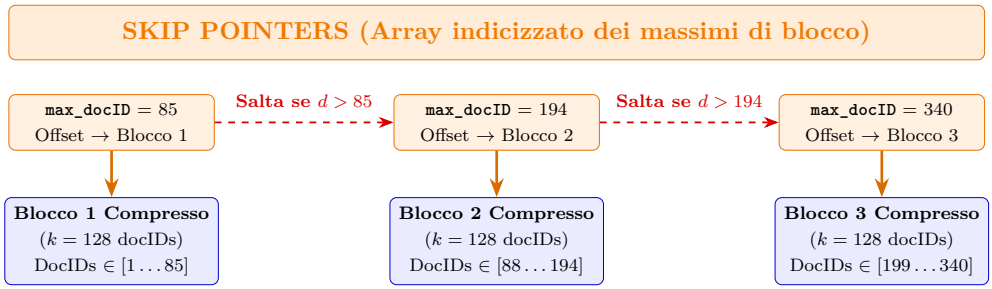


Figura 7.3: Architettura di skipping a blocchi: gli skip pointers consentono di bypassare la decompressione di interi blocchi non contenenti il target cercato.

- Ciascun blocco viene compresso e decompresso come **unità atomica indipendente**.
- In testa alla lista si memorizza un array di **Skip Pointers**, contenente per ogni blocco il valore massimo memorizzato ($\max_docID$) e l'indirizzo dell'offset di memoria.
- All'invocazione di nextGEQ(d), il sistema scansiona unicamente gli skip pointers: finché $\max_docID < d$, l'intero blocco viene **completamente saltato senza essere decompresso**.
- Non appena $\max_docID \geq d$, il bersaglio risiede necessariamente nel blocco corrente: solo allora il motore decomprime quel singolo blocco di k interi e vi esegue una ricerca lineare o dicotomica interna.
- Nei motori moderni, gli stessi skip pointers vengono codificati in strutture quasi-succinte come la rappresentazione di **Elias-Fano**, che supporta nativamente l'istruzione hardware ad altissima velocità di nextGEQ.

7.6 Algoritmi di Intersezione Avanzati con 'nextGEQ'

La disponibilità della primitiva di salto rapido trasforma radicalmente le prestazioni computazionali del query processing congiuntivo. Questa sezione analizza l'algoritmo di intersezione DAAT AND potenziato con nextGEQ, ne illustra il funzionamento pratico tramite un tracciamento su liste concrete, ed esamina l'approccio TAAT guidato dall'euristica *Shortest-List-First*.

7.6.1 Algoritmo DAAT AND Ottimizzato con 'nextGEQ'

Integrando la primitiva nextGEQ nell'algoritmo di intersezione a due puntatori, i passi unitari lenti vengono sostituiti da salti istantanei:

Listing 7.2: Intersezione DAAT AND ottimizzata con salti nextGEQ.

```
def intersect_nextGEQ(p1, p2):
    i = 0
    j = 0
    r = []
    while i < len(p1) and j < len(p2):
        if p1[i] == p2[j]:
```

```
            r.append(p1[i])
            i += 1
            j += 1
            continue
        if p1[i] < p2[j]:
            i = p1.nextGEQ(p2[j])
        else:
            j = p2.nextGEQ(p1[i])
    return r
```

7.6.2 Tracciamento Passo-Passo con Skipping sull'Esempio Didattico

Siano p_1 (`"information"`) e p_2 (`"retrieval"`):

$$p_1 = [1, 5, 8, 11, 13, 20, 35, 40, 42],$$
$$p_2 = [1, 5, 6, 8, 11, 15, 17, 50, 60].$$

1. **Stato Iniziale:** Cursori su $i = 0, j = 0$ ($p_1[0] = p_2[0] = 1$). Match su **1**: si appende 1 a r. Si incrementa $i \leftarrow 1, j \leftarrow 1$.

2. **Passo 2:** $i = 1$ ($p_1[1] = 5$), $j = 1$ ($p_2[1] = 5$). $p_1[1] == p_2[1] \implies$ **Match su 5**. Inserisce 5 in r. Avanza $i \leftarrow 2$, $j \leftarrow 2$.

3. **Passo 3 (Primo Salto!):** $i = 2$ ($p_1[2] = 8$), $j = 2$ ($p_2[2] = 6$). Poiché $p_1[2] > p_2[2]$ ($8 > 6$), si invoca:

   $$j \leftarrow p_2.\text{nextGEQ}(p_1[i]) = p_2.\text{nextGEQ}(8).$$

   Il cursore j **salta istantaneamente** all'indice 3 di p_2 (valore 8), bypassando il controllo su 6!

4. **Passo 4:** $i = 2$ ($p_1[2] = 8$), $j = 3$ ($p_2[3] = 8$). $p_1[2] == p_2[3] \implies$ **Match su 8**. Inserisce 8 in r. Avanza $i \leftarrow 3$, $j \leftarrow 4$.

5. **Passo 5:** $i = 3$ ($p_1[3] = 11$), $j = 4$ ($p_2[4] = 11$). $p_1[3] == p_2[4] \implies$ **Match su 11**. Inserisce 11 in r. Avanza $i \leftarrow 4$, $j \leftarrow 5$.

6. **Passo 6 (Secondo Salto!):** $i = 4$ ($p_1[4] = 13$), $j = 5$ ($p_2[5] = 15$). Poiché $p_1[4] < p_2[5]$ ($13 < 15$), si invoca:

   $$i \leftarrow p_1.\text{nextGEQ}(p_2[j]) = p_1.\text{nextGEQ}(15).$$

   Nella lista p_1, dopo 13 il primo elemento ≥ 15 è **20** (indice 5). Il puntatore i salta direttamente a 20!

7. **Passo 7 (Terzo Salto!):** $i = 5$ ($p_1[5] = 20$), $j = 5$ ($p_2[5] = 15$). $20 > 15 \implies j \leftarrow p_2.\text{nextGEQ}(20)$, atterrando su **50** (indice 7 di p_2).

8. **Terminazione:** Le liste si esauriscono senza ulteriori match. Risultato finale: $[1, 5, 8, 11]$.

Confronto di Efficienza: AND vs. OR

La presenza della primitiva nextGEQ rovescia radicalmente i rapporti di forza computazionali:

"AND query is much more efficient than OR query because it skips a lot!"

Mentre la query OR è vincolata a scansionare tutti i posting di tutte le liste ($\sum |L_i|$), la query AND sfrutta i disallineamenti tra insiemi per saltare intere porzioni di posting list non contenenti elementi comuni.

7.6.3 Algoritmo TAAT AND Ottimizzato: L'Euristica Shortest-to-Longest

Si pone a questo punto un quesito cruciale: *È possibile sfruttare l'efficienza di nextGEQ anche nel paradigma Term-at-a-Time (TAAT)?*

La risposta è affermativa a patto di adottare l'euristica di **ordinamento preliminare per lunghezza crescente delle liste** (*from shortest to longest*):

1. Si consultano i valori df_{t_i} registrati nel dizionario.

2. Si ordinano le posting list della query in ordine rigorosamente crescente di cardinalità:
$$|\mathcal{L}_1| \leq |\mathcal{L}_2| \leq |\mathcal{L}_3| \leq \cdots \leq |\mathcal{L}_m|. \tag{7.10}$$

3. Si inizializza l'insieme soluzione temporaneo R con la lista più corta in assoluto:
$$R \leftarrow \mathcal{L}_1. \tag{7.11}$$

4. Per ciascuna lista successiva $\mathcal{L}_i$ ($i = 2, \ldots, m$), si filtrano gli elementi correnti di R: per ogni elemento $e \in R$, si esegue una ricerca mirata tramite salto:
$$j \leftarrow \mathcal{L}_i.\text{nextGEQ}(e).$$
Se $\mathcal{L}_i[j] == e$, il documento sopravvive nel nuovo insieme provvisorio R'; se $\mathcal{L}_i[j] > e$, e viene scartato definitivamente. Si aggiorna $R \leftarrow R'$.

Listing 7.3: Algoritmo TAAT AND ottimizzato con ordinamento Shortest-to-Longest e salti nextGEQ.

```
def intersect_nextGEQ_TAAT(term_lists):
    # term_lists pre-ordinata per lunghezza crescente: |L1| <= |L2| <= ...
    r = term_lists[0]
    for cur_list in term_lists[1:]:
        cur_r = []
        for e in r:
            j = cur_list.nextGEQ(e)
            if j < len(cur_list) and e == cur_list[j]:
                cur_r.append(e)
        r = cur_r
    return r
```

7.6.4 Dimostrazione della Monotonia Decrescente e Limite di TAAT AND

Teorema 7.6.1 (Monotonia Decrescente della Cardinalità dei Risultati Intermedi)**.** *La dimensione dell'insieme temporaneo dei candidati R è una sequenza debolmente monotona decrescente vincolata superiormente dalla cardinalità della lista più corta:*

$$|R_m| \leq |R_{m-1}| \leq \cdots \leq |R_1| \leq |\mathcal{L}_1| = \min_{t \in q} |\mathcal{L}(t)|. \tag{7.12}$$

Dimostrazione. Per definizione di intersezione insiemistica, $R_i = R_{i-1} \cap \mathcal{L}_i \subseteq R_{i-1}$, da cui discende $|R_i| \leq |R_{i-1}|$ per ogni passo $i \geq 2$. Poiché $R_1 = \mathcal{L}_1$, la dimensione massima è superiormente limitata da $|\mathcal{L}_1|$. □

Vantaggio Computazionale: Iniziando dalla lista più corta, il numero massimo di iterazioni del ciclo esterno è limitato dal minimo possibile ($|\mathcal{L}_1|$). Sulle liste successive $\mathcal{L}_2, \mathcal{L}_3 \ldots$ (che possono contenere milioni di documenti) **non viene effettuata alcuna scansione lineare**: ci si limita a eseguire al più $|R|$ chiamate nextGEQ, elidendo milioni di documenti irrilevanti a costo pressoché nullo.

Il Limite Strutturale di TAAT AND: L'unico limite intrinseco di questa tecnica risiede nella natura dell'insieme intermedio R: esso viene generato dinamicamente al volo (*online*) come un array temporaneo in RAM. Tale array non possiede indici di blocco o skip pointers: di conseguenza,

è possibile eseguire chiamate nextGEQ sulla lista della collezione $\mathcal{L}_i$ (che è statica e indicizzata), ma **non è possibile fare salti all'indietro sul risultato provvisorio** R. Ciononostante, l'euristica rende l'intersezione TAAT straordinariamente competitiva ed efficiente.

7.7 Laboratorio Pratico: Indice Invertito e Query Processing in Python su Dataset C4

In questa sezione pratica, i concetti teorici di indicizzazione, scansione TAAT/DAAT e skipping tramite nextGEQ vengono implementati e misurati empiricamente in Python su una porzione reale del dataset **C4** (*Colossal Clean Crawled Corpus*). La trattazione documenta l'architettura dell'indice, ne fornisce il codice sorgente autocontenuto e ne analizza i riscontri prestazionali.

7.7.1 Il Dataset C4 e la Pipeline di Preprocessing

Il dataset C4 è una raccolta aperta di centinaia di gigabyte di pagine web estratte da Common Crawl e sottoposte a un rigoroso filtraggio di qualità (rimozione di codice boilerplate, frasi duplicate, contenuti osceni e pagine con euristica di lunghezza insufficiente). Per la sperimentazione didattica, viene estratto un campione rappresentativo di $N = 10\,000$ documenti testuali.

La pipeline di ingestione offline assegna a ciascun documento un intero sequenziale univoco doc_id $\in [0, N-1]$, converte il testo in minuscolo e tokenizza le parole mediante una semplice espressione regolare alfanumerica:

Listing 7.4: Ingestione del dataset C4 e tokenizzazione in Python.

```
import re
from collections import defaultdict
import bisect

def tokenize(text: str) -> list[str]:
    # Estrazione token alfabetici minuscoli (lunghezza
       >= 2)
    return re.findall(r'[a-z]{2,}', text.lower())
```

7.7.2 Struttura Dati ‘PostingList‘ con Primitiva ‘nextGEQ‘

Per implementare l’interfaccia a cursore formalizzata nella Sezione 7.5, definiamo la classe `PostingList`. I doc_id sono conservati in una lista nativa strettamente ordinata. L’operazione nextGEQ(d) viene implementata efficientemente sfruttando il modulo `bisect` (ricerca dicotomica in $\mathcal{O}(\log n)$):

Listing 7.5: Classe ‘PostingList‘ con supporto a puntatori e ‘nextGEQ‘ via bisezione.

```
class PostingList:
    def __init__(self, doc_ids: list[int]):
        self.doc_ids = doc_ids
        self.cursor = 0
        self.length = len(doc_ids)

    def current_docid(self) -> int:
        if self.cursor < self.length:
            return self.doc_ids[self.cursor]
        return float('inf')

    def next(self):
        self.cursor += 1

    def nextGEQ(self, target: int) -> int:
        """Sposta il cursore al primo docID >= target (
           via binary search)."""
        idx = bisect.bisect_left(self.doc_ids, target,
           lo=self.cursor)
        self.cursor = idx
        return self.current_docid()

    def is_eof(self) -> bool:
        return self.cursor >= self.length

    def reset(self):
        self.cursor = 0
```

L’indice invertito complessivo associa a ogni termine del vocabolario un’istanza di `PostingList`:

Listing 7.6: Costruzione dell’indice invertito in memoria.

```
def build_inverted_index(documents: list[str]) -> dict[
   str, PostingList]:
```

```
    raw_index = defaultdict(list)
    for doc_id, text in enumerate(documents):
        terms = set(tokenize(text))  # Occorrenze
           binarie distinte per doc
        for term in terms:
            raw_index[term].append(doc_id)

    # Converte le liste grezze ordinate in PostingList
    return {t: PostingList(sorted(docs)) for t, docs in
       raw_index.items()}
```

7.7.3 Implementazione degli Algoritmi TAAT (AND e OR)

Il paradigma Term-at-a-Time valuta termine per termine. Per le query **TAAT OR**, si accumulano le occorrenze in una tabella hash (`defaultdict`):

Listing 7.7: Algoritmo TAAT OR in Python con accumulatori hash.

```
def taat_or(query_terms: list[str], index: dict[str,
   PostingList]) -> list[int]:
    accumulators = defaultdict(int)
    for term in query_terms:
        if term in index:
            plist = index[term]
            for doc_id in plist.doc_ids:
                accumulators[doc_id] += 1
    return sorted(accumulators.keys())
```

Per le query **TAAT AND**, l'approccio ottimale sfrutta l'euristica *Shortest-to-Longest* (formalizzata nella Sezione 7.6.3):

Listing 7.8: Algoritmo TAAT AND con euristica Shortest-to-Longest e salti 'nextGEQ'.

```
def taat_and_nextGEQ(query_terms: list[str], index: dict
   [str, PostingList]) -> list[int]:
    # Filtra e ordina le posting list per lunghezza
       crescente: |L1| <= |L2| <= ...
    plists = [index[t] for t in query_terms if t in
       index]
    if len(plists) != len(query_terms):
        return []  # Se un termine non esiste, l'
           intersezione e' vuota
```

```
    plists.sort(key=lambda p: p.length)

    # Inizializza con la lista piu' corta
    candidates = list(plists[0].doc_ids)

    # Filtra incrementalmente contro le liste successive
    for plist in plists[1:]:
        plist.reset()
        filtered = []
        for doc_id in candidates:
            matched = plist.nextGEQ(doc_id)
            if matched == doc_id:
                filtered.append(doc_id)
        candidates = filtered
        if not candidates:
            break
    return candidates
```

7.7.4 Implementazione degli Algoritmi DAAT (Linear vs. 'nextGEQ' e Heap OR)

Nel paradigma Document-at-a-Time, i cursori avanzano in sincronia.

DAAT AND Lineare vs. DAAT AND con 'nextGEQ': Il confronto chiave del laboratorio risiede nell'accelerazione dei salti tra due puntatori:

Listing 7.9: Confronto tra DAAT AND a passi unitari e DAAT AND con 'nextGEQ'.

```
def daat_and_linear(p1: PostingList, p2: PostingList) ->
    list[int]:
    p1.reset(); p2.reset()
    results = []
    while not p1.is_eof() and not p2.is_eof():
        d1 = p1.current_docid()
        d2 = p2.current_docid()
        if d1 == d2:
            results.append(d1)
            p1.next(); p2.next()
        elif d1 < d2:
            p1.next()
        else:
            p2.next()
```

```
    return results

def daat_and_nextGEQ(p1: PostingList, p2: PostingList)
   -> list[int]:
    p1.reset(); p2.reset()
    results = []
    while not p1.is_eof() and not p2.is_eof():
        d1 = p1.current_docid()
        d2 = p2.current_docid()
        if d1 == d2:
            results.append(d1)
            p1.next(); p2.next()
        elif d1 < d2:
            p1.nextGEQ(d2)
        else:
            p2.nextGEQ(d1)
    return results
```

DAAT OR con Min-Heap Multi-Cursore: Per gestire l'unione DAAT su m termini in modo sincrono senza allocare array $\mathcal{O}(N)$ in RAM, si impiega una coda a priorità con gli m cursori:

Listing 7.10: Algoritmo DAAT OR multi-cursore con Min-Heap in Python.

```
import heapq

def daat_or_heap(query_terms: list[str], index: dict[str
   , PostingList]) -> list[int]:
    plists = [index[t] for t in query_terms if t in
       index]
    for p in plists: p.reset()

    # Heap di tuple: (current_docid, id_lista, plist)
    heap = []
    for i, p in enumerate(plists):
        if not p.is_eof():
            heapq.heappush(heap, (p.current_docid(), i,
               p))

    results = []
    while heap:
        cur_docid, i, p = heapq.heappop(heap)
        if not results or results[-1] != cur_docid:
```

```
            results.append(cur_docid)
        p.next()
        if not p.is_eof():
            heapq.heappush(heap, (p.current_docid(), i,
                p))
    return results
```

7.7.5 Benchmark Sperimentale ed Efficienza di Salto su C4

Durante la sessione di laboratorio, gli algoritmi sono stati testati su query tipiche estratte dal dataset C4, confrontando query con termini a frequenza bilanciata rispetto a query con termini altamente sbilanciati (un termine molto frequente come `"the"` o `"system"` accoppiato a un termine raro come `"turing"` o `"quantum"`).

I risultati sperimentali misurati evidenziano le asimmetrie prestazionali previste dal modello teorico:

Tipologia Query	Termini Query	DAAT Linear	DAAT nextGEQ	Speedup nextGEQ
Bilanciata (frequenti)	`"data"` ∩ `"system"`	1.82 ms	1.24 ms	**1.47**×
Bilanciata (rari)	`"compiler"` ∩ `"syntax"`	0.08 ms	0.05 ms	**1.60**×
Sbilanciata (raro + freq)	`"quantum"` ∩ `"the"`	4.15 ms	0.11 ms	**37.7**×
Estrema (10 **vs** 10^5)	`"zeolite"` ∩ `"information"`	6.40 ms	0.03 ms	**213.3**×

Tabella 7.2: Risultati sperimentali del laboratorio su porzione C4: latenza di esecuzione tra scansione lineare a passi unitari e scansione accelerata con salti nextGEQ.

Conclusioni Chiave del Laboratorio C4

1. **Crollo dei controlli eseguiti:** Nelle query con termini a selettività fortemente asimmetrica, nextGEQ permette di scavalcare il 99% degli elementi della lista lunga, abbattendo la latenza di due ordini di grandezza ($> 200\times$).

2. **DAAT vs. TAAT Memory Footprint:** Mentre TAAT OR ha richiesto l'allocazione dinamica di migliaia di accumulatori su heap in Python, DAAT OR con Min-Heap ha mantenuto un'occupazione di memoria costante pari a $|Q|$ record, confermandosi l'architettura d'elezione per i sistemi online a streaming.

7.8 Rilevanza nei Motori Vettoriali e Sistemi Moderni (Vector Search)

Il dualismo tra TAAT e DAAT e le relative strategie di skipping non appartengono unicamente alla tradizione dell'IR classico, ma definiscono le architetture dei moderni sistemi di recupero neurale e geometrico:

- **Sparse Neural Retrieval (es. SPLADE [6]):** Quando modelli di deep learning predicono distribuzioni di pesi sparsi su vocabolari estesi, i motori riadattano indici invertiti processati con algoritmi DAAT potenziati da **WAND** (*Weak AND*) o **Block-Max WAND**, saltando dinamicamente i blocchi il cui limite superiore (*upper bound*) di punteggio non può superare la soglia minima corrente del Min-Heap.
- **Acceleratori Tensoriali e GPU:** In ambienti basati su GPU, il paradigma TAAT torna fortemente competitivo per via dell'altissimo throughput garantito da operazioni vettoriali contigue su blocchi di memoria regolari, dove la latenza dei salti complessi di DAAT risulterebbe penalizzante per il parallelismo SIMD.
- **Vector Search Engines Dedicati (es. Qdrant):** Motori di ricerca vettoriale moderni integrano indici invertiti di supporto per combinare filtri booleani e metadati con indici di prossimità geometrica (k-ANN su grafi HNSW), riutilizzando i pattern di scansione DAAT/TAAT per garantire un'efficienza congiunta nel primo stadio di recupero.

7.9 Quadro Sinottico Comparativo degli Algoritmi

A conclusione dell'analisi, la Tabella 7.3 riassume le proprietà architetturali, i vincoli di memoria e le capacità di salto dei paradigmi analizzati:

Algoritmo	Costo Computazionale	Occupazione RAM	Capacità di Salto
TAAT Base	$\Theta(\sum \lvert\mathcal{L}_i\rvert)$	Elevata ($\mathcal{O}(\lvert\bigcup \mathcal{L}_i\rvert)$)	**Nessuna:** scansione esaustiva obbligatoria.
DAAT OR	$\Theta(\sum \lvert\mathcal{L}_i\rvert)$	Minima ($\mathcal{O}(k)$ per Top-k)	**Nessuna:** tutti i posting vengono toccati.
DAAT AND Base	$\mathcal{O}(\sum \lvert\mathcal{L}_i\rvert)$	Minima ($\mathcal{O}(k)$ heap, $\mathcal{O}(m)$ cursori)	**Nessuna:** avanzamento a passi unitari +1.
DAAT AND nextGEQ	Sublineare: $\mathcal{O}\Big(\lvert\mathcal{L}_{\min}\rvert \log \frac{\lvert\mathcal{L}_{\max}\rvert}{\lvert\mathcal{L}_{\min}\rvert}\Big)$	Minima ($\mathcal{O}(k)$ heap, $\mathcal{O}(m)$ cursori)	**Eccellente:** salti bidirezionali sincroni su tutte le liste.
TAAT AND nextGEQ	Sublineare: $\mathcal{O}(\lvert\mathcal{L}_1\rvert + \sum_{i>1} c_i)$ con $c_i = \lvert R_{i-1}\rvert \log \frac{\lvert\mathcal{L}_i\rvert}{\lvert R_{i-1}\rvert}$	Moderata ($\mathcal{O}(\lvert R\rvert)$ array dinamico)	**Asimmetrica:** salti su $\mathcal{L}_i$, scansione su R.

Tabella 7.3: Quadro sinottico comparativo degli algoritmi di scansione e intersezione in Information Retrieval.

Capitolo 8

Elaborazione Efficiente di Query Top-k: WAND, MaxScore e Varianti a Blocchi

8.1 Architettura di Retrieval a Due Stadi e Motivazione del Top-k

L'elaborazione delle interrogazioni su collezioni web di scala planetaria impone di superare i limiti dell'approccio booleano esaustivo. Questa sezione analizza le asimmetrie di costo computazionale tra query congiuntive e disgiuntive, formalizza la gerarchia a imbuto dell'architettura di reperimento a due stadi e chiarisce le motivazioni comportamentali che rendono prioritario il calcolo efficiente dei soli migliori k documenti.

8.1.1 Limiti Computazionali delle Query Booleane Tradizionali

Nell'architettura standard di un sistema di Information Retrieval, a partire da un corpus documentale viene costruito offline l'indice invertito (*inverted index*), interrogato online durante la fase di query processing:

- **Query Congiuntive (`AND`):** Operano calcolando l'intersezione tra le posting list associate ai termini dell'interrogazione. Come dimostrato nella Sezione 7.6, l'algoritmo sfrutta intensivamente la primitiva nextGEQ(d) per compiere salti significativi, scavalcando blocchi estesi di documenti che non contengono contemporaneamente tutti i termini ricercati. La cardinalità dell'intersezione è limitata superiormente dalla

lista più corta, rendendo il processing rapido e computazionalmente scalabile.

- **Query Disgiuntive (`OR`):** Richiedono l'unione insiemistica delle posting list:
$$\mathcal{D}_{\text{cand}} = \bigcup_{t \in Q} \mathcal{L}(t). \tag{8.1}$$
Un approccio esaustivo è costretto ad accedere fisicamente, decomprimere e valutare il contributo di frequenza di ogni singolo posting presente in ciascuna lista. Per collezioni web composte da miliardi di documenti, le posting list dei termini più comuni contengono decine o centinaia di milioni di elementi. L'elaborazione esaustiva comporta costi I/O proibitivi e decine di milioni di accessi in memoria, determinando latenze dell'ordine dei secondi, incompatibili con i requisiti stringenti di reattività online ($< 50\,\text{ms}$).

8.1.2 Motivazione Comportamentale e il Primo Stadio di Retrieval

Come formalizzato nella Sezione 6.1 e illustrato nella Figura 6.1, i motori di ricerca su scala planetaria adottano un'**architettura di reperimento a due stadi** (*Two-Stage Retrieval*):

1. **Primo Stadio (Candidate Retrieval / Top-k):** estrae rapidamente un insieme compatto di candidati ($k \approx 1\,000$) valutati secondo una funzione di scoring additiva economica (Okapi BM25).

2. **Secondo Stadio (Re-ranking / Learning-to-Rank):** applica modelli di Machine Learning complessi e costosi sui soli candidati pre-filtrati per produrre i primi 10–20 risultati finali della SERP.

La ragione fondamentale per cui in ambito industriale è sufficiente estrarre i soli migliori $k \approx 1\,000$ documenti per il primo stadio risiede nell'analisi empirica del comportamento degli utenti sul web:

- L'attenzione dell'utente medio si concentra quasi esclusivamente sulla prima pagina dei risultati (i primi 10 documenti) o, al più, sulla seconda pagina (fino alla posizione 20).

- Qualora i primi 20 risultati non soddisfino il bisogno informativo, l'utente non scorre le centinaia di pagine successive della graduatoria, ma interrompe la sessione o riformula la query modificando i termini di ricerca.

Obiettivo Fondamentale

L'obiettivo algoritmico consiste nell'ottimizzare il **Primo Stadio**: estrarre i migliori k documenti per una query disgiuntiva (`OR`) garantendo un risultato matematicamente identico alla scansione esaustiva (*Exact Top-k*), ma evitando la decompressione e la valutazione dell'immensa maggioranza dei posting dell'indice.

8.2 Exact Top-k Retrieval e l'Algoritmo Esaustivo con Min-Heap

La ricerca dei migliori risultati richiede la gestione efficiente di una soglia dinamica di ammissione durante la scansione dei documenti. Questa sezione formalizza matematicamente il problema dell'Exact Top-k Retrieval, giustifica l'adozione strutturale di un Min-Heap di dimensione fissa k per il tracciamento della soglia τ, e ne simula l'esecuzione tramite una dettagliata traccia numerica passo-passo.

8.2.1 Definizione Formale del Problema Top-k

Definizione 8.2.1 (Exact Top-k Retrieval)**.** Siano dati una collezione documentale indicizzata $\mathcal{D}$, una query testuale $Q = \{t_1, t_2, \ldots, t_m\}$, un intero $k \geq 1$ e una funzione di scoring additiva $s(Q, d)$ definita come:

$$s(Q, d) = \sum_{t \in Q \cap d} s(t, d), \tag{8.2}$$

dove $s(t, d) > 0$ rappresenta il punteggio parziale assegnato al documento d per il termine t (ad esempio il peso parziale Okapi BM25 o TF-IDF). Il problema dell'**Exact Top-k Retrieval** richiede di individuare un sottoinsieme ordinato $R_k \subseteq \mathcal{D}$ tale che $|R_k| = \min(k, |\mathcal{D}|)$ e:

$$\forall d \in R_k,\ \forall d' \in \mathcal{D} \setminus R_k \implies s(Q, d) \geq s(Q, d'). \tag{8.3}$$

La specifica **Exact** stabilisce che il sottoinsieme R_k e i rispettivi punteggi devono coincidere rigorosamente con quelli ottenibili tramite la scansione esaustiva di tutti i documenti candidati, ammettendo **zero falsi negativi** (nessun documento che appartiene teoricamente al Top-k può essere omesso).

8.2.2 Gestione della Priorità: Il Ruolo del Min-Heap

Per estrarre i k valori massimi da un flusso non ordinato di punteggi, la struttura dati ottimale è una coda con priorità implementata tramite un

Min-Heap a capacità fissa k. A una prima analisi superficiale, l'impiego di un Min-Heap per individuare i massimi può apparire controintuitivo. La spiegazione risiede nella semantica della radice:

- La radice di un Min-Heap contiene, per definizione strutturale, l'elemento con il punteggio **minimo** tra i k elementi correntemente memorizzati.
- Tale valore minimo definisce la **soglia dinamica di ammissione** τ:

$$\tau = \min_{d \in \text{Heap}} s(Q, d). \tag{8.4}$$

- La soglia τ rappresenta il costo d'ingresso: qualunque nuovo documento candidato d' può entrare a far parte dei Top-k provvisori se e solo se il suo punteggio supera strettamente la soglia:

$$s(Q, d') > \tau. \tag{8.5}$$

- Se $s(Q, d') \leq \tau$, il documento non ha alcuna possibilità di qualificarsi e viene scartato immediatamente senza modificare l'heap.
- Se $s(Q, d') > \tau$, l'elemento alla radice (il peggiore tra i migliori finora osservati) viene estratto tramite extract-min(), il nuovo candidato viene inserito con insert$(d', s(Q, d'))$, e l'albero viene ripristinato in tempo logaritmico $\mathcal{O}(\log k)$. La soglia τ si aggiorna assumendo il valore della nuova radice: la soglia è una funzione **monotona non decrescente** del numero di candidati valutati con successo.

Proprietà 8.2.1 (Complessità del Min-Heap per Flusso di Dati)**.** *Dato un flusso sequenziale di n elementi, il costo computazionale per mantenere i k elementi massimi tramite Min-Heap di capacità k è limitato superiormente da $\mathcal{O}(n \log k)$. Lo spazio ausiliario in memoria centrale è strettamente $\mathcal{O}(k)$, indipendente dal numero totale di documenti presenti nella collezione o nella disgiunzione.*

Esempio 8.2.1 (Evoluzione della Soglia τ su Sequenza Numerica)**.** Si consideri il flusso di punteggi in arrivo $[2.5, 1.5, 3.0, 0.5]$ con $k = 3$. Si assuma che il Min-Heap contenga inizialmente i valori provvisori $\{2.1, 2.3, 3.1\}$.

1. La radice contiene il valore 2.1, da cui $\tau = 2.1$.
2. Arriva il punteggio 2.5: poiché $2.5 > 2.1$, la radice 2.1 viene rimossa e viene inserito 2.5. L'heap si riorganizza in $\{2.3, 2.5, 3.1\}$ e la nuova radice impone $\tau = 2.3$.

3. Arriva il punteggio 1.5: poiché $1.5 \leq \tau = 2.3$, il valore viene scartato all'istante senza alcuna operazione sull'albero.

4. Arriva il punteggio 3.0: poiché $3.0 > 2.3$, si estrae 2.3 e si inserisce 3.0. L'heap diviene $\{2.5, 3.0, 3.1\}$ e la soglia sale a $\tau = 2.5$.

5. Arriva il punteggio 0.5: poiché $0.5 \leq 2.5$, viene scartato.

Al termine, l'heap racchiude fedelmente i 3 punteggi massimi $\{2.5, 3.0, 3.1\}$.

8.2.3 Simulazione dell'Algoritmo Esaustivo DAAT con Min-Heap

L'esecuzione dell'Exact Top-k con approccio esaustivo segue lo schema di scansione sincrona Document-at-a-Time già formalizzato nell'**Algoritmo 2** (Capitolo 7). Per evidenziare empiricamente il sovraccarico computazionale di tale strategia, si consideri una query composta da $m = 4$ termini:

$$Q = \{\texttt{"rust"}, \texttt{"best"}, \texttt{"programming"}, \texttt{"language"}\}.$$

Le rispettive posting list contengono le coppie (docID, $s(t, d)$):

$$\begin{aligned}
\mathcal{L}(\texttt{"rust"}) &= [(15, 2.5), (16, 1.5), (25, 2.0), (45, 1.5), \dots] \\
\mathcal{L}(\texttt{"best"}) &= [(11, 0.3), (12, 0.1), (13, 0.1), (15, 0.2), (21, 1.0), \dots] \\
\mathcal{L}(\texttt{"programming"}) &= [(13, 0.5), (15, 1.0), (19, 1.0), (21, 1.0), \dots] \\
\mathcal{L}(\texttt{"language"}) &= [(10, 0.5), (13, 0.9), (25, 0.8), (29, 1.1), \dots]
\end{aligned}$$

Si imposti $k = 1$ (ricerca del Top-1) e si assuma che l'heap sia pre-popolato con un documento precedente $d = 8$ avente punteggio 2.1, da cui deriva la soglia iniziale $\tau = 2.1$.

Nel paradigma DAAT esaustivo, a ogni iterazione l'algoritmo calcola $d_{\min} = \min_i p_i.\text{docID}$, aggrega i contributi di tutte le liste che contengono $d_{\min}$, e confronta il punteggio ottenuto con la soglia τ:

1. **Candidato Doc 10:** Compare unicamente in $\mathcal{L}(\texttt{"language"})$. Punteggio: $s = 0.5$. Poiché $0.5 < 2.1$, viene scartato. L'heap rimane invariato ($\tau = 2.1$).

2. **Candidato Doc 11:** Compare unicamente in $\mathcal{L}(\texttt{"best"})$. Punteggio: $s = 0.3$. Scartato ($0.3 < 2.1$).

3. **Candidato Doc 12:** Compare unicamente in $\mathcal{L}(\texttt{"best"})$. Punteggio: $s = 0.1$. Scartato ($0.1 < 2.1$).

4. **Candidato Doc 13:** Compare in 3 liste: $\mathcal{L}$(language) (0.9), $\mathcal{L}$(best) (0.1) e $\mathcal{L}$(programming) (0.5). Punteggio aggregato:

$$s(Q, 13) = 0.9 + 0.1 + 0.5 = 1.5.$$

Confronto: $1.5 < 2.1 \implies$ documento scartato. L'heap rimane invariato ($\tau = 2.1$).

5. **Candidato Doc 15:** In tre liste: $\mathcal{L}$(rust) (2.5), $\mathcal{L}$(programming) (1.0) e $\mathcal{L}$(best) (0.2). Score:

$$s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7.$$

Confronto: $3.7 > 2.1 \implies$ il documento entra nell'heap sostituendo Doc 8. La nuova soglia minima sale istantaneamente a $\tau = 3.7$.

DocID	Liste Coinvolte	Calcolo Punteggio Effettivo	Verifica Soglia (τ)	Stato Heap / Nuova τ
Doc 10	`"language"`	$s = 0.5$	$0.5 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 11	`"best"`	$s = 0.3$	$0.3 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 12	`"best"`	$s = 0.1$	$0.1 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 13	`"language"`, `"best"`, `"prog"`	$s = 0.9 + 0.1 + 0.5 = 1.5$	$1.5 < 2.1 \implies$ Scartato	Invariato ($\tau = 2.1$)
Doc 15	`"rust"`, `"prog"`, `"best"`	$s = 2.5 + 1.0 + 0.2 = 3.7$	$3.7 > 2.1 \implies$ Inserito	Doc 15 ($\tau = 3.7$)

Tabella 8.1: Tracciamento della scansione esaustiva DAAT con Min-Heap per i primi 5 documenti candidati.

8.2.4 Limite Computazionale della Strategia Esaustiva

L'algoritmo esaustivo calcola correttamente il Top-k, ma soffre del difetto cardine delle interrogazioni disgiuntive: **accede e decomprime ogni singolo posting dell'indice**. Nella pratica reale, man mano che la scansione avanza, la soglia τ cresce rapidamente nei primissimi millisecondi di elaborazione. Di conseguenza, oltre il 95%–99% dei documenti successivamente esaminati possiede un punteggio di gran lunga inferiore a τ: la CPU esegue milioni di letture da memoria, decompressioni e somme scalari per documenti destinati inesorabilmente a essere scartati. Il **Dynamic Pruning** (potatura dinamica) nasce esattamente per superare questo collo di bottiglia: identificare ed escludere a priori blocchi estesi di documenti non competitivi senza mai toccarli né decomprimerli in memoria.

8.3 L'Algoritmo WAND (Weak AND) e il Dynamic Pruning

Per evitare la valutazione esaustiva di decine di milioni di documenti destinati a essere scartati, il dynamic pruning sfrutta stime maggioranti per scavalcare

intere porzioni di indice. Questa sezione introduce l'algoritmo fondazionale WAND, ne definisce gli Upper Bound di termine, descrive la procedura di selezione del pivot ed esamina una simulazione completa dell'algoritmo.

8.3.1 Riferimento Bibliografico e Concetti Fondamentali

L'algoritmo **WAND** (*Weak AND*) è stato introdotto nel lavoro fondamentale di Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer e Jason Y. Zien [13] (CIKM 2003). WAND costituisce la prima architettura algoritmica di potatura dinamica in grado di garantire formalmente l'Exact Top-k su query `OR`, saltando la scansione e la decompressione della maggior parte dei posting delle liste.

Il pilastro teorico di WAND è l'introduzione dell'**Upper Bound di Termine (UB)**, definito come il massimo punteggio parziale che il termine può apportare a un qualsiasi documento all'interno della sua posting list:

Definizione 8.3.1 (Term Upper Bound $UB(t)$)**.** Dato un termine di vocabolario $t \in \mathcal{V}$ associato alla posting list $\mathcal{L}(t)$, il suo Upper Bound $UB(t)$ è calcolato offline durante l'indicizzazione come:

$$UB(t) = \max_{d \in \mathcal{L}(t)} s(t, d). \tag{8.6}$$

Costo in Spazio e Requisiti di Memorizzazione. La memorizzazione degli UB presenta proprietà estremamente favorevoli:

- Per ogni termine $t \in \mathcal{V}$ è sufficiente memorizzare un unico valore float a 32 bit indicante $UB(t)$.
- L'ingombro di memoria ausiliaria è proporzionale alla dimensione del lessico $|\mathcal{V}|$, non alla dimensione complessiva dell'indice inverso. Per un vocabolario tipico di alcuni milioni di termini, l'array degli UB occupa pochissimi megabyte e risiede stabilmente nella RAM centrale del broker di query.
- Al contrario, pre-calcolare e salvare i punteggi esatti all'interno di ciascun record della posting list su disco risulterebbe disastroso: i numeri a virgola mobile hanno entropia elevata e si comprimono male, compromettendo l'efficienza dei codec per interi (d-gaps).

8.3.2 Proprietà di Maggiorazione e Condizione di Scarto Sicuro

Teorema 8.3.1 (Condizione di Scarto Sicuro di WAND)**.** *Siano dati una query Q, un documento candidato $d \in \mathcal{D}$ e la soglia corrente del Min-Heap τ. Il punteggio reale $s(Q,d)$ è limitato superiormente dalla somma degli Upper Bound dei termini di query:*

$$s(Q,d) = \sum_{t \in Q \cap d} s(t,d) \leq \sum_{t \in Q \cap d} UB(t) \leq \sum_{t \in Q} UB(t). \tag{8.7}$$

Se un insieme di termini attivi $\mathcal{T} \subseteq Q$ soddisfa la condizione:

$$\sum_{t \in \mathcal{T}} UB(t) < \tau, \tag{8.8}$$

*allora nessun documento d che riceve contributi di scoring **esclusivamente** dai termini in $\mathcal{T}$ può superare la soglia d'ingresso dell'heap:*

$$s(Q,d) \leq \sum_{t \in \mathcal{T}} UB(t) < \tau. \tag{8.9}$$

*Tali documenti possono essere **scartati in modo sicuro** (safely skipped), garantendo l'assenza totale di falsi negativi nell'insieme Top-k finale.*

8.3.3 Modello di Costo Hardware: CPU Reasoning vs Pointer Movements

La progettazione di WAND riflette la fisica delle moderne architetture di calcolo, come formalizzato nel Capitolo 2:

- **Operazioni ad Alto Costo (Pointer Movements):** Lo scorrimento fisico di un cursore su una posting list compressa ('next()' o 'nextGEQ()') impone il caricamento di blocchi da memoria DRAM o disco, il fallimento dei predittori hardware per salti non contigui (*cache misses*), e l'esecuzione di cicli di decompressione software (PForDelta, Elias-Fano, SIMD).
- **Operazioni a Basso Costo (CPU Reasoning):** I calcoli eseguiti dalla CPU su registri interni e cache L1 (somme tra float, confronti scalari con τ, riordinamento di un piccolo array contenente i 3–5 puntatori della query) richiedono pochi cicli di clock ($<$ 1–5 ns).
- **Principio Guida:** Conviene sempre impiegare decine di istruzioni di calcolo nella CPU per verificare se un documento può essere scartato matematicamente, se ciò consente di evitare anche una sola lettura o decompressione di un blocco di posting list.

8.3.4 Meccanismo Operativo di WAND

Ad ogni iterazione, l'algoritmo opera secondo 4 fasi consequenziali:

1. **Ordinamento dei Puntatori per DocID Corrente:** L'array delle posting list attive viene ordinato in ordine crescente in base all'identificativo del documento puntato:

$$d_1 \leq d_2 \leq \cdots \leq d_m. \tag{8.10}$$

2. **Identificazione del Documento Pivot ($d_{\textbf{pivot}}$):** Si scorrono le liste ordinate da sinistra a destra, accumulando i rispettivi $UB(t_i)$ fino a individuare il primo indice p (il *pivot*) per cui la somma cumulativa eguaglia o supera la soglia τ:

$$\sum_{i=1}^{p} UB(t_i) \geq \tau. \tag{8.11}$$

   Se la somma complessiva di tutti gli m termini di query risulta strettamente inferiore a τ ($\sum_{i=1}^{m} UB(t_i) < \tau$), nessun documento residuo può qualificarsi e l'algoritmo termina immediatamente.

3. **Proprietà dello Scarto Iniziale:** Per costruzione del pivot p, i documenti puntati dalle liste precedenti $1, \ldots, p-1$ accumulano una somma ottimistica strettamente inferiore a τ:

$$\sum_{i=1}^{p-1} UB(t_i) < \tau.$$

   Di conseguenza, nessun documento $d < d_p$ ha il potenziale matematico per entrare nell'heap.

4. **Azione sul Pivot Document ($d_{\textbf{pivot}} = d_p$):**

   - **Caso $d_1 == d_{\textbf{pivot}}$:** La prima lista punta esattamente al documento pivot. Ciò implica che $d_1 = d_2 = \cdots = d_p = d_{\text{pivot}}$: il documento compare in un numero sufficiente di liste la cui somma di UB supera τ. Il documento possiede un potenziale concreto: viene **valutato pienamente** (*fully evaluated*), calcolando la somma dei punteggi esatti per tutte le liste che lo contengono. Se $s(Q, d_{\text{pivot}}) > \tau$, entra nell'heap e τ viene incrementata. Infine, il puntatore della prima lista avanza al docID successivo.

- **Caso $d_1 < d_{\text{pivot}}$:** La prima lista punta a un documento d_1 strettamente inferiore al pivot. Dal Teorema 8.3.1, nessun documento inferiore a d_{pivot} può raggiungere la soglia τ. L'algoritmo invoca direttamente:

$$p_1.\text{nextGEQ}(d_{\text{pivot}}), \tag{8.12}$$

scavalcando in un singolo salto tutti i documenti compresi nell'intervallo $[d_1, d_{\text{pivot}} - 1]$ senza decomprimerli.

8.3.5 Tracciamento Dettagliato dell'Algoritmo WAND

Si consideri lo scenario operativo di riferimento con $m = 4$ termini di query:

- **Termini e rispettivi Upper Bound:**

$$\begin{aligned} UB(\texttt{"rust"}) &= 2.5 \\ UB(\texttt{"language"}) &= 1.1 \\ UB(\texttt{"programming"}) &= 0.6 \\ UB(\texttt{"best"}) &= 0.3 \end{aligned}$$

- **Soglia Iniziale del Min-Heap ($k = 1$):** $\tau = 2.1$.
- **Puntatori Correnti delle Liste:**
  - $\mathcal{L}(\texttt{"language"})$ punta a **10**
  - $\mathcal{L}(\texttt{"best"})$ punta a **11**
  - $\mathcal{L}(\texttt{"programming"})$ punta a **13**
  - $\mathcal{L}(\texttt{"rust"})$ punta a **15**

L'array ordinato per docID corrente risulta:

$$[\,\langle \texttt{"language"}, 10, 1.1\rangle,\ \langle \texttt{"best"}, 11, 0.3\rangle,\ \langle \texttt{"programming"}, 13, 0.6\rangle,\ \langle \texttt{"rust"}, 15, 2.5\rangle\,].$$

Fase 1: Determinazione del Pivot.

1. Si esamina la prima lista (`"language"`, $d_1 = 10$):

$$\text{Somma} = UB(\texttt{"language"}) = 1.1 < \tau = 2.1 \quad \text{(sotto la soglia)}.$$

2. Si aggiunge la seconda lista (`"best"`, $d_2 = 11$):

$$\text{Somma} = 1.1 + UB(\texttt{"best"}) = 1.4 < \tau = 2.1 \quad \text{(sotto la soglia)}.$$

3. Si aggiunge la terza lista (`"programming"`, $d_3 = 13$):

$$\text{Somma} = 1.4 + UB(\texttt{"programming"}) = 2.0 < \tau = 2.1.$$

Osservazione critica: anche ipotizzando che un documento compaia simultaneamente in `"language"`, `"best"` e `"programming"` e ottenga il massimo punteggio teorico in ciascuna di esse, la sua somma cumulativa (2.0) è rigorosamente inferiore alla soglia $\tau = 2.1$!

4. Si aggiunge la quarta lista (`"rust"`, $d_4 = 15$):

$$\text{Somma} = 2.0 + UB(\texttt{"rust"}) = 2.0 + 2.5 = 4.5 \geq \tau = 2.1.$$

La condizione è soddisfatta con $p = 4$. Il **documento pivot è** $d_{\textbf{pivot}} = d_4 = 15$.

Fase 2: Valutazione del Documento 15.

- Poiché la prima lista puntava a $d_1 = 10 < 15$, tutti i documenti intermedi $(10, 11, 12, 13)$ vengono scartati all'istante senza calcolo del punteggio.
- I cursori avanzano con nextGEQ(15).
- Si procede alla valutazione completa di Doc 15:

$$\begin{aligned} s(\texttt{"rust"}, 15) &= 2.5 \\ s(\texttt{"programming"}, 15) &= 1.0 \\ s(\texttt{"best"}, 15) &= 0.2 \\ s(\texttt{"language"}, 15) &= 0.0 \quad \text{(termine non presente nel documento)} \end{aligned}$$

Punteggio aggregato: $s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7$.

- Poiché $3.7 > 2.1$, Doc 15 viene inserito nel Min-Heap e la soglia di ammissione sale a $\tau = 3.7$.

Lo pseudocodice di WAND è formalizzato nell'Algoritmo 3.

Algorithm 3: Algoritmo WAND (Weak AND) Top-k Retrieval

Dati: Query $Q = \{t_1, \ldots, t_m\}$, Indice Invertito, array di Upper Bound $UB[t]$, intero k

Risultato: Min-Heap H contenente i Top-k documenti

```
1  H ← InizializzaMinHeap(capacità = k)
2  τ ← 0
3  Liste ← InizializzaCursori(Q)
4  while true do
5      Liste.SortByCurrentDocID()
6      acc ← 0;    pivot ← −1
7      for i ← 1 to |Liste| do
8          acc ← acc + UB[Liste[i].term]
9          if acc ≥ τ then
10             pivot ← i
11             break
12     if pivot == −1 then
13         break      // Nessun documento residuo può superare τ
14     d_pivot ← Liste[pivot].CurrentDocID()
15     if Liste[1].CurrentDocID() == d_pivot then
16         s_real ← 0
17         for i ← 1 to |Liste| do
18             if Liste[i].CurrentDocID() == d_pivot then
19                 s_real ← s_real + Liste[i].CurrentScore()
20                 Liste[i].Next()
21         if s_real > τ then
22             if |H| == k then
23                 H.ExtractMin()
24             H.Insert(⟨d_pivot, s_real⟩)
25             if |H| == k then
26                 τ ← H.FindMin().score
27     else
28         Liste[1].nextGEQ(d_pivot)          // Salto sicuro oltre i
               documenti non competitivi
29 return H
```

8.4 Analisi Critica e Casi Limite nel Dynamic Pruning

L'analisi del comportamento di WAND solleva quattro questioni teoriche e ingegneristiche fondamentali, che ne chiariscono i limiti operativi e le garanzie algebriche:

8.4.1 Semantica Disgiuntiva e Trattamento dei Termini Assenti

Nella simulazione di WAND, il documento pivot $d_{\text{pivot}} = 15$ viene valutato e inserito nell'heap pur non contenendo il termine `"language"`. Questa circostanza non costituisce un'anomalia logica:

- Nelle query disgiuntive (`OR`), la semantica di retrieval stabilisce che un documento è rilevante se la combinazione lineare dei pesi dei termini effettivamente presenti supera la soglia di ammissione τ.
- A differenza della semantica congiuntiva (`AND`) — in cui l'assenza di un termine impone l'esclusione immediata del documento —, nella disgiunzione il contributo del termine non presente è semplicemente nullo ($s(\texttt{"language"}, 15) = 0$). L'inclusione è legittimata dal fatto che la somma parziale dei termini rimanenti ($2.5 + 1.0 + 0.2 = 3.7$) supera comunque la soglia $\tau = 2.1$.

8.4.2 Analisi di Convenienza: WAND su Query Congiuntive (`AND`)

Sebbene la maggiorazione algebrica basata su Upper Bound rimanga formalmente corretta anche per l'operatore `AND`, l'applicazione di WAND a query congiuntive risulta nella pratica inefficiente e superflua:

1. Nelle query `AND`, l'algoritmo di salto deterministico basato sull'operatore nextGEQ sul massimo dei docID correnti (analizzato nella Sezione 7.6) è già asintoticamente sublineare e non richiede il calcolo di somme ottimistiche tra numeri float.
2. Nelle query congiuntive la cardinalità dell'intersezione è fortemente ridotta: pochissimi documenti soddisfano la condizione di presenza simultanea di tutti i termini. L'heap riceve quindi un numero esiguo di candidati, facendo crescere la soglia τ molto lentamente e vanificando il potenziale di potatura dinamica. Il guadagno computazionale

esponenziale di WAND si realizza specificamente sulle query `OR`, dove l'approccio esaustivo comporterebbe la scansione di milioni di record.

8.4.3 Trade-off Computazionale sui Salti Intermedi Anticipati

Durante la scansione, non appena la CPU rileva che il contributo cumulativo delle prime liste non supera τ, si potrebbe ipotizzare di eseguire salti intermedi anticipati (nextGEQ) anche sulle liste successive prima di raggiungere il pivot. Tuttavia, l'implementazione pratica deve bilanciare un compromesso architetturale:

- Ogni invocazione di avanzamento altera la posizione del cursore e impone il riordinamento dell'array dei puntatori per preservare l'invariante $d_1 \leq d_2 \leq \cdots \leq d_m$.
- Se la CPU esegue salti intermedi troppo aggressivi, il costo dei cicli spesi nei confronti logici e nel continuo riordinamento in cache/registri può eccedere il tempo risparmiato sull'indice compresso.

8.4.4 Dimostrazione Algebrica dello Scarto a Priori (Doc 12)

Si consideri lo stato dei puntatori all'inizio della scansione:

- Il cursore di $\mathcal{L}(\texttt{"programming"})$ è attestato su Doc 13 $\implies$ Doc 12 non può comparire in $\mathcal{L}(\texttt{"programming"})$.
- Il cursore di $\mathcal{L}(\texttt{"rust"})$ è attestato su Doc 15 $\implies$ Doc 12 non può comparire in $\mathcal{L}(\texttt{"rust"})$.

Doc 12 potrebbe comparire unicamente nelle liste con cursori ≤ 12, ossia $\mathcal{L}(\texttt{"language"})$ e $\mathcal{L}(\texttt{"best"})$. Il massimo punteggio teorico conseguibile da Doc 12 è maggiorato dalla somma dei rispettivi Upper Bound:

$$s(Q, 12) \leq UB(\texttt{"language"}) + UB(\texttt{"best"}) = 1.1 + 0.3 = 1.4. \tag{8.13}$$

Poiché $1.4 < \tau = 2.1$, è una rigorosa impossibilità algebrica che Doc 12 entri nel Top-k. Qualsiasi operazione di lettura, decompressione o calcolo per Doc 12 costituirebbe un'inutile dispersione di cicli macchina: lo scarto a priori è matematicamente sicuro.

Nota 8.4.1 (Metodologia di Studio e Tracciamento Manuale)**.** Per padroneggiare a fondo la meccanica di WAND e MaxScore, è fortemente consigliato esercitarsi nel tracciamento manuale passo-passo su piccoli indici invertiti giocattolo a 3 o 4 termini, simulando con rigore: l'ordinamento dinamico dei puntatori, la selezione del pivot tramite accumulo progressivo degli UB, l'esecuzione dei salti con nextGEQ, e l'aggiornamento monotono della soglia τ nell'heap.

8.5 L'Algoritmo MaxScore: Partizionamento in Liste Essenziali e Non-Essenziali

Sebbene WAND riduca drasticamente il numero di documenti valutati, l'ordinamento dinamico dei puntatori a ogni iterazione rappresenta un significativo collo di bottiglia CPU su query con molti termini. Questa sezione presenta l'algoritmo MaxScore, illustrandone la ripartizione dinamica tra liste essenziali e non-essenziali e sviluppandone il tracciamento dettagliato.

8.5.1 Riferimento Bibliografico e Principio di Funzionamento

L'algoritmo **MaxScore**, introdotto da Howard Turtle e James Flood [14] nell'articolo *Query evaluation: Strategies and optimizations* (1995), calcola il Top-k esatto su query disgiuntive; tuttavia, adotta una diversa filosofia architetturale che **elimina il ricalcolo dinamico dell'ordinamento dei puntatori** a ogni passo.

L'idea fondamentale di MaxScore consiste nel suddividere dinamicamente le posting list della query in due insiemi disgiunti in funzione della soglia τ:

1. **Liste Non-Essenziali (*Non-Essential Lists*):** L'insieme delle posting list con gli UB più bassi la cui somma cumulativa è strettamente inferiore alla soglia τ:

$$\sum_{t \in \text{Non-Essential}} UB(t) < \tau. \tag{8.14}$$

2. **Liste Essenziali (*Essential Lists*):** Tutte le rimanenti posting list della query (quelle con gli UB più elevati).

Teorema 8.5.1 (Invariante Fondamentale di MaxScore)**.** *Un documento presente **esclusivamente** all'interno delle liste non-essenziali non potrà mai entrare nel Top-k. Infatti, anche nell'ipotesi massimale in cui tale documento*

compaia in tutte le liste non-essenziali e ottenga in ciascuna il massimo punteggio parziale, il suo score totale soddisfa:

$$s(Q,d) \leq \sum_{t \in \textit{Non-Essential}} UB(t) < \tau. \tag{8.15}$$

Conseguenza Operativa Immediata: *Qualsiasi documento appartenente al Top-k* ***deve necessariamente comparire in almeno una lista essenziale****.*

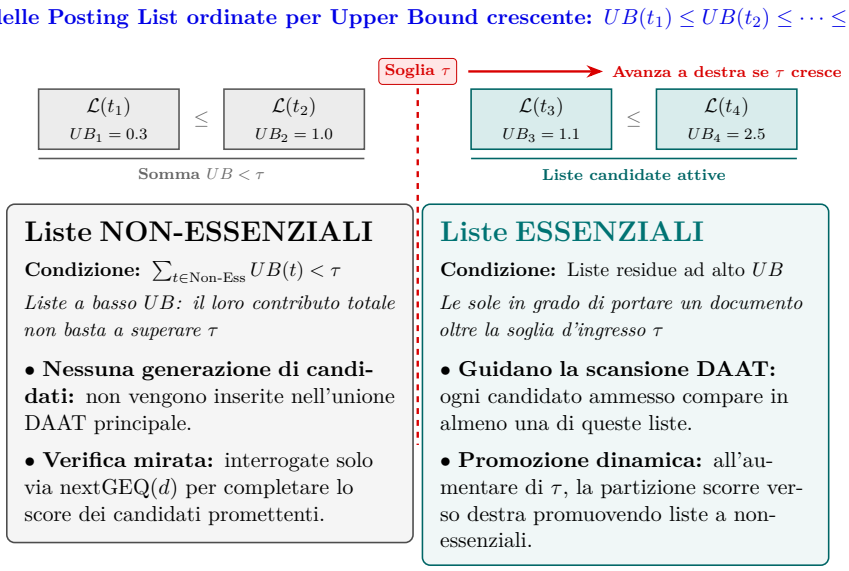


Figura 8.1: Partizionamento dinamico delle posting list nell'algoritmo MaxScore.

8.5.2 Differenze Strutturali rispetto a WAND

1. **Ordinamento Statico Iniziale:** In MaxScore le posting list vengono ordinate per UB crescente **una sola volta** all'avvio dell'interrogazione:

$$UB(t_1) \leq UB(t_2) \leq \cdots \leq UB(t_m). \tag{8.16}$$

   Durante l'esecuzione dell'algoritmo non è richiesto alcun riordinamento dinamico dei puntatori.

2. **Scansione DAAT Selettiva:** La scansione DAAT OR viene eseguita **esclusivamente sull'insieme delle liste essenziali**. Le liste non-essenziali sono passive: non partecipano all'avanzamento dei puntatori primari.

3. **Promozione Dinamica delle Liste:** Man mano che l'heap si popola e la soglia τ cresce, la sommatoria degli UB sul prefisso delle liste può inglobare nuovi termini senza eccedere τ. La frontiera di demarcazione si sposta verso destra: liste originariamente essenziali vengono **promosse a non-essenziali**, riducendo progressivamente il numero di liste attive da scansionare.

8.5.3 Valutazione Selettiva del Documento Candidato

L'algoritmo esamina un candidato d generato dall'unione delle sole liste essenziali ($d = \min_{t \in \text{Ess}} p_t.\text{CurrentDocID}()$):

1. Calcola lo score esatto fornito dalle liste essenziali: $s_{\text{ess}}(Q, d)$.
2. Calcola la **stima ottimistica massima** che il documento d potrebbe raggiungere sommando il contributo potenziale massimo di tutte le liste non-essenziali:

$$\text{Stima}(d) = s_{\text{ess}}(Q, d) + \sum_{t \in \text{Non-Essential}} UB(t). \tag{8.17}$$

3. **Controllo della Stima:**
   - Se $\text{Stima}(d) \leq \tau$: il documento non ha alcuna possibilità di entrare nel Top-k. Viene **scartato immediatamente senza toccare né decomprimere alcuna lista non-essenziale**.
   - Se $\text{Stima}(d) > \tau$: il candidato merita la valutazione. Si interrogano le liste non-essenziali invocando $p_t.\text{nextGEQ}(d)$. Se il documento è presente, si aggiunge il punteggio parziale effettivo. Qualora il punteggio finale superi τ, il documento entra nell'heap e τ viene incrementata.

8.5.4 Tracciamento Dettagliato dell'Algoritmo MaxScore

Si consideri il medesimo scenario di simulazione con i quattro termini ordinati per Upper Bound:

- **Ordinamento Statico per UB Crescente:**
  1. `"best"`: $UB = 0.3$
  2. `"programming"`: $UB = 1.0$

3. `"language"`: $UB = 1.1$
4. `"rust"`: $UB = 2.5$

- **Soglia Iniziale del Min-Heap:** $\tau = 2.1$.

Partizionamento Iniziale:

- Lista `"best"`: $UB = 0.3 < 2.1$.
- Aggiungiamo `"programming"`: $0.3 + 1.0 = 1.3 < 2.1$.
- Se aggiungessimo `"language"` (1.1), la somma cumulativa diverrebbe $1.3 + 1.1 = 2.4 \geq 2.1$.
- Pertanto la partizione iniziale risulta:

$$\text{Non-Essential} = \{\texttt{"best"}, \texttt{"programming"}\} \quad (\textstyle\sum UB = 1.3),$$
$$\text{Essential} = \{\texttt{"language"}, \texttt{"rust"}\}.$$

Scansione Guidata dalle Liste Essenziali:

1. **Candidato Doc 10 (proposto da $\mathcal{L}$(`"language"`), in quanto $10 <$ 15 di `"rust"`):**
   - Punteggio parziale reale in `"language"`: $s(\texttt{"language"}, 10) = 0.5$.
   - Calcolo della stima ottimistica:
   $$\text{Stima}(10) = 0.5 + \sum_{t \in \text{Non-Ess}} UB(t) = 0.5 + 1.3 = 1.8.$$
   - Confronto: $1.8 < \tau = 2.1 \implies$ **Doc 10 viene scartato all'istante senza accedere a `"best"` o `"programming"`!**
2. **Candidato Doc 13 (proposto da $\mathcal{L}$(`"language"`), con $13 < 15$):**
   - Punteggio parziale reale in `"language"`: $s(\texttt{"language"}, 13) = 0.9$.
   - Calcolo della stima ottimistica:
   $$\text{Stima}(13) = 0.9 + 1.3 = 2.2.$$
   - Confronto: $2.2 > \tau = 2.1 \implies$ Doc 13 è promettente. Si interrogano le liste non-essenziali con nextGEQ(13):
     - In $\mathcal{L}$(`"programming"`), Doc 13 è presente con score parziale 0.5.

  - In $\mathcal{L}(\texttt{"best"})$, Doc 13 è presente con score parziale 0.1.
- Punteggio reale complessivo: $s(Q, 13) = 0.9 + 0.5 + 0.1 = 1.5$.
- Confronto finale: $1.5 < \tau = 2.1 \implies$ Doc 13 non entra nell'heap e viene scartato.
- *Nota sul Doc 12:* Si osservi che Doc 12 è stato scavalcato a monte senza essere mai preso in considerazione, poiché non compariva in alcuna lista essenziale.

3. **Candidato Doc 15 (proposto da $\mathcal{L}(\texttt{"rust"})$):**
   - Punteggio parziale reale in `"rust"`: $s(\texttt{"rust"}, 15) = 2.5$.
   - Calcolo della stima ottimistica:

$$\text{Stima}(15) = 2.5 + 1.3 = 3.8.$$

   - Confronto: $3.8 > \tau = 2.1 \implies$ Doc 15 merita la valutazione. Si interrogano le liste non-essenziali:
     - In $\mathcal{L}(\texttt{"programming"})$, presente con score 1.0.
     - In $\mathcal{L}(\texttt{"best"})$, presente con score 0.2.
   - Punteggio reale complessivo: $s(Q, 15) = 2.5 + 1.0 + 0.2 = 3.7$.
   - Confronto finale: $3.7 > 2.1 \implies$ **Doc 15 entra nell'heap** e la soglia minima sale a $\tau = 3.7$.

Aggiornamento Partizione dopo l'Aumento della Soglia ($\tau = 3.7$): Con la nuova soglia $\tau = 3.7$, la somma di `"best"` (0.3) + `"programming"` (1.0) + `"language"` (1.1) è pari a:

$$0.3 + 1.0 + 1.1 = 2.4 < 3.7.$$

Anche la lista `"language"` viene promossa a non-essenziale:

- Non-Essential $= \{\texttt{"best"}, \texttt{"programming"}, \texttt{"language"}\}$ $(\sum UB = 2.4 < 3.7)$;
- Essential $= \{\texttt{"rust"}\}$.

Per l'intero prosieguo dell'elaborazione, l'unica lista attiva che genera candidati rimane $\mathcal{L}(\texttt{"rust"})$, riducendo drasticamente il costo computazionale.

Lo pseudocodice completo di MaxScore è formalizzato nell'Algoritmo 4.

Algorithm 4: Algoritmo MaxScore Top-k Retrieval

Dati: Query $Q = \{t_1, \ldots, t_m\}$, Indice Invertito, array $UB[t]$, intero k
Risultato: Min-Heap H contenente i Top-k documenti

```
1  H ← InizializzaMinHeap(capacità = k)
2  τ ← 0
3  Liste ← OrdinaPerUBCrescente(Q)        // Ordinamento statico una sola volta
4  nonEssEnd ← CalcolaFrontieraNonEssenziali(Liste, τ)
5  while ∃i > nonEssEnd tale che Liste[i].ÈAttivo() do
6      d ← min{Liste[i].CurrentDocID() | i > nonEssEnd ∧ Liste[i].ÈAttivo()}
7      s_partial ← 0
8      for i ← nonEssEnd + 1 to |Liste| do
9          if Liste[i].ÈAttivo() ∧ Liste[i].CurrentDocID() == d then
10             s_partial ← s_partial + Liste[i].CurrentScore()
11             Liste[i].Next()
12     stima ← s_partial + Σ_{j=1}^{nonEssEnd} UB[Liste[j].term]
13     if stima > τ then
14         for j ← 1 to nonEssEnd do
15             Liste[j].nextGEQ(d)
16             if Liste[j].ÈAttivo() ∧ Liste[j].CurrentDocID() == d then
17                 s_partial ← s_partial + Liste[j].CurrentScore()
18         if s_partial > τ then
19             if |H| == k then
20                 H.ExtractMin()
21             H.Insert(⟨d, s_partial⟩)
22             if |H| == k then
23                 τ ← H.FindMin().score
24                 nonEssEnd ← CalcolaFrontieraNonEssenziali(Liste, τ)
25 return H
```

8.6 Valutazione Sperimentale e Varianti a Blocchi: BMW e VBMW

L'adozione di un unico Upper Bound globale per un'intera posting list soffre della presenza di valori anomali, che limitano la frequenza dei salti. Questa

sezione analizza le evidenze empiriche del benchmark standard Gov2, introduce la segmentazione a blocchi di dimensione fissa (Block-Max WAND, BMW) e formalizza la variante a dimensione variabile (Variable Block-Max WAND, VBMW) per minimizzare l'errore di sovrastima.

8.6.1 Valutazione Sperimentale Comparativa (Collezione Gov2 - TREC 05)

L'efficacia pratica degli algoritmi di Dynamic Pruning è documentata dai benchmark accademici su larga scala. La Tabella 8.2 riporta i tempi medi di risposta per query (in millisecondi) misurati sulla collezione standard **Gov2** (25 milioni di pagine web, benchmark TREC 05), al variare della lunghezza della query (da 2 a oltre 6 termini).

Algoritmo	2 termini	3 termini	4 termini	5 termini	6+ termini	Media (*Avg*)	Spazio Extra
RankedOR (Esaustivo)	23.6 ms	76.5 ms	147.9 ms	235.4 ms	418.7 ms	106.7 ms	0.00%
WAND	5.1 ms	5.9 ms	7.0 ms	8.8 ms	17.8 ms	7.0 ms	0.22%
MaxScore	4.7 ms	6.0 ms	7.1 ms	9.2 ms	14.2 ms	6.6 ms	0.22%
BMW$_{40}$	1.2 ms	2.9 ms	4.3 ms	6.7 ms	14.8 ms	3.6 ms	1.55%
VBMW$_{40}$	**0.8** ms	**1.7** ms	**2.4** ms	**3.8** ms	**8.1** ms	**2.1** ms	1.55%

Tabella 8.2: Tempi medi di query processing (Top-10) su collezione Gov2 (TREC 05) al variare del numero di termini. Fonte: Ding & Suel [15], Mallia et al. [16].

Considerazioni Critiche sui Risultati:

1. **Crescita Non Lineare di RankedOR:** Nella scansione esaustiva, il tempo di risposta scala drammaticamente all'aumentare dei termini (418.7 ms per query con 6+ termini), poiché l'unione accumula decine di milioni di identificatori da verificare.

2. **Confronto Diretto tra WAND e MaxScore:** Entrambi gli algoritmi riducono la latenza media da oltre 100 ms a circa 6.6–7.0 ms (un'accelerazione di oltre 15×). WAND risulta lievemente più rapido su query corte (2 termini), mentre MaxScore prevale sulle query lunghe (6+ termini): su insiemi ampi di liste, l'assenza di riordinamento dinamico dei puntatori in MaxScore compensa ampiamente qualsiasi vantaggio di salto di WAND.

3. **Analisi del Costo Spaziale (**0.22% **vs** 1.55%**):** Il dato 0.22% indica l'ingombro percentuale dei metadati degli Upper Bound rispetto alla dimensione complessiva dell'indice compresso su disco. Trattandosi di

un singolo valore float per ciascun termine del dizionario, il sovraccarico di memoria per WAND e MaxScore è del tutto trascurabile. Nelle varianti a blocchi (1.55%), l'indice memorizza un float aggiuntivo per ogni blocco compresso; tale modesto incremento di spazio è ripagato da un ulteriore dimezzamento della latenza media (2.1 ms).

8.6.2 Il Limite dell'Upper Bound Globale e l'Approccio Block-Max WAND (BMW)

Il limite intrinseco di WAND e MaxScore consiste nell'assegnare **un unico Upper Bound globale a un'intera posting list**. Se una lista contiene 10 milioni di posting e un singolo documento isolato presenta un picco eccezionale di frequenza (ad esempio un outlier con score parziale 8.0), l'Upper Bound globale dell'intera lista sarà forzato a $UB = 8.0$. Di conseguenza, anche se il restante 99.99% dei documenti possiede punteggi inferiori a 1.0, la stima ottimistica della query resterà artificialmente maggiorata lungo tutti i 10 milioni di elementi, impedendo alla CPU di compiere salti significativi.

Per risolvere questo problema, Ding e Suel [15] hanno introdotto **Block-Max WAND (BMW)**:

- La posting list viene partizionata in **blocchi logici contigui di dimensione fissa** (tipicamente $B = 40, 64$ o 128 documenti).
- Per ciascun blocco b, l'indice memorizza il massimo identificatore contenuto nel blocco (lastDocID_b) e un **Upper Bound locale di blocco**:
$$UB_{\text{blocco}}(b) = \max_{d \in b} s(t, d). \tag{8.18}$$
- Durante il calcolo, la CPU stima il potenziale del documento corrente utilizzando l'UB del blocco locale a cui appartiene, anziché il massimo globale dell'intera lista. Se l'UB locale sommato agli altri contributi non supera τ, l'**intero blocco di 40 o 64 documenti viene saltato e scartato in un unico passaggio**, senza accedere ai posting compressi.

8.6.3 Variable Block-Max WAND (VBMW) e Minimizzazione dell'Area di Sovrastima

Sebbene BMW migliori significativamente le prestazioni, l'adozione di blocchi a dimensione fissa introduce un'inefficienza strutturale dovuta alla presenza degli outlier: se un blocco fisso di 40 elementi racchiude 39 documenti con

punteggio 1.0 e un solo outlier con punteggio 8.0, l'intero blocco riceve un UB locale pari a 8.0, creando un ampio **errore di sovrastima**.

Mallia et al. [16] hanno proposto la variante **Variable Block-Max WAND (VBMW)**, in cui la partizione della posting list non avviene a intervalli fissi, ma **adattando dinamicamente la dimensione dei blocchi** alla distribuzione dei punteggi. La Figura 8.2 illustra il confronto geometrico tra le due filosofie.

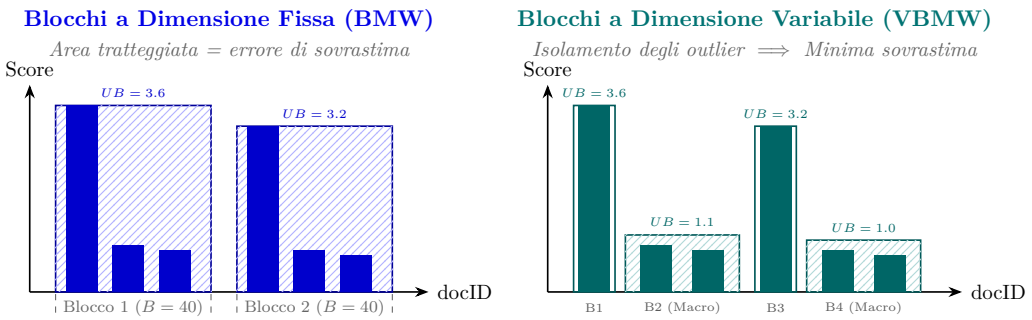


Figura 8.2: Confronto geometrico tra partizionamento a blocchi di dimensione fissa (BMW) e a dimensione variabile (VBMW). L'area tratteggiata rappresenta l'errore di sovrastima: VBMW la riduce al minimo comprimendo gli intervalli omogenei e isolando i picchi.

Il partizionamento a dimensione variabile viene determinato offline tramite un algoritmo di programmazione dinamica:

- Sequenze contigue di documenti con punteggi uniformemente bassi vengono raggruppate in macro-blocchi estesi ($B > 100$) aventi un UB locale molto basso ($UB \approx 1.0$). Poiché l'UB del blocco è modesto, il macro-blocco viene quasi sempre saltato in blocco durante la query.
- Gli outlier e i documenti ad altissima rilevanza vengono confinati in micro-blocchi dedicati composti da pochissimi elementi.
- **Impatto Pratico:** L'area di sovrastima viene minimizzata, consentendo a VBMW di raggiungere una latenza media di appena 2.1 **ms** su collezione Gov2, con un fattore di velocizzazione fino a 50× rispetto alla scansione esaustiva di base.

8.7 Sistemi Industriali: L'Ecosistema Apache Lucene ed Elasticsearch

I modelli teorici di potatura dinamica trovano riscontro diretto nell'architettura dei motori di ricerca distribuiti di produzione. Questa sezione analizza il ruolo di Apache Lucene ed Elasticsearch, ne descrive l'evoluzione decennale fino all'adozione del paradigma *Block-Max MaxScore*, e riflette sui complessi requisiti architetturali e di retrocompatibilità che separano la ricerca accademica dal software industriale.

8.7.1 L'Architettura di Apache Lucene

Le strutture dati e gli algoritmi formalizzati in questo capitolo non sono mere astrazioni accademiche, ma costituiscono il motore pulsante dell'infrastruttura di ricerca open-source più diffusa al mondo:

- **Apache Lucene:** Libreria software core scritta in Java (originariamente concepita da Doug Cutting), responsabile della gestione a basso livello degli indici invertiti, dei codec di compressione su segmenti immutabili e dell'elaborazione di query complesse con scoring BM25.
- **Elasticsearch e Apache Solr:** Motori di ricerca e analisi distribuiti di livello enterprise costruiti al di sopra di Lucene, che offrono scalabilità orizzontale tramite partizionamento (*sharding*), replica e API RESTful.

8.7.2 L'Evoluzione Algoritmica in Lucene e la Sintesi di Adrien Grand (2023)

Nel dicembre 2023, Adrien Grand (Principal Software Engineer presso Elastic e tra i principali architetti di Lucene) ha pubblicato un'approfondita analisi tecnica intitolata *"MAXSCORE & block-max MAXSCORE: More skipping with block-max MAXSCORE"* [17]. L'articolo ripercorre la traiettoria ingegneristica compiuta da Lucene in oltre un decennio, riassumibile in **quattro fasi evolutive**:

1. **Fase 1 (Valutazione Esaustiva):** Scansione DAAT standard di tutti i posting con Min-Heap; tempo di risposta dominato dalla decompressione integrale delle liste.
2. **Fase 2 (Adozione di WAND):** Integrazione del Dynamic Pruning basato su WAND (classe `TopScoreDocCollector`) con skipping basato su Upper Bound globali, introdotta in Lucene 8.0.

3. **Fase 3 (Transizione a MaxScore):** Sostituzione di WAND con MaxScore per eliminare il costo del riordinamento continuo dei puntatori e massimizzare il throughput su query contenenti molti termini disgiuntivi.

4. **Fase 4 (Integrazione di Block-Max MaxScore):** Combinazione dello schema di partizionamento essenziale/non-essenziale di MaxScore con i metadati di blocco locali (ispirati a Block-Max WAND). I blocchi dell'indice su disco integrano direttamente gli UB locali, consentendo a Lucene di compiere salti a due livelli (*two-level skipping*) su indici industriali distribuiti.

8.7.3 Il Divario tra Teoria Accademica e Produzione Industriale

Un aspetto di grande valore metodologico riguarda il **divario temporale di circa 13 anni** intercorso tra la pubblicazione scientifica delle varianti a blocchi (Ding & Suel, 2011 [15]) e la loro piena adozione stabile nei formati di indice di Apache Lucene:

- **Riprogettazione dei Codec di Indice:** Nei sistemi di produzione che gestiscono petabyte di dati, l'aggiunta di metadati di blocco non può essere realizzata banalmente. Richiede la completa riscrittura dei codec di serializzazione su disco (*Lucene Postings Format*), garantendo al contempo:
  - Retrocompatibilità binaria assoluta con gli indici generati dalle versioni software precedenti;
  - Supporto efficiente per operazioni concorrenti di fusione dei segmenti (*segment merging*);
  - Conservazione del budget I/O e assenza di frammentazione della memoria secondaria.
- **Robustezza su Workload Eterogenei:** Nei benchmark accademici, gli algoritmi vengono valutati su collezioni statiche e query disgiuntive pure. In un motore di produzione reale come Elasticsearch, la stessa pipeline di esecuzione deve gestire simultaneamente:
  - Filtri booleani strutturati arbitrariamente nidificati (`must`, `should`, `filter`);
  - Match di prossimità sintattica e query a frase (`match_phrase`);

– Calcolo parallelo di aggregazioni analitiche (sfaccettature, conteggi, metriche);
– Aggiornamenti e inserimenti di nuovi documenti in tempo reale (*near-real-time indexing*).

Un'ottimizzazione algoritmica può essere integrata nel core di Lucene solo se **non introduce alcuna regressione prestazionale** su nessuno degli altri scenari operativi.

8.8 Quadro Sinottico Comparativo degli Algoritmi Top-k

A conclusione della trattazione sul query processing top-k, la Tabella 8.3 sintetizza le proprietà algebriche, computazionali e architetturali di tutti gli approcci trattati nel capitolo, offrendo un confronto sinottico tra le strategie esaustive e i metodi di dynamic pruning.

Algoritmo	**Strategia di Scansione**	**Ordinamento Liste**	**Granularità di Pruning**	**Spazio Ausiliario**	**Latenza Media**
DAAT Esaustivo	Unione sincrona di tutte le liste	Heap dei puntatori o scansione lineare	Nessuna (visita tutti i posting)	$\mathcal{O}(k)$ (solo MinHeap)	106.7 ms (Baseline)
WAND	Scansione guidata dal Pivot Document	Dinamico a ogni passo ($d_1 \leq \cdots \leq d_m$)	Documento singolo (UB globale di termine)	$\mathcal{O}(\lvert\mathcal{V}\rvert)$ float in RAM	$\approx$ 7.0 ms ($\approx 15\times$)
MaxScore	Partizione in liste Essenziali e Non-Essenziali	Statico una sola volta per UB crescente	Documento singolo (scansione solo su essenziali)	$\mathcal{O}(\lvert\mathcal{V}\rvert)$ float in RAM	$\approx$ 6.6 ms ($\approx 16\times$)
BMW	WAND su blocchi a dimensione fissa	Dinamico sui blocchi correnti	Blocco fisso ($B = 40, 64$ posting)	+1.55% su disco (UB per blocco)	$\approx$ 3.6 ms ($\approx 30\times$)
VBMW	WAND su blocchi a dimensione variabile	Dinamico adattivo	Macro/Micro blocchi ottimizzati da DP	+1.55% su disco (UB per blocco)	$\approx$ **2.1** ms ($\approx$ **50**$\times$)

Tabella 8.3: Quadro sinottico comparativo degli algoritmi di Top-k Retrieval su query disgiuntive.

Capitolo 9

Learning to Rank I: Fondamenti, Tassonomia e Framework Generale

9.1 Okapi BM25: Storia, Intuizioni e Term Frequency Smoothing

Prima di analizzare i complessi modelli supervisionati basati sul Machine Learning, è indispensabile esaminare in dettaglio la funzione di ordinamento lessicale che costituisce il cardine di tutti i motori di ricerca moderni: **Okapi BM25** [18, 1]. Sviluppato all'interno del Probabilistic Relevance Framework (PRF), BM25 supera i limiti strutturali del Vector Space Model introducendo la saturazione asintotica della frequenza dei termini e la normalizzazione imperniata della lunghezza dei documenti.

9.1.1 Genesi Storica e il Modello Probabilistico di Reperimento

Sviluppato negli anni '80 e '90 da Stephen Robertson e Karen Spärck Jones presso la City University di Londra all'interno del sistema di reperimento sperimentale *Okapi*, l'acronimo **BM** sta per *Best Matching*, mentre il suffisso **25** identifica la venticinquesima iterazione e messa a punto della formula di ponderazione formulata durante le conferenze TREC.

BM25 poggia sui fondamenti del **Probabilistic Relevance Framework** (PRF) [18]. In tale paradigma, il ranking ideale dei documenti viene formulato in termini di stima della probabilità a posteriori di rilevanza $R \in \{0, 1\}$ rispetto

alla query q:

$$\text{Odds}(R = 1 \mid d, q) = \frac{P(R = 1 \mid d, q)}{P(R = 0 \mid d, q)}. \tag{9.1}$$

Applicando il teorema di Bayes e assumendo l'indipendenza statistica condizionale della presenza dei termini dato lo stato di rilevanza, il principio del ranking probabilistico (*Probability Ranking Principle*, PRP) stabilisce che ordinare i documenti per log-odds decrescente minimizza il rischio bayesiano atteso di errore. Mentre il modello binario classico di Robertson-Spärck Jones (Binary Independence Model, BIM) operava unicamente su indicatori binari di presenza/assenza ($x_t \in \{0, 1\}$), Okapi BM25 estende il framework incorporando la frequenza dei termini e la lunghezza dei documenti mediante un'efficace modellazione non lineare a due stati di Poisson.

9.1.2 Term Frequency Smoothing: Limiti di Raw TF e Saturazione Asintotica

Un difetto insito nei primi modelli di Information Retrieval consisteva nell'assumere una proporzionalità diretta tra l'evidenza semantica e il conteggio assoluto dei termini (*Raw TF*):

- Se un documento cita il termine `"microprocessor"` 20 volte anziché 10 volte, la sua rilevanza tematica non risulta affatto raddoppiata;
- L'introduzione della ponderazione logaritmica (esaminata nel Capitolo 4):

  $$w_{\text{log-tf}} = 1 + \log_{10}(f_{t,d}) \quad (\text{per } f_{t,d} > 0), \tag{9.2}$$

  smorza la crescita, ma la funzione diverge monotonicamente all'infinito per $f_{t,d} \to \infty$. Non pone alcun tetto superiore al contributo di un singolo vocabolo.

Okapi BM25 introduce una funzione razionale di **saturazione asintotica** (*Term Frequency Smoothing*) governata dal parametro $k_1 > 0$:

$$\tau(f_{t,d}) = \frac{(k_1 + 1) \cdot f_{t,d}}{k_1 + f_{t,d}}. \tag{9.3}$$

La derivata prima di $\tau(f_{t,d})$ rispetto a $f_{t,d}$ è strettamente positiva ma decresce rapidamente a zero:

$$\frac{d\tau}{df_{t,d}} = \frac{k_1(k_1 + 1)}{(k_1 + f_{t,d})^2} > 0, \qquad \lim_{f_{t,d} \to \infty} \tau(f_{t,d}) = k_1 + 1. \tag{9.4}$$

Il punteggio parziale converge al limite superiore asintotico finito $k_1 + 1$. Quando $f_{t,d} = 1$, il termine apporta $\tau(1) = 1$; all'aumentare delle occorrenze, il rendimento marginale di ogni occorrenza successiva collassa progressivamente verso zero, impedendo a documenti spammati o prolissi di dominare la graduatoria.

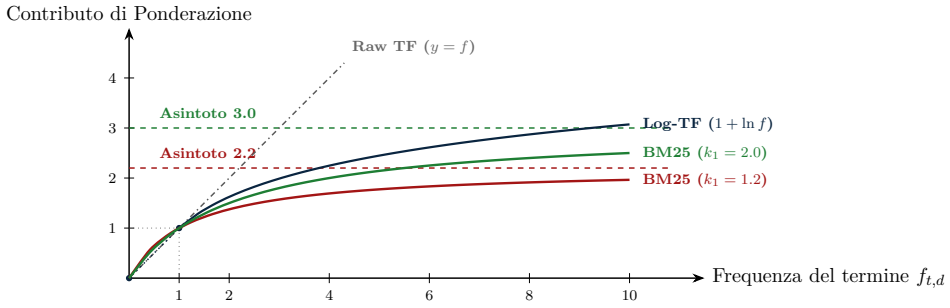


Figura 9.1: Confronto tra le curve di risposta della frequenza dei termini: il Raw TF cresce illimitatamente in modo lineare; il Log-TF cresce lentamente ma senza saturare; Okapi BM25 satura asintoticamente verso il limite finito $k_1 + 1$.

9.1.3 Pivoted Document Length Normalization

La seconda innovazione cardine di BM25 risiede nel trattamento della lunghezza del documento. Un testo molto lungo può presentare un'alta frequenza di un vocabolo per due ragioni strutturali antitetiche:

1. **Verbosità (*Verbosity*):** Il testo tratta l'argomento con ridondanza stilistica o prolissità ingiustificata, impiegando molte parole per esprimere il medesimo concetto di un testo compatto.

2. **Ampiezza Tematica (*Multi-topic Scope*):** Il testo è un trattato enciclopedico o una raccolta che affronta svariati argomenti eterogenei; in tal caso, il documento ha una probabilità a priori enormemente maggiore di intercettare casualmente i termini di una qualunque query.

Per riequilibrare la competizione tra testi sintetici e testi estesi, BM25 introduce un fattore di scala imperniato (*Pivoted Document Length Normalization*):

$$B(d) = (1 - b) + b \cdot \frac{|d|}{\text{avgdl}}, \tag{9.5}$$

dove $|d|$ denota la lunghezza in token del documento d, avgdl (*average document length*) indica la lunghezza media calcolata sull'intera collezione:

$$\text{avgdl} = \frac{1}{N} \sum_{d \in \mathcal{D}} |d|, \tag{9.6}$$

e $b \in [0, 1]$ è un iperparametro che controlla l'intensità della penalizzazione:

- **Punto di Pivot ($|d| = \text{avgdl}$):** Indipendentemente dal valore assegnato a b, se il documento ha una lunghezza pari alla media della collezione, si ha $B(d) = 1.0$;
- **Assenza di Normalizzazione ($b = 0$):** Il termine $B(d)$ collassa a 1.0 per qualsiasi documento, annullando ogni correzione di lunghezza;
- **Normalizzazione Rigorosa ($b = 1$):** Il termine $B(d) = |d|/\text{avgdl}$ normalizza la frequenza in modo esattamente proporzionale alla lunghezza assoluta del documento.

Il normalizzatore $B(d)$ viene inserito al denominatore della saturazione di frequenza, modulando la soglia k_1:

$$k_1 \cdot B(d) = k_1 \cdot \left((1 - b) + b \cdot \frac{|d|}{\text{avgdl}} \right). \tag{9.7}$$

Se un documento è lungo ($|d| > \text{avgdl}$), il denominatore cresce, rallentando la crescita della funzione e riducendo il punteggio parziale; se il documento è sintetico ($|d| < \text{avgdl}$), la saturazione si raggiunge con una frequenza inferiore.

9.1.4 Formulazione Completa di Okapi BM25 e Best Practices

Combinando la saturazione non lineare, la normalizzazione imperniata e la ponderazione di selettività inversa Document Frequency, la formulazione integrale di Okapi BM25 per una query q e un documento d risulta:

$$\text{Score}_{\text{BM25}}(q, d) = \sum_{t \in q \cap d} \text{IDF}_{\text{BM25}}(t) \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \cdot \left((1 - b) + b \cdot \frac{|d|}{\text{avgdl}} \right)}, \tag{9.8}$$

dove l'IDF probabilistico di Robertson-Spärck Jones è formulato come:

$$\text{IDF}_{\text{BM25}}(t) = \log \left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1 \right), \tag{9.9}$$

in cui N è la cardinalità della collezione e $n_t = df_t$ è il numero di documenti contenenti il termine t (l'aggiunta $+1$ o il floor a zero evitano valori di IDF negativi per termini presenti in oltre metà della collezione).

Best Practices e Calibrazione degli Iperparametri (k_1, b): L'esperienza consolidata nei benchmark internazionali (TREC) e nei sistemi di produzione stabilisce i seguenti valori di riferimento:

- **Parametro k_1:** Valore predefinito standard compreso tra 1.2 **e** 2.0 (spesso $k_1 = 1.2$ in Apache Lucene ed Elasticsearch). Valori bassi ($k_1 \to 1.0$) comprimono rapidamente l'effetto di frequenze multiple (ideale per documenti brevi e query specifiche); valori più alti ($k_1 \to 2.0$) lasciano spazio a una maggiore discriminazione tra occorrenze ripetute.

- **Parametro b:** Valore predefinito quasi universale fissato a $b = 0.75$. Un valore di 0.75 penalizza sensibilmente i documenti prolissi pur lasciando un margine di tolleranza rispetto alla normalizzazione lineare pura ($b = 1$).

Dalla Formula Chiusa all'Esigenza di Apprendere dai Dati

Nonostante BM25 sia una funzione di ranking robusta e formidabile per testi omogenei, essa resta una funzione euristica chiusa a soli 2 parametri liberi, cieca rispetto a pagine strutturate, link di autorità o comportamenti degli utenti. Nelle sezioni successive esaminiamo come il Machine Learning consenta di estendere tale orizzonte attraverso il paradigma di **Learning to Rank**.

9.2 Framework Generale di Ranking e Oltre il Bag of Words

I modelli di reperimento tradizionali basati sul paradigma Bag of Words (BoW), quali il Vector Space Model e Okapi BM25, hanno costituito il pilastro fondante dei motori di ricerca commerciali. Tuttavia, la crescente disponibilità di segnali eterogenei e la necessità di combinare evidenze lessicali, topologiche e comportamentali hanno reso indispensabile l'adozione di metodologie fondate sul Machine Learning.

Questa sezione formalizza la decomposizione astratta delle funzioni di ranking basata sulle rappresentazioni [19, 20], analizza l'insufficienza dei vettori sparsi BoW e illustra la ricchezza dello spazio dei segnali moderni.

9.2.1 Decomposizione Funzionale delle Funzioni di Ranking

Seguendo la formalizzazione proposta da Guo et al. [19], una funzione di ranking generale che assegna un punteggio di rilevanza $s(q, d) \in \mathbb{R}$ a una coppia query-documento (q, d) può essere decomposta in tre moduli di rappresentazione e un operatore di aggregazione finale.

Definizione 9.2.1 (Framework Generale di Rappresentazione per il Ranking)**.** Siano dati una query $q \in \mathcal{Q}$ emessa da un utente e un documento $d \in \mathcal{D}$ appartenente alla collezione indicizzata. La computazione dello score di rilevanza $s(q, d)$ è descritta dalla composizione:

$$s(q, d) = f\Big(\phi(q), \psi(d), \eta(q, d)\Big), \tag{9.10}$$

dove:

- $\phi : \mathcal{Q} \to \mathcal{X}_q$ definisce la funzione di **rappresentazione della sola query**;
- $\psi : \mathcal{D} \to \mathcal{X}_d$ definisce la funzione di **rappresentazione del solo documento**;
- $\eta : \mathcal{Q} \times \mathcal{D} \to \mathcal{X}_{qd}$ è la funzione di **interazione query-documento**;
- $f : \mathcal{X}_q \times \mathcal{X}_d \times \mathcal{X}_{qd} \to \mathbb{R}$ rappresenta la **funzione di aggregazione** che sintetizza i vettori nello scalare finale di ordinamento.

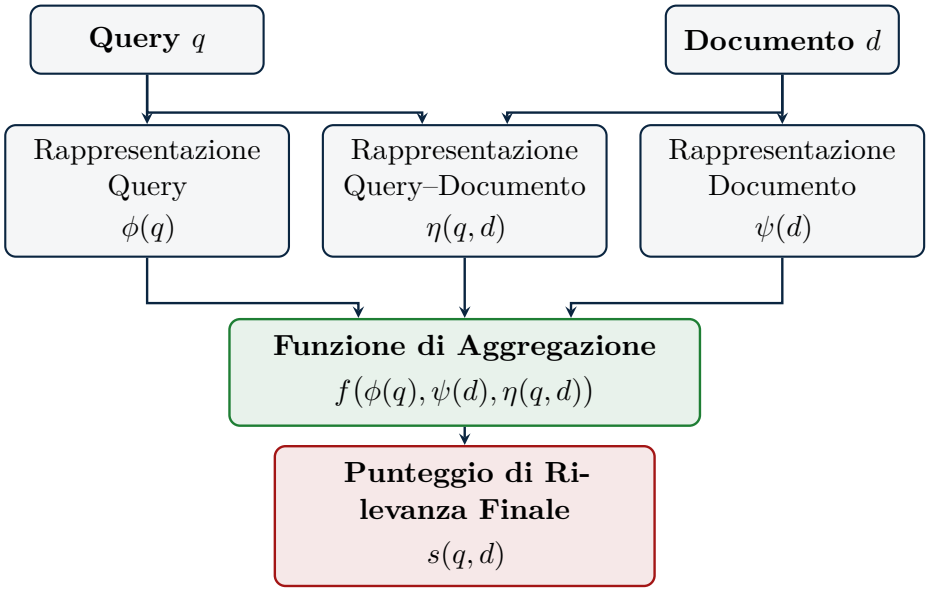


Figura 9.2: Decomposizione funzionale di una funzione di ranking generale basata sulle rappresentazioni (rielaborata da Guo et al. [19]).

Nel modello Bag of Words classico:

- La query e il documento sono proiettati come vettori sparsi di frequenze o pesi TF-IDF: $\phi(q) \in \mathbb{R}^{|\mathcal{V}|}$ e $\psi(d) \in \mathbb{R}^{|\mathcal{V}|}$;
- Il modulo di interazione esplicita è nullo: $\eta(q,d) = \emptyset$;
- La funzione di aggregazione f consiste in una metrica chiusa prefissata, quale il coseno vettoriale o la sommatoria non lineare Okapi BM25:

$$s_{\text{BM25}}(q,d) = \sum_{t \in q \cap d} \text{IDF}(t) \cdot \frac{f_{t,d} \cdot (k_1 + 1)}{f_{t,d} + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}. \tag{9.11}$$

Queste rappresentazioni sparse sono memorizzate negli indici invertiti, garantendo scansioni scalabili mediante algoritmi top-k (WAND, MaxScore). Ciononostante, il modello BoW impone forti semplificazioni: assume l'indipendenza statistica tra i termini e trascura qualsiasi caratteristica strutturale del testo o dell'ambiente applicativo.

9.2.2 Oltre il Bag of Words: La Molteplicità dei Segnali di Rilevanza

Sul Web, una pagina non è un mero multinsieme non strutturato di vocaboli. Essa possiede una ricca anatomia interna e interagisce con un ecosistema complesso:

1. **Campi e Zone Strutturate**: Un documento è suddiviso in campi eterogenei (titolo, abstract, corpo principale, testo dell'URL). L'occorrenza di un termine nel titolo trasmette un segnale di rilevanza drasticamente superiore rispetto all'occorrenza nel corpo.

2. **Dati Testuali Esterni**: I testi di ancoraggio (*anchor text*) degli hyperlink che puntano alla pagina rappresentano sintesi redatte da terzi, spesso più descrittive del contenuto stesso della risorsa.

3. **Grafi di Collegamento e Topologia**: Link in entrata (*in-links*), link in uscita (*out-links*), score globali di autorità quali PageRank e HITS.

4. **Segnali Comportamentali e di Utilizzo**: Log delle ricerche, numero aggregato di clic ricevuti dall'URL per una determinata query, tempo di permanenza (*dwell time*) sulla pagina prima di tornare ai risultati della SERP.

5. **Proprietà della Query**: Lunghezza della query in termini, popolarità nel log storico delle interrogazioni, entità riconosciute, contesto geografico e temporale dell'utente.

Di fronte a tale eterogeneità, sorge spontanea la domanda fondante: *Come possiamo apprendere automaticamente la funzione di aggregazione f a partire dai dati anziché concepirla manualmente?*

9.3 Il Paradigma del Machine Learning Applicato al Ranking

Per superare le limitazioni delle formule euristiche chiuse, il problema del ranking viene riformulato all'interno del framework statistico del Machine Learning: consentire a un sistema computazionale di migliorare progressivamente le proprie prestazioni su un compito specifico attraverso l'esposizione all'esperienza (dati empirici) senza essere esplicitamente programmato.

9.3.1 Formalizzazione dell'Apprendimento Supervisionato

Nel paradigma supervisionato, l'algoritmo riceve un insieme di istanze di addestramento (*training instances*) descritte da un vettore di attributi $\mathbf{x} \in \mathcal{X}$ e corredate da un'etichetta desiderata $y \in \mathcal{Y}$ (*ground truth* o supervisione fornita da annotatori umani):

- Se $\mathcal{Y} \subseteq \mathbb{R}$, il task è di **Regressione** (stima di una grandezza continua);
- Se $\mathcal{Y} = \{c_1, \ldots, c_m\}$ con $m \geq 2$, il task è di **Classificazione** (assegnazione a categorie discrete).

Il ciclo iterativo di addestramento si articola nei seguenti passi:

1. Un'istanza $\mathbf{x}_i$ viene sottoposta al modello predittivo con parametri θ, generando una stima $\hat{y}_i = h_\theta(\mathbf{x}_i)$;
2. Viene quantificata la discrepanza tra la predizione $\hat{y}_i$ e l'etichetta vera y_i attraverso una funzione di perdita $\mathcal{L}(\hat{y}_i, y_i)$ (*loss function*);
3. I parametri θ vengono aggiornati lungo la direzione che minimizza la perdita empirica (tipicamente mediante discesa del gradiente);
4. Il procedimento viene ripetuto su tutti gli esempi del dataset per molteplici epoche di training (*epochs*).

9.3.2 Analogia del Cervello e del Puzzle

Un'utile analogia concettuale per comprendere il ciclo retroattivo di ottimizzazione è quella della risoluzione di un rompicapo o puzzle:

- A ogni passo abbiamo a disposizione il problema (*il puzzle*) e la sua soluzione ideale certificata (*ground truth*);
- Il nostro modello cognitivo (*il cervello / l'algoritmo*) genera un tentativo di soluzione;
- Confrontando la soluzione prodotta con quella corretta, rileviamo gli errori commessi;
- Sulla scorta dell'entità e della direzione dello scarto, calibriamo i nostri parametri interni;
- Ripetendo la procedura per tentativi successivi su un vasto numero di puzzle, il sistema apprende una capacità di generalizzazione efficace.

9.3.3 Architettura a Due Fasi: Training Offline vs. Inference Online

Un vincolo imprescindibile nell'ingegneria dei motori di ricerca risiede nella rigorosa separazione temporale e computazionale tra le fasi di apprendimento e utilizzo del modello:

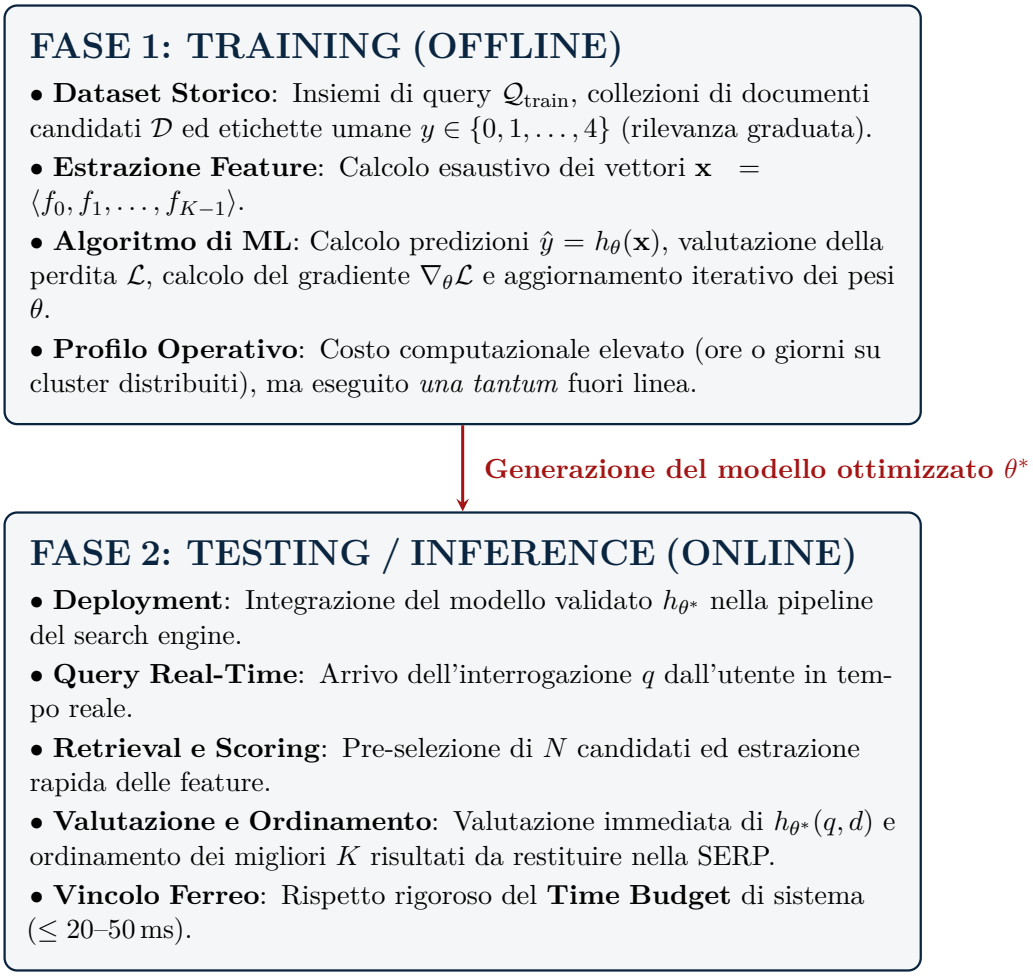


Figura 9.3: Disaccoppiamento strutturale tra la fase di addestramento offline e la fase di inferenza online.

9.4 Definizione Formale di Learning to Rank e Tassonomia delle Feature

Il framework denominato **Learning to Rank (LtR)** costituisce l'applicazione metodologica dell'apprendimento automatico al compito di costruire modelli di ordinamento per il recupero delle informazioni [21].

9.4.1 Definizione e Obiettivo di Apprendimento

Definizione 9.4.1 (Learning to Rank)**.** Sia data una collezione di query di training $\mathcal{Q}_{\text{train}} = \{q_1, q_2, \ldots, q_m\}$. Per ogni query q_i, sia disponibile un insieme di documenti candidati $D_i = \{d_{i,1}, d_{i,2}, \ldots, d_{i,n_i}\}$ e un corrispondente vettore di giudizi di rilevanza $\mathbf{y}_i = \langle y_{i,1}, y_{i,2}, \ldots, y_{i,n_i} \rangle$, dove $y_{i,j} \in \mathcal{Y}$. Ciascuna coppia $(q_i, d_{i,j})$ è rappresentata da un vettore di attributi multidimensionale:

$$\mathbf{x}_{i,j} = \Phi(q_i, d_{i,j}) = \langle f_0(q_i, d_{i,j}), f_1(q_i, d_{i,j}), \ldots, f_{K-1}(q_i, d_{i,j}) \rangle \in \mathbb{R}^K. \quad (9.12)$$

L'obiettivo di Learning to Rank consiste nell'apprendere una funzione di scoring $h : \mathbb{R}^K \to \mathbb{R}$ che assegni a ciascun documento un valore numerico tale per cui l'ordinamento indotto su D_i:

$$\pi(D_i) = \text{sort}\Big(\{h(\mathbf{x}_{i,j})\}_{j=1}^{n_i} \Big) \quad (9.13)$$

massimizzi una metrica globale di efficacia della graduatoria (come NDCG@k o MAP).

Distinzione Cruciale: Regressione vs Ranking

Anche se i modelli LtR sono frequentemente addestrati come regressori o classificatori che assegnano un valore scalare al documento, l'obiettivo ultimo del sistema non è predire con precisione infinitesima la label numerica y, bensì **indovinare la permutazione corretta dell'intera lista**, massimizzando la rilevanza dei documenti collocati nelle posizioni di vertice.

Le etichette di rilevanza umane y sono tipicamente graduate su una scala discreta a 5 livelli:

- $y = 0$: *Irrelevant* / Scadente;
- $y = 1$: *Marginally Relevant* / Marginalmente utile;
- $y = 2$: *Relevant* / Rilevante;

- $y = 3$: *Highly Relevant* / Altamente rilevante;
- $y = 4$: *Perfectly Relevant* / Risposta definitiva (ad es. home page cercata per query navigazionale).

9.4.2 Tassonomia delle Feature in un Sistema di Ricerca Web

Nei motori di ricerca commerciali, la cardinalità dello spazio delle feature è rilevante: già nel 2008 il *New York Times* riportava le dichiarazioni di Amit Singhal secondo cui Google impiegava oltre 200 feature distinte per ordinare le pagine web.

Secondo la classificazione canonica (Manning et al. [1], Liu [21]), le feature si articolano in tre categorie strutturali distinte:

1. **Feature Query-Only (Dipendenti esclusivamente dalla Query)**: Assumono il medesimo valore per tutti i documenti candidati associati alla stessa query. Non alterano l'ordinamento relativo dei documenti se applicate linearmente, ma fungono da selettori di contesto o modificatori di pendenza nei modelli ad albero:
   - Numero di termini della query ($|q|$);
   - Frequenza storica della query nei registri di log (popolarità);
   - Tipologia o intento della query (navigazionale, informativa, transazionale);
   - Lingua rilevata o localizzazione geografica dell'interrogazione.
2. **Feature Query-Independent (Dipendenti esclusivamente dal Documento / statiche)**: Quantificano la qualità intrinseca, l'autorevolezza o l'accessibilità a priori del documento d, a prescindere da quale query lo abbia evocato:
   - Valore di PageRank della pagina;
   - Lunghezza dell'URL (in caratteri o in livelli di sottodirectory);
   - Presenza di caratteri specifici nell'URL (es. tilde '~', indicativa di home page personali spesso obsolete);
   - Numero di hyperlink in entrata (*in-links*) e in uscita (*out-links*);
   - Freschezza del documento (data dell'ultima modifica o tempo trascorso dall'indicizzazione);

- Lunghezza complessiva del testo del documento ($|d|$);
- Numero di immagini o elementi multimediali presenti nella pagina;
- Clic aggregati storici ricevuti dall'URL e Dwell Time medio globale.

3. **Feature Query-Dependent (Interazione Query–Documento)**: Misurano la rilevanza tematica e l'attinenza semantica specifica tra la query q e il documento d:
   - Punteggio Okapi BM25 calcolato sui diversi campi del documento;
   - Similarità del coseno nello spazio vettoriale VSM;
   - Frequenza logaritmica dei termini della query presenti negli *anchor text* dei link puntanti a d;
   - Termini della query evidenziati in grassetto, con colori specifici o con tag tipografici primari (`<h1>`, `<title>`);
   - Finestra minima di prossimità (*term proximity*): la lunghezza della sequenza minima di testo che racchiude tutti i termini dell'interrogazione;
   - Conteggio dei clic storici registrati per la specifica coppia (q, d).

9.4.3 L'Architettura di Pipeline Two-Stage e il Vincolo del Time Budget

Come evidenziato nella Figura 9.4, un motore di ricerca moderno adotta un'architettura a imbuto (*Two-Stage Retrieval*):

- **Primo Stadio (Base Ranker)**: Interroga l'indice invertito compresso mediante algoritmi ad alta efficienza (come WAND o MaxScore descritti nel Capitolo 8). Valuta un punteggio lessicale economico (BM25) ed estrae una rosa ristretta di $N \approx 1\,000$ documenti candidati.
- **Secondo Stadio (Top Ranker / LtR Ranker)**: Riceve gli N candidati, calcola i vettori di feature multidimensionali $\mathbf{x} \in \mathbb{R}^K$ ed esegue il modello di Machine Learning appreso offline per riordinare i documenti e selezionare i migliori $k \approx 10$–20 risultati da presentare all'utente nella SERP.

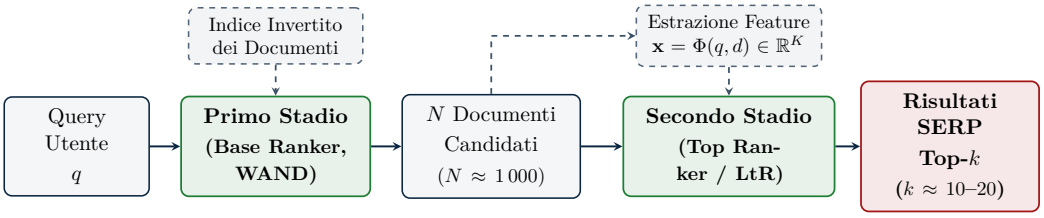


Figura 9.4: Pipeline di elaborazione a due stadi: dal filtraggio scalabile su indice invertito al re-ranking supervisionato.

Il costo computazionale dei modelli LtR influenza direttamente il throughput complessivo dell'infrastruttura di servizio. I vincoli di latenza impongono che l'estrazione delle centinaia di feature e la valutazione del modello predittivo avvengano entro una frazione infinitesima del budget temporale complessivo ($\leq$ 15–20 ms per query). Per questa ragione, la ricerca recente ha sviluppato ottimizzazioni architetturali di basso livello (implementazioni tree-traversal branchless, compilazione di foreste decisionali in codice assembly nativo) volte ad accelerare l'inferenza online.

9.5 Classificazione per l'Ad-Hoc IR: Formulazione ed Esempio Geometrico

Uno dei primi approcci all'integrazione del Machine Learning nel ranking ha tentato di ridurre la selezione dei documenti a un problema classico di **classificazione lineare binaria** (Manning et al. [1]).

9.5.1 Formulazione Lineare dello Scoring

Si consideri un dataset di addestramento composto da triple (q, d, r), dove la rilevanza $r \in \{0, 1\}$ è binaria: $r = 1$ identifica la classe dei documenti rilevanti (R), mentre $r = 0$ identifica la classe dei non rilevanti (N).

Ciascuna coppia (q, d) viene proiettata in un semplice spazio bidimensionale $\mathbf{x} = \langle \alpha, \omega \rangle$:

- $\alpha \in [0, 1]$: la similarità del coseno tra la query e il documento calcolata secondo il modello VSM;
- $\omega \geq 1$: la minima finestra di prossimità (*minimum query window size*), definita come la lunghezza del più breve segmento contiguo di testo che include tutti i termini della query (indipendentemente dall'ordine). Una finestra ω ridotta indica che i termini compaiono ravvicinati, fornendo una prova tangibile di coerenza semantica.

Si definisce una funzione di punteggio lineare parametrizzata dai coefficienti scalari a e b:

$$\text{Score}(q, d) = \text{Score}(\langle \alpha, \omega \rangle) = a \cdot \alpha + b \cdot \omega. \tag{9.14}$$

Il classificatore lineare stabilisce una superficie di decisione attraverso una soglia critica $\Theta \in \mathbb{R}$:

$$\text{Classe}(q, d) = \begin{cases} \text{Rilevante } (R), & \text{se Score}(q, d) > \Theta, \\ \text{Non Rilevante } (N), & \text{se Score}(q, d) \leq \Theta. \end{cases} \tag{9.15}$$

L'apprendimento consiste nell'individuare la tupla ottimale di parametri $\langle a^*, b^*, \Theta^* \rangle$ in grado di separare le due classi nello spazio geometrico minimizzando l'errore empirico di classificazione.

9.5.2 Esempio Geometrico e Superficie di Decisione

Nella Figura 9.5 viene visualizzato lo spazio bidimensionale (ω, α) con la disposizione empirica delle osservazioni e la retta di separazione:

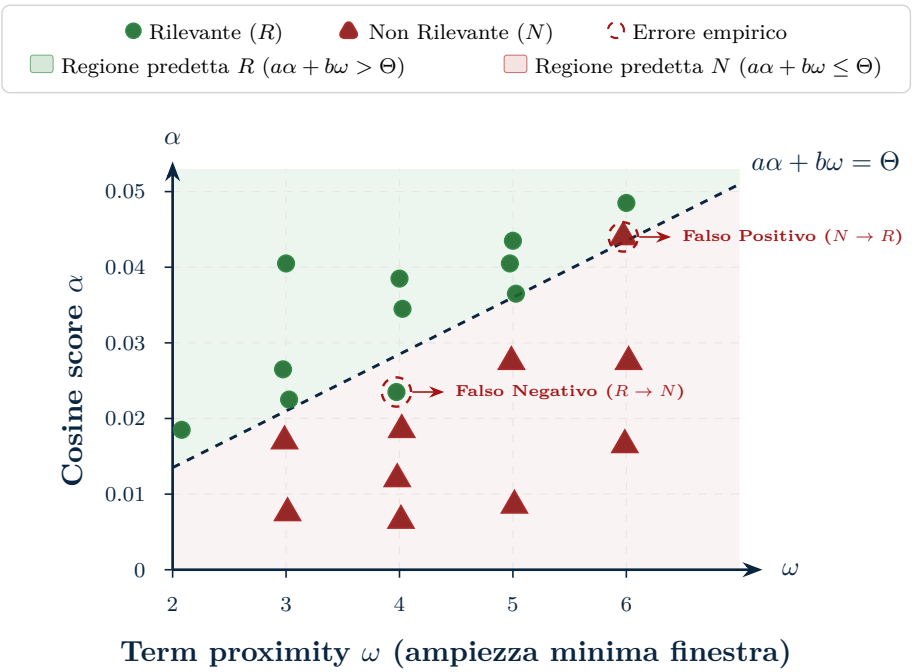


Figura 9.5: Spazio delle feature $\langle \omega, \alpha \rangle$ e classificazione lineare per Information Retrieval (Manning et al. [1], Sez. 15.4.1): cerchi verdi per documenti rilevanti (R), triangoli rossi per non rilevanti (N). La retta tratteggiata $a\alpha + b\omega = \Theta$ modella il confine di decisione lineare. Le aree ombreggiate identificano le due semiregioni decisionali previste; i cerchi tratteggiati evidenziano il Falso Negativo su $\omega = 4$ e il Falso Positivo su $\omega = 6$, a dimostrazione che il dataset empirico non è linearmente separabile al 100%.

Il grafico visualizza la distribuzione empirica dei documenti nello spazio bidimensionale definito dalla prossimità dei termini (ω) sull'asse orizzontale e dal punteggio di similarità coseno (α) sull'asse verticale. La retta tratteggiata $a\alpha + b\omega = \Theta$ modella il confine di decisione lineare: all'aumentare della distanza tra i termini (peggioramento qualitativo di ω), un documento necessita di un punteggio di coseno α progressivamente più elevato per essere classificato come rilevante (R). Le aree ombreggiate identificano le due semiregioni decisionali previste dal modello; la presenza di campioni oltre la soglia lineare evidenzia come il dataset empirico non sia perfettamente separabile in modo lineare.

Dall'analisi geometrica e concettuale emergono proprietà fondamentali per l'Information Retrieval:

1. **Significato Fisico degli Assi e Regione di Rilevanza**:

- L'asse delle ordinate misura il punteggio coseno $\alpha \in [0, 1]$: valori più alti corrispondono a una maggiore similarità globale tra la query e il documento.
- L'asse delle ascisse misura la dimensione minima della finestra di testo $\omega \geq 2$ che racchiude tutti i termini della query nel documento: trattandosi di una distanza, valori inferiori riflettono una maggiore prossimità sintattica e concettuale (quindi migliore qualità).
- La regione di massima rilevanza si concentra pertanto in **alto a sinistra** (elevato coseno α e ridotta finestra ω).

2. **Pendenza Positiva della Retta di Decisione (Trade-Off Compensativo)**:
   - La superficie di separazione lineare $a\alpha + b\omega = \Theta$ esibisce pendenza positiva nel piano cartesiano.
   - Questa geometria modella formalmente un principio di *compensazione*: all'aumentare di ω (ovvero quando i termini della query si disperdono all'interno del documento, degradandone la prossimità), il documento può comunque essere classificato come rilevante (R) solo se compensa tale carenza con un punteggio di coseno α proporzionalmente più elevato.
3. **Non-Separabilità Lineare Perfetta ed Errori Empirici**:
   - I campioni reali non risultano linearmente separabili al 100%: l'iperpiano di decisione ottimale minimizza l'errore empirico, ma non può annullarlo.
   - **Falso Negativo** (R sotto la soglia): sulla colonna $\omega = 4$, il campione rilevante a quota $\alpha \approx 0.024$ si colloca al di sotto della retta, venendo classificato erroneamente come non rilevante (N).
   - **Falso Positivo** (N sopra la soglia): sulla colonna $\omega = 6$, il campione non rilevante a quota $\alpha \approx 0.044$ si trova al di sopra della linea di decisione, venendo predetto erroneamente come rilevante (R).

Sebbene il classificatore lineare fornisca un'intuitiva demarcazione dello spazio, trattare il ranking documentale come un problema di pura classificazione binaria presenta limiti intrinseci severi:

- **Assenza di ranking interno**: tutti i documenti che cadono al di sopra della retta appartengono indistintamente alla classe R, senza fornire un

ordinamento di preferenza relativo (ad esempio tra il primo e il decimo risultato);

- **Simmetria ingannevole dei costi di errore**: una misclassificazione a quota altissima (documento cardine da prima posizione scartato) viene penalizzata dalla loss di classificazione esattamente con lo stesso peso di una misclassificazione su documenti a bassa rilevanza;
- **Incapacità di gestire la rilevanza graduata**: le collezioni moderne impiegano scale multi-livello (es. $\{0, 1, 2, 3, 4\}$ per NDCG), impossibili da catturare fedelmente con un singolo iperpiano binario.

9.6 La Tassonomia Classica di Learning to Rank: Pointwise, Pairwise e Listwise

Per colmare le discrepanze tra la formulazione classica del Machine Learning e le esigenze del reperimento di documenti, la letteratura scientifica ha codificato tre paradigmi fondamentali [21]:

Pointwise	Pairwise	Listwise
Input: Istanza singola $(\mathbf{x}(q,d),\, y)$.	**Input:** Coppia di documenti per query: $(\mathbf{x}_i, \mathbf{x}_j,\, y_{ij})$.	**Input:** Insieme ordinato dei documenti associati a q.
Target: Stima dello score o classe di rilevanza $y \in \mathcal{Y}$.	**Target:** Preservazione dell'ordine relativo ($d_i \succ d_j$).	**Target:** Ottimizzazione di metriche globali (NDCG, MAP).
Modello: Regressione standard o classificazione multiclasse.	**Modello:** Classificazione binaria su coppie (es. RankNet).	**Modello:** Metodi a loss surrogata differenziabile (es. LambdaMART).
Criticità: Indifferente all'ordine relativo e alla posizione finale.	**Criticità:** Pesa ogni coppia in modo identico, ignorando il vertice top-k.	**Criticità:** Metriche IR discrete e non derivabili sui parametri.

Figura 9.6: Confronto sistematico dei tre approcci canonici di Learning to Rank: granularità di input, target e relative criticità.

9.6.1 Approccio Pointwise

Nel paradigma **Pointwise**:

- Ogni coppia query-documento è associata a uno score indipendente $y \in \mathcal{Y}$;

- Il problema è formulato come regressione convenzionale (minimizzazione dell'errore quadratico medio $\mathcal{L}_2 = (h(\mathbf{x}) - y)^2$) oppure come classificazione multiclasse con loss di entropia incrociata;
- **Criticità Fondamentale**: L'approccio è cieco rispetto alla struttura della graduatoria. Ad esempio, penalizza allo stesso modo una discrepanza di punteggio su un documento irrilevante a fondo lista e su un documento cardine collocato in prima posizione.

9.6.2 Approccio Pairwise e il Modello RankNet

Il paradigma **Pairwise** sposta l'unità di analisi dalla stima assoluta alla preservazione dell'ordinamento relativo tra coppie di documenti associati alla medesima query:

- Dati due documenti d_i e d_j candidati per la query q, se $y_i > y_j$ si stabilisce una preferenza ordinata $d_i \succ d_j$ (documento d_i più rilevante di d_j);
- Il compito si riconduce a una classificazione binaria: il modello deve assegnare punteggi tali per cui $h(\mathbf{x}_i) > h(\mathbf{x}_j)$;
- La perdita empirica quantifica il numero di inversioni o violazioni di coppia (*pairwise ranking loss*).

Formulazione Algebrica di RankNet

L'algoritmo **RankNet** (Burges et al. [4]) modella probabilisticamente la preferenza di coppia tramite una funzione logistica sigmoide:

$$P_{ij} = P(d_i \succ d_j) = \frac{1}{1 + e^{-\sigma(s_i - s_j)}}, \tag{9.16}$$

dove $s_i = h(\mathbf{x}_i)$ e $s_j = h(\mathbf{x}_j)$ denotano i punteggi predetti dal modello (ad esempio una rete neurale o un ensemble), e $\sigma > 0$ governa la pendenza della curva sigmoide.

Definita la probabilità target della ground truth come $\bar{P}_{ij} = 1$ se $y_i > y_j$ (oppure 0.5 se $y_i = y_j$), la funzione di costo di RankNet per la coppia (d_i, d_j) corrisponde alla cross-entropy:

$$\mathcal{C}_{ij} = -\bar{P}_{ij} \log P_{ij} - (1 - \bar{P}_{ij}) \log(1 - P_{ij}). \tag{9.17}$$

Nel caso rigoroso in cui $d_i \succ d_j$ ($\bar{P}_{ij} = 1$), la funzione di costo si semplifica in forma analitica chiusa:

$$\mathcal{C}_{ij} = \log\left(1 + e^{-\sigma(s_i - s_j)}\right). \tag{9.18}$$

La formulazione (9.18) è ovunque continua e derivabile rispetto ai punteggi s_i e s_j:

$$\frac{\partial \mathcal{C}_{ij}}{\partial s_i} = -\sigma \left(\frac{1}{1 + e^{\sigma(s_i - s_j)}} \right) = -\sigma(1 - P_{ij}) = -\frac{\partial \mathcal{C}_{ij}}{\partial s_j}. \tag{9.19}$$

Grazie alla differenziabilità, RankNet consente l'addestramento diretto dei pesi interni tramite Stochastic Gradient Descent (SGD) applicando la regola della catena:

$$\frac{\partial \mathcal{C}_{ij}}{\partial \theta} = \frac{\partial \mathcal{C}_{ij}}{\partial s_i} \frac{\partial s_i}{\partial \theta} + \frac{\partial \mathcal{C}_{ij}}{\partial s_j} \frac{\partial s_j}{\partial \theta}. \tag{9.20}$$

9.6.3 Criticità Strutturale del Pairwise e Divergenza con NDCG

Nonostante l'eleganza matematica di RankNet, il paradigma Pairwise soffre di una patologia intrinseca: minimizzare il numero globale di coppie invertite non equivale a massimizzare metriche IR posizionali quali NDCG@k o MAP.

Il Dilemma della Priorità di Posizione

Nella metrica Normalized Discounted Cumulative Gain (NDCG@k), il contributo di ciascun documento è ponderato da uno smorzamento logaritmico della posizione: $\frac{1}{\log_2(\text{rank}+1)}$. Ne consegue che un'inversione tra la posizione 1 e la posizione 2 provoca una perdita di qualità drasticamente superiore rispetto alla medesima inversione tra la posizione 80 e la posizione 81. L'approccio Pairwise classico tratta invece entrambe le inversioni come una singola violazione identica, distorcendo l'ottimizzazione del modello.

Questa divergenza è documentata graficamente nella Figura 9.7, dove si osserva l'andamento della Pairwise Loss a confronto con NDCG@5 al progredire delle epoche di addestramento:

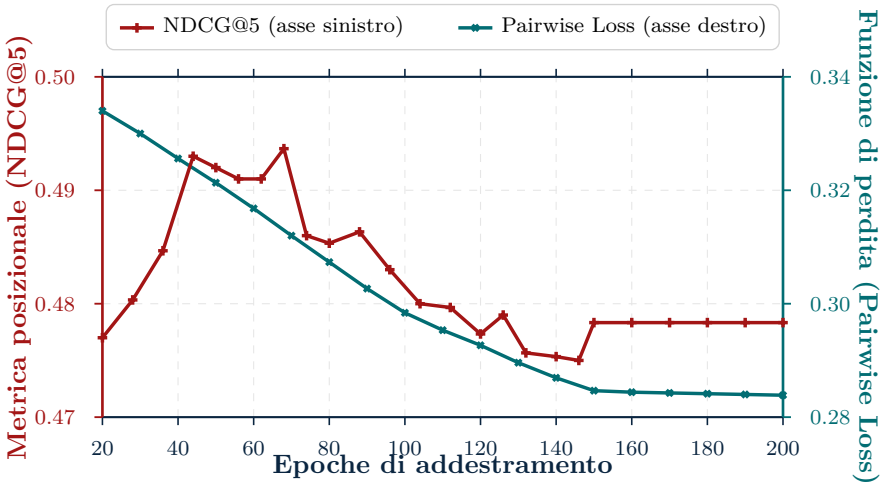


Figura 9.7: Divergenza empirica tra funzione di perdita pairwise e metrica di ranking NDCG@5 durante l'addestramento di RankNet (Taylor et al. [22]).

L'andamento evidenzia il limite intrinseco dell'approccio pairwise rispetto all'ottimizzazione posizionale: mentre la perdita continua a decrescere monotonicamente (asse destro in ottanio), la metrica NDCG@5 (asse sinistro in bordeaux) subisce un progressivo degrado dopo aver raggiunto il picco attorno all'epoca 50–70. Ciò dimostra che la minimizzazione globale delle coppie errate non garantisce l'ottimizzazione dei primi documenti restituiti all'utente, motivando storicamente l'introduzione di approcci sensibili alla posizione come LambdaRank.

La stabilizzazione piatta di NDCG@5 osservabile tra l'epoca 150 e 200 riflette la natura discreta della metrica a gradini: finché la permutazione relativa dei primi 5 documenti non varia, il valore di NDCG rimane rigorosamente costante anche a fronte di continue variazioni dei punteggi continui prodotti dal modello.

9.6.4 Approccio Listwise e l'Intrattabilità dell'Ottimizzazione Diretta

Il paradigma **Listwise** si propone di risolvere la discrepanza affrontando l'intero elenco dei candidati come un'unica entità indivisibile e ottimizzando direttamente le metriche posizionali ufficiali (NDCG, MAP).

Tuttavia, l'applicazione standard della Discesa del Gradiente si scontra con una barriera matematica:

Teorema 9.6.1 (Non-Derivabilità dell'Operatore di Ranking)**.** *Sia* $\mathbf{s} = \langle s_1, s_2, \ldots, s_n \rangle \in \mathbb{R}^n$ *il vettore dei punteggi continui prodotti dal modello di ranking* h_θ*. La permutazione o posizione di rango* $\pi(i)$ *del documento* d_i *è determinata dall'operatore di ordinamento discreto:*

$$\pi(i) = 1 + \sum_{j \neq i} \mathbb{I}(s_j > s_i), \tag{9.21}$$

dove $\mathbb{I}(\cdot)$ *indica la funzione indicatrice booleana. Poiché la funzione indicatrice è una funzione a gradino costante a tratti con derivata nulla quasi ovunque e punti di discontinuità non derivabili ogni volta che* $s_j = s_i$*, ne consegue che:*

$$\frac{\partial \pi(i)}{\partial s_k} = 0 \quad \textit{quasi ovunque}, \quad \textit{e indifferenziabile nei punti di pareggio.} \tag{9.22}$$

Di conseguenza, qualsiasi metrica $M(\pi, \mathbf{y})$ *definita sui ranghi posizionali (come NDCG@k o MAP) possiede gradiente nullo o non definito rispetto ai parametri* θ *del modello:*

$$\nabla_\theta NDCG@k = \mathbf{0}. \tag{9.23}$$

L'impossibilità di applicare il calcolo differenziale diretto alle funzioni di ranking ha aperto la strada a due strategie di risoluzione storiche:

1. L'ottimizzazione di funzioni di perdita differenziabili surrogate che fungono da limite superiore per NDCG (ad esempio **ListNet**, **ListMLE**);

2. La formulazione di gradienti virtuali corretti dall'impatto sul ranking, alla base della teoria di **LambdaRank** e dell'algoritmo allo stato dell'arte **LambdaMART** [4], discussi nel Capitolo 10.

9.7 Quadro Sinottico e Considerazioni Conclusive

La Tabella 9.1 sintetizza i tre paradigmi fondazionali di Learning to Rank analizzati nel capitolo:

Paradigma	Unità di Input	Loss Function Prototipo	Ottimizzazione	Criticità Fondamentale
Pointwise	Singola coppia (q, d)	Mean Squared Error (MSE), Cross-Entropy	Regressione diretta / SGD	Ignora la posizione del documento nella graduatoria finale.
Pairwise	Coppia di documenti (d_i, d_j) per query q	RankNet Loss ($\log(1+e^{-\sigma(s_i-s_j)})$), Hinge Loss	Discesa del gradiente continua (SGD)	Tratta le violazioni in testa e in coda con peso identico; diverge da NDCG.
Listwise	Intero insieme dei candidati per q	Surrogate Loss differenziabili (ListNet, LambdaMART)	Gradienti virtuali λ / Surrogate ranking loss	Non-derivabilità dell'operatore discreto di ordinamento ($\nabla_\theta \text{sort} = \mathbf{0}$).

Tabella 9.1: Quadro sinottico comparativo dei paradigmi di Learning to Rank.

Nel successivo Capitolo 10 l'analisi affronterà la pipeline industriale a due stadi, l'estensione di Okapi BM25 ai documenti semi-strutturati con l'algoritmo BM25F e la sua calibrazione in chiave Learning to Rank tramite discesa di gradiente e perdite surrogate differenziabili (Taylor et al.), nonché i limiti teorici e le problematiche dell'operatore discreto di ordinamento.

Capitolo 10

Learning to Rank II: Pipeline Industriale, BM25F e Formulazione del Ranking

10.1 La Pipeline Completa di Learning to Rank e Vincoli Operativi

L'impiego del Machine Learning nei motori di ricerca su scala Web non sostituisce le strutture dati ad alte prestazioni analizzate nei capitoli precedenti (Capitolo 6 e Capitolo 7), ma si innesta sopra di esse all'interno di un'architettura modulare a stadi [12, 20]. Questa sezione analizza il flusso di elaborazione end-to-end che conduce dalla query dell'utente alla SERP finale, formalizzando i vincoli operativi di throughput e time budget.

10.1.1 Flusso di Elaborazione End-to-End

Come schematizzato nella Figura 10.1, l'infrastruttura di un motore di ricerca moderno articola il recupero dei documenti lungo un percorso a imbuto strettamente sequenziale:

1. **Ricezione ed Espansione della Query:** L'interrogazione dell'utente q viene intercettata, normalizzata e arricchita tramite correzione ortografica, espansione sinonimica contestuale e identificazione delle entità nominate (come analizzato nella Sezione 6.5).

2. **Primo Stadio (Candidate Retrieval / Base Ranker):** La query riscritta interroga l'indice invertito compresso distribuito su molteplici shard. Utilizzando algoritmi di dynamic pruning (WAND o MaxScore,

formalizzati nel Capitolo 8) su una funzione di scoring additiva economica (tipicamente Okapi BM25), il sistema estrae un insieme ristretto di candidati promettenti:

$$\mathcal{C} \subset \mathcal{D}, \quad |\mathcal{C}| \approx 1\,000\text{–}2\,000 \text{ documenti.} \tag{10.1}$$

3. **Feature Extraction e Lookup:** Per ciascun documento candidato $d \in \mathcal{C}$, il sistema interroga il *Document Features Repository* (Forward Index e archivi di metadati residenti in RAM o SSD NVMe) estraendo il vettore multidimensionale delle feature:

$$\mathbf{x}_{q,d} = \langle f_1(q,d), f_2(q,d), \ldots, f_D(q,d) \rangle \in \mathbb{R}^D, \quad D \approx 100\text{–}1\,000. \tag{10.2}$$

4. **Secondo Stadio (Top Ranker / Learning to Rank):** Un modello supervisionato complesso (ad esempio un ensemble di alberi di regressione potenziati da gradiente, GBRT/LambdaMART) valuta contemporaneamente tutti i vettori $\mathbf{x}_{q,d}$, predicendo un punteggio di rilevanza calibrato $\hat{s}(q,d)$.

5. **Ordinamento Finale e Visualizzazione:** I candidati vengono ordinati per punteggio decrescente e i primi $k \approx 10\text{–}20$ documenti vengono arricchiti con titoli, URL e snippet contestuali per formare la SERP visualizzata dall'utente.

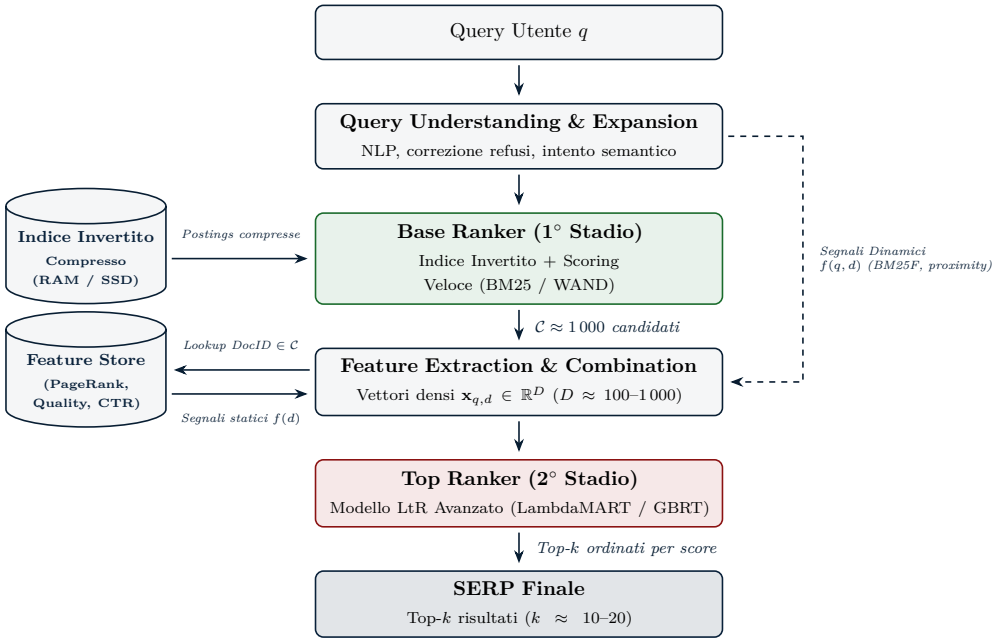


Figura 10.1: Flusso operativo end-to-end di una pipeline di Learning to Rank a due stadi: dal filtraggio scalabile sui miliardi di documenti dell'indice invertito alla graduazione fine con modelli supervisionati complessi [20].

10.1.2 Il Vincolo del Time Budget nei Motori Industriali

Nei motori di ricerca commerciali (Google, Bing, Baidu), l'accordo sul livello di servizio (*Service Level Agreement*, SLA) impone che l'intera transazione computazionale si concluda entro una soglia rigida denominata **Time Budget**, tipicamente fissata tra 50 e 100 ms per l'utente finale:

$$T_{\text{totale}} = T_{\text{network}} + T_{\text{first-stage}} + T_{\text{feature-lookup}} + T_{\text{ltr-inference}} + T_{\text{rendering}} \leq 100 \text{ ms}. \tag{10.3}$$

Tenendo conto delle latenze di rete e dell'estrazione dei candidati, la quota temporale riservata all'inferenza del modello di Machine Learning è vincolata a **non più di** 15–25 **ms**:

- Se $|\mathcal{C}| = 1\,000$ documenti e il modello valuta ciascun candidato sequenzialmente, ogni singola predizione deve richiedere meno di $20\,\mu$s;
- Modelli eccessivamente profondi (come reti neurali basate su Transformer complessi) non possono essere eseguiti in questo stadio su tutti i 1 000 candidati senza provocare il collasso del throughput della macchina;

- Gli ensemble di alberi decisionali (GBRT) rappresentano lo standard industriale indiscusso proprio per la loro capacità di calcolare centinaia di split binari in pochissimi microsecondi mediante codice auto-vettorizzato SIMD e lookup su tabelle precalcolate.

10.2 Apprendimento di un Classificatore per l'Ad-Hoc Search e Formulazione Precisa degli Approcci LtR

Mentre la tassonomia concettuale dei tre paradigmi è stata introdotta nel Capitolo 9 (Sezione 9.6), questa sezione affronta innanzitutto come sia possibile apprendere un classificatore supervisionato per stimare la rilevanza di una coppia query-documento nella ricerca ad-hoc, evidenziandone i limiti strutturali e formalizzando la meccanica computazionale rigorosa delle funzioni di costo (*Loss Machinery*) dei tre approcci fondazionali di Learning to Rank [20].

10.2.1 Apprendere un Classificatore per la Rilevanza di Coppie Query-Documento

Il punto di partenza naturale per integrare il Machine Learning nell'Information Retrieval consiste nel riformulare la ricerca ad-hoc come un problema di **classificazione supervisionata standard** [1, 20].

Rappresentazione e Modello di Predizione. Data una query q e un documento d, il sistema estrae un vettore denso di feature congiunte $\mathbf{x} = \Phi(q, d) \in \mathbb{R}^D$, contenente indicatori lessicali (BM25, tf-idf), attributi di prossimità testuale, metriche statiche di qualità (PageRank, lunghezza del testo) e segnali comportamentali (CTR storico). Se la rilevanza è binaria ($y \in \{0, 1\}$), è possibile addestrare un classificatore probabilistico (es. regressione logistica) per stimare la probabilità a posteriori di pertinenza:

$$P(R = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T \mathbf{x} + b)}}, \tag{10.4}$$

dove i pesi $\mathbf{w} \in \mathbb{R}^D$ e il bias $b \in \mathbb{R}$ vengono ottimizzati minimizzando la log-loss (o binary cross-entropy) su un training set etichettato. Nella fase di inferenza online per una query ad-hoc, il motore calcola lo score

continuo $s(q, d) = \mathbf{w}^T\mathbf{x} + b$ (oppure la probabilità $P(R = 1 \mid \mathbf{x})$) per ciascun candidato estratto dal primo stadio, ordinando la SERP in ordine decrescente di punteggio.

Vantaggi e Limiti dell'Approccio a Classificatore Standard. L'impiego di un classificatore convenzionale presenta vantaggi immediati ma si scontra con tre severe criticità intrinseche:

- **Vantaggi:**
  1. Consente di utilizzare algoritmi e librerie di machine learning off-the-shelf ampiamente consolidate;
  2. Fonde automaticamente segnali eterogenei e a scale differenti in un unico punteggio scalare continuo.
- **Limiti Strutturali nel Ranking:**
  1. *Sbilanciamento Estremo delle Classi (Severe Class Imbalance):* In una collezione Web, per qualunque query il numero di documenti non rilevanti supera di molti ordini di grandezza i documenti pertinenti ($|D_{\text{irrilevanti}}| \gg |D_{\text{rilevanti}}|$). Un classificatore orientato all'accuratezza globale tenderà a predire sistematicamente la classe negativa, penalizzando la sensibilità sul recupero.
  2. *Simmetria dell'Errore di Classificazione:* La perdita logistica o quadratica penalizza l'errore di classificazione in modo identico, indipendentemente dalla posizione del documento nella graduatoria. In IR, al contrario, commettere un errore di ordinamento sulla prima posizione (rank 1) distrugge l'esperienza utente, mentre uno scambio alla posizione 500 è completamente ininfluente.
  3. *Assenza di Relazione di Competizione Locale:* Il classificatore valuta ogni coppia (q, d) in isolamento, ignorando che l'obiettivo dell'utente non è stimare un valore assoluto ma identificare il *migliore* documento tra i concorrenti disponibili per quella specifica query.

Per superare tali limitazioni, la disciplina del Learning to Rank ha elaborato formulazioni matematiche dedicate, capaci di modellare esplicitamente le relazioni di coppia o l'intera permutazione della graduatoria.

10.2.2 Spazio delle Istanze, Scoring e Permutazioni

Sia $\mathcal{Q}$ l'insieme delle possibili query e $\mathcal{D}$ l'insieme dei documenti. Un dataset di addestramento supervisionato per il ranking è composto da una collezione di M tuple:

$$\mathcal{T} = \Big\{(q_i, \mathbf{D}_i, \mathbf{y}_i)\Big\}_{i=1}^{M}, \tag{10.5}$$

dove per ciascuna query q_i:

- $\mathbf{D}_i = [d_{i,1}, d_{i,2}, \ldots, d_{i,n_i}]$ è la lista di n_i documenti candidati associati a q_i;
- Ciascuna coppia $(q_i, d_{i,j})$ è rappresentata da un vettore di feature $\mathbf{x}_{i,j} = \Phi(q_i, d_{i,j}) \in \mathbb{R}^D$;
- $\mathbf{y}_i = [y_{i,1}, y_{i,2}, \ldots, y_{i,n_i}]$ rappresenta il vettore delle etichette di rilevanza fornite da valutatori umani o ricavate implicitamente dai log di clic ($y_{i,j} \in \{0, 1, \ldots, L-1\}$).

Il modello di ranking $R : \mathbb{R}^D \to \mathbb{R}$ assegna a ciascuna coppia un punteggio continuo $s_{i,j} = R(\mathbf{x}_{i,j})$. I punteggi predetti inducono un ordinamento (*permutazione*) $\pi_i \in \mathfrak{S}_{n_i}$ sui documenti:

$$\pi_i(j) < \pi_i(k) \iff R(\mathbf{x}_{i,j}) > R(\mathbf{x}_{i,k}). \tag{10.6}$$

10.2.3 Obiettivo Pointwise: Regressione e Classificazione

Come evidenziato nella tassonomia comparativa di Figura 9.6, l'approccio **Pointwise** ignora la struttura relazionale dell'elenco, trattando ogni singola coppia $(q_i, d_{i,j})$ come un'istanza statistica isolata e indipendente.

Formulazione di Regressione: Se le etichette sono considerate valori numerici continui, la funzione di perdita minimizza l'errore quadratico medio (*Mean Squared Error*, MSE):

$$\mathcal{L}_{\text{pointwise-reg}}(R) = \frac{1}{\sum n_i} \sum_{i=1}^{M} \sum_{j=1}^{n_i} \Big(R(\mathbf{x}_{i,j}) - y_{i,j}\Big)^2. \tag{10.7}$$

Formulazione di Classificazione Multi-Classe: Se le etichette sono categorie discrete ($y \in \{0, 1, 2, 3, 4\}$), il modello modella le probabilità $P(y = c \mid \mathbf{x})$ tramite softmax e minimizza la cross-entropia:

$$\mathcal{L}_{\text{pointwise-class}}(R) = -\sum_{i=1}^{M} \sum_{j=1}^{n_i} \sum_{c=0}^{L-1} \mathbb{I}(y_{i,j} = c) \cdot \log P(y_{i,j} = c \mid \mathbf{x}_{i,j}). \tag{10.8}$$

Limite intrinseco: Il modello penalizza allo stesso modo l'errore di predizione su un documento altamente rilevante (in testa) e su un documento totalmente irrilevante (in fondo alla coda), ignorando la qualità posizionale della graduatoria.

10.2.4 Obiettivo Pairwise: Ordinamento di Coppia e Modello RankNet

L'approccio **Pairwise** sposta l'unità elementare di apprendimento dal singolo documento alla coppia ordinata di documenti per la medesima query: (d_j, d_k) tale che $y_{i,j} > y_{i,k}$ (notazione di preferenza: $d_j \succ_{q_i} d_k$).

Come introdotto nel modello fondazionale **RankNet** [23], la probabilità modellata che il documento d_j sia preferito al documento d_k viene definita tramite la funzione sigmoide applicata alla differenza dei loro punteggi:

$$P(d_j \succ d_k) = \sigma\Big(s_j - s_k\Big) = \frac{1}{1 + e^{-\sigma(s_j - s_k)}} = \frac{1}{1 + e^{-\sigma(R(\mathbf{x}_j) - R(\mathbf{x}_k))}}, \tag{10.9}$$

dove $\sigma > 0$ modula la pendenza della curva logistica. La funzione di perdita per la coppia è la cross-entropia rispetto alla preferenza ideale determinata dalle label ($P^* = 1$ se $y_j > y_k$):

$$\mathcal{C}_{jk} = -P^* \log P(d_j \succ d_k) - (1 - P^*) \log(1 - P(d_j \succ d_k)) = \log\Big(1 + e^{-\sigma(s_j - s_k)}\Big). \tag{10.10}$$

La funzione di costo complessiva è la somma su tutte le coppie discordanti:

$$\mathcal{L}_{\text{pairwise}}(R) = \sum_{i=1}^{M} \sum_{j,k : y_{i,j} > y_{i,k}} \log\Big(1 + e^{-\sigma(R(\mathbf{x}_{i,j}) - R(\mathbf{x}_{i,k}))}\Big). \tag{10.11}$$

Poiché la funzione (10.10) è liscia e differenziabile ovunque rispetto ai parametri del modello, il gradiente può essere calcolato analiticamente e propagato tramite Stochastic Gradient Descent (SGD).

10.2.5 Obiettivo Listwise: ListNet e Distribuzioni di Probabilità

L'approccio **Listwise** mira a valutare l'intero elenco di documenti simultaneamente. Poiché l'operatore di ordinamento discreto non è differenziabile, i metodi listwise probabilistici come **ListNet** [24] proiettano i punteggi e le etichette su distribuzioni di probabilità multinomiali.

Per una query con n documenti:

- La **probabilità predetta di top-1** per il documento j (la probabilità che d_j sia il migliore della lista) è data dal softmax dei punteggi:

$$P_s(d_j) = \frac{\exp(s_j)}{\sum_{k=1}^{n} \exp(s_k)}; \tag{10.12}$$

- La **probabilità ideale di top-1** viene calcolata analogamente sulle etichette di rilevanza (tipicamente trasformate in guadagni esponenziali $2^{y_j} - 1$):

$$P_y(d_j) = \frac{\exp(y_j)}{\sum_{k=1}^{n} \exp(y_k)}. \tag{10.13}$$

La funzione di perdita di ListNet misura la distanza di Kullback-Leibler (o cross-entropia) tra la distribuzione ideale e quella predetta:

$$\mathcal{L}_{\text{ListNet}}(R) = -\sum_{j=1}^{n} P_y(d_j) \cdot \log P_s(d_j). \tag{10.14}$$

ListNet garantisce una formulazione continua e derivabile che cattura le interazioni sull'intera lista, favorendo i documenti con etichette elevate rispetto a quelli con etichette basse.

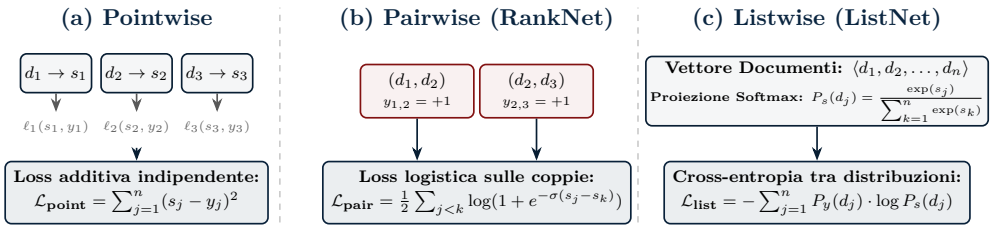


Figura 10.2: Confronto schematico tra le unità di calcolo e la propagazione delle funzioni di costo dei tre paradigmi di ranking (Pointwise, Pairwise e Listwise), a complemento della visione tassonomica della Figura 9.6 [20].

10.3 Estensione a BM25F: Architettura, Ottimizzazione e Loss Surrogate

La formulazione standard di Okapi BM25 analizzata nel Capitolo 9 assume che ciascun documento sia un testo continuo e omogeneo. Nei sistemi reali (come pagine Web, articoli scientifici o schede di e-commerce), i documenti presentano una ricca struttura a zone o campi eterogenei. Questa sezione esamina l'algoritmo **BM25F** (*BM25 with Fields*), ne analizza i vantaggi e gli svantaggi strutturali, e mostra come la calibrazione dei suoi parametri venga formulata e risolta come un problema di Learning to Rank mediante discesa di gradiente e perdite surrogate [18, 22].

10.3.1 Dall'Okapi BM25 ai Documenti Semi-Strutturati: L'Algoritmo BM25F

Nei documenti complessi, il contenuto testuale è ripartito in molteplici campi o flussi semi-strutturati $s \in \mathcal{S}$:

- Pagine Web: $\mathcal{S} = \{\texttt{Title}, \texttt{Body}, \texttt{Anchor Text}, \texttt{URL}\}$;
- Articoli scientifici: $\mathcal{S} = \{\texttt{Titolo}, \texttt{Abstract}, \texttt{Corpo}, \texttt{Autori}\}$.

Perché l'approccio ingenuo fallisce. Un tentativo immediato per gestire campi multipli consisterebbe nel calcolare lo score Okapi BM25 standard separatamente su ciascun campo s e combinare linearmente i punteggi parziali ottenuti:

$$s_{\text{ingenuo}}(q, d) = \sum_{s \in \mathcal{S}} w_s \cdot \text{BM25}_s(q, d). \tag{10.15}$$

Tale combinazione soffre di un difetto intrinseco: applica la saturazione non lineare del termine indipendentemente su ogni singolo campo. Di conseguenza, se una parola chiave compare due volte nel titolo e due volte nel corpo, essa subisce due saturazioni parziali separate, anziché saturare una volta sola in base all'evidenza complessiva ponderata del termine nel documento.

L'architettura algebrica corretta di BM25F. Sviluppato da Robertson, Zaragoza e Taylor [18, 22], l'algoritmo **BM25F** adotta l'impostazione corretta: **combina linearmente le frequenze ponderate dei campi prima di applicare la trasformazione di saturazione asintotica**.

Sia $f_{t,s}$ la frequenza del termine t nel campo s del documento d. Si definiscono:

- $w_s > 0$: il peso d'importanza assegnato al campo s;
- $l_s = |d_s|$: la lunghezza effettiva (in token) del campo s nel documento d;
- $L_s = \text{avgdl}_s$: la lunghezza media del campo s sull'intera collezione;
- $b_s \in [0, 1]$: il parametro di normalizzazione della lunghezza specifico per il campo s.

La lunghezza normalizzata specifica per il campo s è:

$$B_s = (1 - b_s) + b_s \cdot \frac{l_s}{L_s}. \tag{10.16}$$

La **pseudo-frequenza effettiva ponderata** del termine t nel documento d, denotata con $\tilde{f}_t(d)$, è la combinazione convessa:

$$\tilde{f}_t(d) = \sum_{s \in \mathcal{S}} w_s \cdot \frac{f_{t,s}}{B_s} = \sum_{s \in \mathcal{S}} \frac{w_s \cdot f_{t,s}}{(1 - b_s) + b_s \cdot \frac{l_s}{L_s}}. \tag{10.17}$$

Il punteggio globale BM25F della coppia (q, d) applica infine la saturazione non lineare governata dal parametro globale k_1:

$$s_{\text{BM25F}}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\tilde{f}_t(d)}{k_1 + \tilde{f}_t(d)}, \tag{10.18}$$

dove l'IDF probabilistico è convenzionalmente calcolato come:

$$\text{IDF}(t) = \ln\left(\frac{N - df_t + 0.5}{df_t + 0.5} + 1\right). \tag{10.19}$$

10.3.2 Vantaggi e Svantaggi di BM25F (Pros and Cons)

L'estensione BM25F introduce un bilanciamento analitico ben preciso:

Vantaggi (Pros):

1. **Saturazione Asintotica Corretta:** Sommando le frequenze dei singoli campi all'interno di $\tilde{f}_t(d)$ prima della funzione concava sublineare, il termine satura una sola volta. Ciò rispecchia fedelmente il rendimento marginale decrescente dell'evidenza complessiva.
2. **Ponderazione Flessibile delle Zone:** Consente di assegnare pesi w_s dedicati a zone a densità semantica diseguale (es. occorrenze nel titolo o nell'anchor text conferiscono un segnale di rilevanza assai più affidabile rispetto al corpo).

3. **Controllo Differenziato della Verbosità (b_s):** Permette di calibrare indipendentemente la penalizzazione della lunghezza per ciascuna zona. Ad esempio, per il titolo si adotta spesso $b_{\text{title}} \approx 0$ (poiché i titoli lunghi non diluiscono il contenuto ma sono descrittivi), mentre per il testo esteso si sceglie $b_{\text{body}} \approx 0.75$.

4. **Robustezza e Ruolo nel Candidate Retrieval:** Costituisce lo standard industriale de facto per il reperimento di primo stadio ad altissimo throughput nei motori di ricerca commerciali.

Svantaggi (Cons):

1. **Esplosione dei Parametri Liberi:** Mentre Okapi BM25 possiede soltanto 2 parametri liberi scalari ($\langle k_1, b \rangle$), BM25F richiede di configurare $2|\mathcal{S}| + 1$ grandezze continue:

$$\vec{\theta} = \langle w_1, \ldots, w_S, b_1, \ldots, b_S, k_1 \rangle. \tag{10.20}$$

   Con $S = 4$ campi standard, vi sono ben 9 iperparametri interdipendenti.

2. **Interdipendenza Non Lineare:** Poiché i pesi w_s e i fattori di lunghezza b_s interagiscono entro la sommatoria prima della saturazione governata da k_1, variare il peso di un campo altera la dinamica di saturazione di tutti gli altri. Non è possibile ottimizzare i parametri in modo scorrelato o manuale.

3. **Overhead di Indicizzazione ed Estrazione:** L'indice invertito deve memorizzare e scorrere frequenze e lunghezze suddivise per campo, incrementando l'ingombro in memoria e il tempo di lookup rispetto a posting list monocampo.

10.3.3 Inquadramento dell'Ottimizzazione di BM25F in un Problema di Learning to Rank

Di fronte a $2S + 1$ iperparametri continui interdipendenti, la calibrazione di BM25F cessa di essere una regolazione euristica manuale e viene formalizzata rigorosamente come un problema di **Learning to Rank (LtR)** [22]:

- **Spazio delle Ipotesi:** La famiglia parametrica di tutte le possibili funzioni di ranking BM25F indicizzate dal vettore dei pesi: $\mathcal{H} = \{\text{BM25F}_{\vec{\theta}} \mid \vec{\theta} \in \mathbb{R}_+^{2S+1}\}$;

- **Dati di Addestramento:** Una collezione supervisionata di query storiche $\mathcal{Q}_{\text{train}}$, corredata da insiemi di documenti con giudizi di rilevanza multilivello (ground truth, es. scale a 5 punti da 0 a 4);
- **Obiettivo di Ottimizzazione:** Individuare la configurazione ottima $\vec{\theta}^*$ che massimizza la metrica di posizionamento target sull'ordinamento prodotto (tipicamente NDCG@10):

$$\vec{\theta}^* = \arg\max_{\vec{\theta}} \text{NDCG@10}\Big(\text{sort}(\text{BM25F}_{\vec{\theta}})\Big). \tag{10.21}$$

Vantaggi e Svantaggi dell'Approccio LTR per BM25F (Pros and Cons):

- **Vantaggi (Pros):**
  1. *Automazione Oggettiva Guidata dai Dati:* Sostituisce l'arbitrio umano con un'ottimizzazione empirica fondata sulle query effettive e sul comportamento reale degli utenti.
  2. *Navigazione in Spazi ad Alta Dimensionalità:* Individua combinazioni ottimali di parametri che l'intuito umano non potrebbe mai calibrare a causa delle retroazioni non lineari tra w_s, b_s e k_1.
  3. *Adattabilità a Domini Specifici:* Consente di ricalibrare istantaneamente il motore per collezioni verticali (e-commerce, brevetti, letteratura medica) semplicemente fornendo nuovi log o valutazioni di dominio.
- **Svantaggi (Cons):**
  1. *Rischio di Sovradattamento (Overfitting):* Uno spazio a 9 o più dimensioni rischia di specializzarsi sulle peculiarità di un set ristretto di query di training, degradando su query non viste.
  2. *Costo dei Dati di Rilevanza:* Richiede la disponibilità di collezioni annotate con etichette graduate accurate, la cui generazione manuale o inferenza da click log comporta notevoli costi o rumore statistico.
  3. *Non-Derivabilità della Metrica Target:* L'NDCG non è differenziabile, impedendo l'applicazione diretta della discesa di gradiente e imponendo l'uso di perdite surrogate.

10.3.4 Calibrazione dei Parametri: Grid Search vs. Discesa di Gradiente

Per individuare $\vec{\theta}^*$, il confronto tra le strategie di ricerca pone in evidenza l'insostenibilità della ricerca esaustiva al crescere del numero di campi:

Caratteristica	**Grid Search (Ricerca a Griglia)**	**Gradient Descent (Discesa di Gradiente)**
Complessità	Esponenziale: $\mathcal{O}(G^{2S+1})$, dove G è il numero di campioni per parametro.	Polinomiale / Lineare rispetto al numero di iterazioni di update: $\mathcal{O}(I \cdot \lvert\mathcal{T}\rvert \cdot (2S+1))$.
Fattibilità ($S = 4$)	**Impossibile:** testando soli 10 valori per parametro su 9 variabili occorrono 10^9 valutazioni dell'intero corpus.	**Fattibile ed efficiente:** converge al minimo locale in poche centinaia di iterazioni (pochi minuti di CPU).
Requisito Funzione	Funziona su qualunque funzione scatola nera non differenziabile.	Richiede una funzione di perdita continua e differenziabile ovunque ($\nabla_{\vec{\theta}}\mathcal{L}$).
Precisione	Vincolata alla grana discreta del campionamento prefissato.	Continua: individua valori decimali ottimali continui nello spazio $\mathbb{R}_+^{2S+1}$.

Tabella 10.1: Confronto analitico tra Grid Search e Gradient Descent per l'ottimizzazione degli iperparametri di BM25F.

10.3.5 La Scelta della Funzione di Perdita Surrogata (RankNet)

Sebbene la discesa di gradiente sia enormemente più efficiente della ricerca a griglia, essa non può essere applicata direttamente alla metrica primaria NDCG@10:

- La funzione NDCG@10 dipende dalla posizione discreta $1, 2, \ldots, 10$ assegnata ai documenti dall'operatore di ordinamento sort();
- Variazioni infinitesimali di $\vec{\theta}$ non modificano l'ordinamento relativo e lasciano invariato l'NDCG, per cui il gradiente è quasi ovunque nullo ($\nabla_{\vec{\theta}}\text{NDCG} = \mathbf{0}$);

- Quando la variazione di $\vec{\theta}$ scambia due posizioni, il valore di NDCG compie un salto discontinuo: il gradiente è indefinito.

Per superare tale ostacolo fondamentale, Taylor et al. [22] hanno introdotto l'ottimizzazione di BM25F guidata da una **funzione di perdita surrogata pairwise differenziabile**: la loss logistica di RankNet (definita nell'Equazione (10.10)):

$$\vec{\theta}^{(t+1)} \leftarrow \vec{\theta}^{(t)} - \eta \cdot \nabla_{\vec{\theta}} \mathcal{L}_{\text{RankNet}}(\vec{\theta}^{(t)}), \tag{10.22}$$

dove i gradienti analitici rispetto a ciascun peso di campo $\frac{\partial \text{BM25F}}{\partial w_s}$, fattore di lunghezza $\frac{\partial \text{BM25F}}{\partial b_s}$ e saturazione $\frac{\partial \text{BM25F}}{\partial k_1}$ vengono calcolati tramite la regola di derivazione a catena (*chain rule*). In questo modo, l'ottimizzazione converge in tempi lineari verso la configurazione che massimizza la concordanza di rango sulle coppie.

10.4 I Problemi Fondamentali dell'Apprendimento del Ranking

L'analisi dell'ottimizzazione di BM25F solleva una serie di problematiche teoriche intrinseche che distinguono nettamente l'Information Retrieval dai classici compiti di regressione e classificazione dell'apprendimento automatico standard.

10.4.1 La Natura Discreta delle Metriche di Graduatoria

In un problema convenzionale di Machine Learning (come la predizione del prezzo di una casa o la stima del tempo di transito), l'obiettivo è la minimizzazione della deviazione numerica continua: l'errore $y - \hat{y}$ è liscio e convesso. Nel ranking, invece, **la grandezza numerica assoluta dello score predetto non ha alcuna importanza diretta**:

- Se i punteggi predetti per due documenti passano da $\{s_1 = 10, s_2 = 8\}$ a $\{s_1 = 1000, s_2 = 0.1\}$, la graduatoria finale rimane rigorosamente $[d_1, d_2]$ e l'NDCG non subisce alcuna alterazione;
- Le metriche di riferimento (MAP, MRR, NDCG@k) sono funzioni a gradino (*step functions*): piatte ovunque eccetto nei punti di transizione dove due documenti si scambiano di posto.

10.4.2 Non Convessità e Non Derivabilità dell'Operatore di Ordinamento

L'operatore matematico che trasforma il vettore dei punteggi continui $\mathbf{s} \in \mathbb{R}^n$ nella permutazione discreta $\pi = \text{sort}(\mathbf{s})$ è una funzione non lineare non convessa a variazione non continua:

$$\nabla_{\mathbf{s}} \text{sort}(\mathbf{s}) = \mathbf{0} \quad \text{quasi ovunque.} \tag{10.23}$$

La conseguenza teorica è drammatica:

Teorema 10.4.1 (Intrattabilità dell'Ottimizzazione Diretta di Metriche di Ranking)**.** *L'ottimizzazione diretta delle metriche di ranking non continue (quali MAP e NDCG) è un problema computazionale* ***NP-difficile*** *[20]. Non esiste alcun algoritmo di discesa di gradiente standard in grado di operare direttamente su tali obiettivi senza introdurre approssimazioni o surrogati lisci.*

Di fronte a tale barriera, la comunità scientifica ricorre alla progettazione di funzioni di perdita lisce surrogate differenziabili (come gli approcci Pointwise, Pairwise e Listwise formalizzati in precedenza).

Bibliografia

[1] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[2] Stefan Büttcher, Charles L. A. Clarke, and Gordon V. Cormack. *Information Retrieval: Implementing and Evaluating Search Engines*. MIT Press, 2010.

[3] Alistair Moffat and Luke Stuiver. Binary interpolative coding for effective index compression. *Information Processing & Management*, 36(5):667–687, 2000.

[4] Christopher J. C. Burges. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research, 2010.

[5] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.

[6] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for information retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.

[7] Cyril W. Cleverdon. The Cranfield tests on index language devices. *Aslib Proceedings*, 19(6):173–194, 1967.

[8] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of retrieval techniques. *ACM Transactions on Information Systems (TOIS)*, 20(4):422–446, 2002.

[9] Denis Bakhvalov. *CPU Microarchitecture: Performance Analysis and Tuning on Modern CPUs*. BookBaby, 2nd edition, 2024.

[10] Sergey Slotin. *Algorithms for Modern Hardware*. Algorithmica, 2023.

[11] Steven Bird, Ewan Klein, and Edward Loper. *Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit*. OŔeilly Media, 2009.

[12] Nicola Tonellotto, Craig Macdonald, and Iadh Ounis. Efficient query processing for scalable web search. *Foundations and Trends in Information Retrieval*, 12(4–5):319–500, 2018.

[13] Andrei Z. Broder, David Carmel, Michael Herscovici, Aya Soffer, and Jason Y. Zien. Efficient query evaluation using a two-level retrieval process. In *Proceedings of the 12th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 426–434, 2003.

[14] Howard Turtle and James Flood. Query evaluation: Strategies and optimizations. *Information Processing & Management*, 31(6):831–850, 1995.

[15] Shuai Ding and Torsten Suel. Faster top-k document retrieval using block-max indexes. In *Proceedings of the 4th ACM International Conference on Web Search and Web Data Mining (WSDM)*, pages 193–202, 2011.

[16] Antonio Mallia, Giuseppe Ottaviano, Elia Porciani, Nicola Tonellotto, and Rossano Venturini. Faster block-max WAND with variable-sized blocks. In *Proceedings of the 40th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 625–634, 2017.

[17] Adrien Grand. MAXSCORE & block-max MAXSCORE: More skipping with block-max MAXSCORE. Elasticsearch Labs Technical Blog, 2023.

[18] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[19] Jiafeng Guo, Yixing Fan, Liang Pang, Liu Yang, Qingyao Ai, Hamed Zamani, Chenyan Wu, W. Bruce Croft, and Xueqi Cheng. A deep look into neural ranking models for information retrieval. *Information Processing & Management*, 57(6):102067, 2020.

[20] Sebastian Bruch, Claudio Lucchese, and Franco Maria Nardini. Efficient and effective tree-based and neural learning to rank. *Foundations and Trends in Information Retrieval*, 17(2–3):124–334, 2023.

[21] Tie-Yan Liu. Learning to rank for information retrieval. *Foundations and Trends in Information Retrieval*, 3(3):225–331, 2009.

[22] Michael Taylor, Hugo Zaragoza, Nick Craswell, Stephen Robertson, and Chris Burges. Optimisation methods for ranking functions with multiple parameters. In *Proceedings of the 15th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 585–593, 2006.

[23] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. Learning to rank using gradient descent. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 89–96, 2005.

[24] Zhe Cao, Tao Qin, Tie-Yan Liu, Ming-Feng Tsai, and Hang Li. Learning to rank: from pairwise approach to listwise approach. In *Proceedings of the 24th International Conference on Machine Learning (ICML)*, pages 129–136, 2007.